

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS – CETAM
ESCOLA DE EDUCAÇÃO PROFISSIONAL – GALILÉIA
CURSO TÉCNICO DE NÍVEL MÉDIO EM DESENVOLVIMENTO DE SISTEMAS

**(SISFROTA – SISTEMA DESKTOP DE CONTROLE DE FROTA E MANUTENÇÃO
DE EMBARCAÇÕES)**

LETÍCIA ALVES SALVADOR
MAGAYVER KAON DO NASCIMENTO SILVA
VITÓRIA MONTEIRO DE CARVALHO
YOSEF KLINSMAN MOREIRA CRUZ

MANAUS/AM

2026

Ambiente de Aprendizagem onde foi realizada a Prática Profissional Supervisionada:

Laboratório de Informática - 02

Endereço do local onde foi realizada a Prática Profissional Supervisionada

Rua Marginal Esquerda, nº 28, bairro Galileia, em Manaus - (CEP 69093-780)

Turno: Matutino

Período de realização: 18/08/2026 a 08/09/2026

Relatório Técnico de Prática Profissional Supervisionada, apresentado como requisito parcial para aprovação no componente curricular/Curso Técnico de Nível Médio em Desenvolvimento de Sistemas.

Nota Final da Prática Profissional Supervisionada da Etapa 01:

(____)_____.

Considerações do Instrutor da Prática Profissional Supervisionada:

Instrutora Priscila Leylianne da Silva Gonçalves

CENTRO DE EDUCAÇÃO TECNOLÓGICA DO AMAZONAS – CETAM
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Avalia Final do Projeto Técnico:

A Avaliação do projeto intitulado (SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações),

desenvolvido pelos estudantes: Letícia Alves Salvador, Vitória Monteiro e Yosef Klinsman Moreira Cruz, está composta de duas partes, sendo:

*Parte 1 - Avaliação do acompanhamento sistemático realizado no decorrer do desenvolvimento do projeto, obtendo a nota:

() _____

*Parte 2 – Entrega do trabalho impresso e socialização da temática com a turma:

() _____

Nota final nesta Etapa II do Componente (Projeto):

() _____.

Considerações do Professor/Instrutor:

.....

Priscila Leylianne da Silva Gonçalves
Instrutor do Projeto(Avaliador)

MANAUS – AM

2026

LISTA DE FIGURAS

Figura 1 – Modelo Conceitual do Banco de Dados(DER)	25
Figura 2 - Modelo Lógico do Banco de Dados	26
Figura 3 - Diagrama de Casos de Uso	29
Figura 4 - Diagrama de Classes do Sistema	33
Figura 5 - Diagrama de Atividades dos Processos.....	34
Figura 6 - Tela de Login	45
Figura 7 - Tela de Usuário - Administrador	46
Figura 8 - Tela de Usuário - Operador/Despachante	46
Figura 9 - Tela de Usuário - Responsável Técnico	47

LISTA DE TABELAS

Tabela 1 - Fases de Execução do Projeto	20
Tabela 2 - Requisitos Funcionais	22
Tabela 3 - Requisitos Não Funcionais(RNF)	23
Tabela 4 - Regras de Negócio(RN)	24
Tabela 5 - Relacionamento entre Casos de Uso e Atores	29
Tabela 6 - Especificação do Caso de Uso: Ator Administrador	31
Tabela 7 - Especificação do Caso de Uso: Ator Operador/Despachante	32
Tabela 8 - Especificação do Caso de Uso: Ator Responsável Técnico	32
Tabela 9 - Dicionário de Dados: Entidade Usuários	34
Tabela 10 - Dicionário de Dados: Entidade Embarcações	36
Tabela 11 - Dicionário de Dados: Entidade Tripulantes	37
Tabela 12 - Dicionário de Dados: Entidade Documentos da Embarcação	38
Tabela 13 - Dicionário de Dados: Entidade Viagens	39
Tabela 14 - Dicionário de Dados: Entidade Manutenções	40
Tabela 15 - Dicionário de Dados: Entidade Abastecimentos	42
Tabela 16 - Dicionário de Dados: Entidade Incidentes	43
Tabela 17 - Orçamento	54
Tabela 18 - Cronograma	55

SUMÁRIO

1 APRESENTAÇÃO.....	9
2 CARACTERIZAÇÃO DO PROBLEMA E JUSTIFICATIVA TÉCNICA.....	12
3 ÁREA DE ABRANGÊNCIA.....	16
4 OBJETIVOS	17
4.1. Objetivo Geral	17
4.2. Objetivos Específicos.....	17
5 METAS.....	18
6 FASES DE EXECUÇÃO	20
6.1 Entrevista (questionário Google Forms)	20
6.2 Requisitos Funcionais(RF)	22
6.3 Requisitos Não Funcionais(RNF).....	23
6.4 Regras de Negócio(RN).....	24
6.5 Modelagem (Banco de Dados).....	25
7 METODOLOGIA.....	29
7.1 Diagrama de Caso de Uso	29
7.2 Diagrama de Classes	33
7.3 Diagrama de Atividades.....	34
7.4 Dicionário de Dados	34
7.5 Uso de Ferramentas de Inteligência Artificial.....	44
7.6 Protótipo do Sistema SISFROTA(Telas).....	45
8 DESCRIÇÃO DAS ATIVIDADES PRÁTICAS NA ETAPA.....	47
8.1 brModelo	47
8.2 Astah.....	48
8.3 Trello	49
8.4 MySQL Workbench	49
8.5 GitHub.....	50
9 ÓRGÃOS ENVOLVIDOS.....	51
10 MECANISMOS E NORMAS DE EXECUÇÃO	52
10.1 Convênios e Parcerias Institucionais.....	52
10.2 Regulamentações Náuticas e Órgãos Normatizadores	53
10.3 Normas Técnicas e Padrões de Desenvolvimento de Software	53
11 ORÇAMENTO	54
12 CRONOGRAMA.....	55
13 CONSIDERAÇÕES FINAIS	56
14 REFERÊNCIAS BIBLIOGRÁFICAS	59

15	ANEXOS	61
15.1	Tela de Login	61
15.2	Tela Dashboard Operador	66
15.3	Tela Dashboard Responsável Técnico.....	78
15.4	Tela Gerenciar Embarcações – Administrador	83
15.5	Tela de Gerenciamento de Tripulações	86
15.6	Tela de Manutenções	90
15.7	Tela de Abastecimento	95
15.8	Tela de Consultas de Horários	97
15.9	Tela de Incidentes	99
15.10	Tela de Viagens.....	101
15.11	Tela de Relatório de Custos	105
15.12	Tela Principal	107
15.13	UsuarioController.java	119
15.14	EmbarcacaoController.java.....	120
15.15	ManutencaoController.java	121
15.16	TripulanteController.java	122
15.17	ViagemController.java	123
15.18	RelatorioController.java	124
15.19	AlertaController.java	125
15.20	AbastecimentoController.java.....	126
15.21	IncidenteController.java.....	126
15.22	ConsultaHorariosController.java	127
15.23	TecnicoController.java	128
15.24	ConexaoDAO.java.....	129
15.25	UsuarioDAO.java.....	129
15.26	EmbarcacaoDAO.java	131
15.27	ManutencaoDAO.java.....	132
15.28	TripulanteDAO.java.....	134
15.29	ViagemDAO.java	135
15.30	AbastecimentoDAO.java	138
15.31	RotaDAO.java	140
15.32	FornecedorDAO.java	140
15.33	ConsultaHorariosDAO.java.....	141
15.34	RelatorioDAO.java.....	142
15.35	TecnicoDAO.java.....	143

15.36 IncidenteDAO.java	146
15.37 Usuário.java	148
15.38 Embarcacao.java.....	149
15.39 Manutencao.java	150
15.40 Tripulante.java	151
15.41 Viagem.java.....	152
15.42 Abastecimento.java.....	153
15.43 Incidente.java	154
15.44 ConsultaHorarioDTO.java.....	155
15.45 CustoConsolidadoDTO.java	156
15.46 EmailService.java	156
15.47 GeradorPDFService.java	157
APÊNDICE A – REGISTRO DE OBSERVAÇÕES DE ATIVIDADE PRÁTICA.....	160

1 APRESENTAÇÃO

O presente Relatório Técnico tem como finalidade apresentar o desenvolvimento do projeto SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações, elaborado como atividade da Prática Profissional Supervisionada do Curso Técnico de Nível Médio em Desenvolvimento de Sistemas. O projeto foi desenvolvido com o objetivo de aplicar, na prática, os conhecimentos adquiridos ao longo do curso, abrangendo áreas como análise de requisitos, modelagem de banco de dados, desenvolvimento de software, documentação técnica e gerenciamento de projetos.

A escolha da temática está relacionada à importância do transporte fluvial para a região amazônica. Em grande parte do Estado do Amazonas, os rios constituem as principais vias de deslocamento de pessoas, mercadorias e suprimentos, tornando as embarcações elementos essenciais para a mobilidade e para o desenvolvimento econômico das comunidades. Nesse cenário, a gestão eficiente das embarcações torna-se indispensável para garantir a segurança das operações, a disponibilidade da frota e a qualidade dos serviços prestados.

Observando essa realidade, identificou-se que muitas organizações ainda realizam o controle de suas embarcações por meio de registros manuais, planilhas eletrônicas ou documentos físicos. Essa prática pode ocasionar dificuldades na organização das informações, atrasos na execução de manutenções preventivas, perda de registros importantes e falhas no acompanhamento da documentação obrigatória. Além disso, a ausência de um sistema informatizado dificulta o acesso rápido às informações necessárias para a tomada de decisões, comprometendo a eficiência da gestão operacional.

Com o intuito de contribuir para a solução desses desafios, foi idealizado o SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações, um sistema desenvolvido para centralizar e organizar informações relacionadas à gestão de frotas náuticas. A proposta consiste em disponibilizar uma ferramenta capaz de auxiliar no gerenciamento das embarcações, da tripulação, das viagens realizadas,

das manutenções preventivas e corretivas, dos abastecimentos, dos incidentes operacionais e da documentação obrigatória das embarcações.

O projeto foi concebido considerando a necessidade de oferecer uma solução prática, intuitiva e eficiente para o gerenciamento das operações fluviais. Para isso, foram definidos requisitos funcionais e não funcionais que possibilitam a realização das principais atividades relacionadas ao controle da frota. Entre as funcionalidades previstas estão o cadastro de embarcações, gerenciamento de tripulantes, registro de viagens, controle de manutenções, monitoramento da documentação obrigatória, emissão de relatórios e geração de alertas relacionados a vencimentos e revisões programadas.

Além das funcionalidades operacionais, o sistema foi projetado para atender diferentes perfis de usuários, permitindo que cada profissional tenha acesso às ferramentas necessárias para desempenhar suas atribuições. Dessa forma, gestores da frota, operadores e responsáveis técnicos pela manutenção podem utilizar o sistema de acordo com suas responsabilidades, contribuindo para uma melhor organização das atividades e para um controle mais eficiente das informações.

O desenvolvimento do SISFROTA também representa uma oportunidade de aplicação dos conhecimentos técnicos adquiridos durante a formação profissional. Durante a elaboração do projeto foram utilizados conceitos relacionados à análise de sistemas, modelagem de dados, levantamento de requisitos, arquitetura de software e banco de dados. Essas atividades permitiram aos integrantes da equipe vivenciar etapas fundamentais do processo de desenvolvimento de sistemas, aproximando os conhecimentos teóricos da prática profissional.

Outro aspecto relevante do projeto é sua contribuição para a modernização dos processos de gestão de frotas. A utilização de recursos informatizados permite reduzir falhas decorrentes de controles manuais, aumentar a confiabilidade dos registros e facilitar o acesso às informações operacionais. Além disso, a centralização dos dados possibilita um melhor acompanhamento das atividades realizadas, auxiliando os gestores na tomada de decisões e no planejamento das ações necessárias para a manutenção e operação das embarcações.

Sob a perspectiva acadêmica, o projeto busca demonstrar a capacidade dos alunos em identificar problemas reais, analisar necessidades específicas e propor soluções tecnológicas adequadas para sua resolução. O desenvolvimento do SISFROTA evidencia a aplicação integrada dos conhecimentos adquiridos ao longo do curso, contemplando desde o levantamento dos requisitos até a definição das funcionalidades necessárias para atender às demandas do contexto estudado.

Portanto, o SISFROTA apresenta-se como uma solução tecnológica voltada ao gerenciamento de frotas e manutenção de embarcações, tendo como principal propósito promover maior controle, organização e segurança das operações fluviais. Espera-se que o sistema contribua para a otimização dos processos de gestão, para a redução de falhas operacionais e para a melhoria da qualidade das informações utilizadas no acompanhamento das atividades relacionadas ao transporte fluvial.

2 CARACTERIZAÇÃO DO PROBLEMA E JUSTIFICATIVA TÉCNICA

O transporte fluvial desempenha um papel fundamental na região amazônica, constituindo-se como um dos principais meios de deslocamento de pessoas, mercadorias e suprimentos entre comunidades ribeirinhas e centros urbanos. Em muitas localidades, os rios representam as principais vias de acesso, tornando as embarcações indispensáveis para a mobilidade da população e para o desenvolvimento das atividades econômicas. Dessa forma, a gestão adequada das embarcações torna-se um fator essencial para garantir a continuidade das operações, a segurança dos passageiros e a eficiência dos serviços prestados.

Nesse cenário, empresas e organizações que operam frotas fluviais precisam manter um controle rigoroso sobre diversos aspectos relacionados às embarcações, incluindo dados cadastrais, documentação obrigatória, histórico de viagens, controle da tripulação, abastecimentos, registros de incidentes e acompanhamento das manutenções preventivas e corretivas. Entretanto, em muitos casos, essas informações ainda são registradas por meio de anotações manuais, planilhas eletrônicas ou documentos físicos, dificultando a organização dos dados e o acompanhamento eficiente das atividades operacionais.

A utilização de métodos manuais para o gerenciamento da frota pode ocasionar diversos problemas, como perda de informações importantes, duplicidade de registros, dificuldades na localização de dados históricos e falhas no controle dos prazos de manutenção e renovação documental. Além disso, a ausência de um sistema informatizado dificulta a geração de relatórios gerenciais e o acompanhamento da situação operacional das embarcações, comprometendo a tomada de decisões por parte dos responsáveis pela administração da frota.

A importância dessa problemática está diretamente relacionada ao impacto que uma gestão inadequada pode causar nas operações de transporte fluvial. Quando uma manutenção preventiva não é realizada dentro do prazo adequado, aumentam-se os riscos de falhas mecânicas e interrupções inesperadas das atividades. Da mesma forma, documentos vencidos ou irregularidades cadastrais podem impedir a operação das embarcações, gerando prejuízos financeiros e comprometendo a prestação dos serviços. Dessa maneira, torna-se evidente a necessidade de

ferramentas que auxiliem no controle e monitoramento das informações relacionadas à frota.

A relevância dessa questão também se estende à comunidade que depende diretamente dos serviços de transporte fluvial. Em diversas regiões do Amazonas, o deslocamento por via fluvial é indispensável para o acesso a serviços de saúde, educação, comércio e outras atividades essenciais. Problemas operacionais decorrentes da falta de controle das embarcações podem afetar a qualidade e a segurança desses serviços, impactando negativamente a população que depende desse meio de transporte para realizar suas atividades cotidianas.

Diante desse contexto, observa-se a crescente utilização de sistemas informatizados voltados para a gestão de frotas em diferentes segmentos do transporte. Soluções tecnológicas têm sido desenvolvidas com o objetivo de auxiliar organizações no controle de veículos, manutenção, documentação e operações logísticas. No setor náutico, sistemas de gerenciamento de embarcações vêm sendo utilizados para melhorar a organização das informações e otimizar os processos relacionados à manutenção e operação das frotas. O projeto SISFROTA insere-se nessa temática ao propor uma solução direcionada ao gerenciamento operacional e técnico de embarcações, atendendo às necessidades específicas do transporte fluvial.

O SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações foi concebido para centralizar as informações relacionadas às embarcações e disponibilizar recursos que auxiliem no gerenciamento das atividades operacionais. O sistema tem como objetivo permitir o cadastro e gerenciamento de embarcações, tripulantes, viagens, abastecimentos, incidentes, manutenções preventivas e corretivas, além do controle da documentação obrigatória. A aplicação também disponibiliza mecanismos para emissão de relatórios e acompanhamento do histórico de operações realizadas pela frota.

Outro aspecto importante do projeto é a definição de perfis específicos de usuários, permitindo que diferentes profissionais tenham acesso às funcionalidades relacionadas às suas responsabilidades. O sistema contempla perfis como Administrador da Frota, Operador ou Despachante e Responsável Técnico pela Manutenção. Essa divisão possibilita uma melhor organização das atividades,

garantindo que cada usuário tenha acesso apenas aos recursos necessários para o desempenho de suas funções.

A proposta do SISFROTA apresenta relação com outras soluções de gestão utilizadas em diferentes áreas, especialmente aquelas voltadas ao controle de ativos, manutenção preventiva e gerenciamento operacional. Entretanto, o diferencial do projeto está na adaptação das funcionalidades às necessidades específicas das operações fluviais, considerando características próprias do transporte por embarcações, como controle de documentação náutica, monitoramento de manutenções, registro de viagens e acompanhamento da tripulação.

A justificativa técnica para o desenvolvimento do SISFROTA fundamenta-se na necessidade de automatizar processos que atualmente são realizados manualmente, reduzindo a ocorrência de erros e proporcionando maior eficiência no gerenciamento das informações. A informatização desses processos contribui para a padronização dos registros, facilita a recuperação de dados históricos e melhora a confiabilidade das informações armazenadas. Além disso, a utilização de um banco de dados centralizado permite que os registros sejam mantidos de forma organizada e consistente, favorecendo a geração de relatórios e consultas gerenciais.

Do ponto de vista tecnológico, o sistema foi projetado utilizando conceitos de Programação Orientada a Objetos (POO), arquitetura MVC e banco de dados relacional, possibilitando uma estrutura organizada, modular e de fácil manutenção. A adoção dessas práticas favorece futuras expansões do sistema, permitindo a inclusão de novas funcionalidades e adaptações conforme as necessidades dos usuários. Além disso, o projeto contempla requisitos relacionados à segurança das informações, validação de dados, controle de acesso por perfil e armazenamento consistente dos registros operacionais.

Entre os benefícios esperados com a implementação do SISFROTA destaca-se a melhoria do controle das embarcações e das atividades de manutenção, reduzindo a possibilidade de atrasos ou falhas decorrentes da ausência de acompanhamento adequado. O sistema também contribui para a redução de custos operacionais, uma vez que o planejamento das manutenções preventivas tende a

minimizar gastos com reparos emergenciais e interrupções inesperadas das atividades.

Sob o aspecto social, a solução pode contribuir para a melhoria da qualidade dos serviços de transporte fluvial oferecidos à população, promovendo maior segurança e confiabilidade nas operações. Uma frota adequadamente monitorada e mantida reduz riscos de acidentes e aumenta a disponibilidade das embarcações para atendimento das demandas da comunidade. Além disso, o acesso rápido às informações facilita a tomada de decisões pelos gestores e contribui para uma administração mais eficiente dos recursos disponíveis.

Em relação aos benefícios organizacionais, o sistema proporciona maior controle sobre documentos, viagens, abastecimentos e históricos de manutenção, permitindo que os responsáveis pela gestão da frota tenham uma visão mais ampla das operações realizadas. A geração de relatórios gerenciais e alertas automáticos auxilia no planejamento das atividades e na identificação de situações que demandam atenção, contribuindo para o aprimoramento dos processos internos.

Portanto, o SISFROTA apresenta-se como uma solução tecnológica voltada à informatização dos processos de gerenciamento de frotas e manutenção de embarcações. Ao centralizar informações operacionais e técnicas em uma única plataforma, o sistema busca promover maior organização, segurança, eficiência e confiabilidade na administração das embarcações, contribuindo para a melhoria dos serviços de transporte fluvial e para o fortalecimento das atividades desenvolvidas na região amazônica.

3 ÁREA DE ABRANGÊNCIA

O projeto SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações foi desenvolvido com base nas necessidades operacionais da empresa fictícia Amazon River Transportes Fluviais Ltda., organização criada para fins acadêmicos e utilizada como estudo de caso durante a Prática Profissional Supervisionada do Curso Técnico em Desenvolvimento de Sistemas.

A Amazon River Transportes Fluviais Ltda. atua no segmento de transporte fluvial de passageiros e cargas na região amazônica, realizando operações entre comunidades ribeirinhas e centros urbanos. A empresa possui uma frota composta por embarcações de diferentes capacidades, sendo responsável pelo transporte seguro de passageiros, mercadorias e suprimentos essenciais para diversas localidades atendidas por suas rotas.

A estrutura organizacional da empresa é composta pelos setores de Administração da Frota, Operações de Navegação e Manutenção Técnica. O setor administrativo é responsável pelo gerenciamento das embarcações, controle documental e acompanhamento dos custos operacionais. O setor de operações coordena as viagens, escalas de tripulação e logística das embarcações. Já o setor de manutenção é responsável pelo planejamento e execução das manutenções preventivas e corretivas, garantindo o funcionamento adequado da frota e a segurança das operações.

A área de abrangência do SISFROTA contempla todos os processos relacionados ao gerenciamento operacional e técnico das embarcações da empresa. O sistema permite controlar informações sobre embarcações, tripulantes, viagens, manutenções, abastecimentos, incidentes e documentação obrigatória, centralizando os dados em uma única plataforma para facilitar o acesso e a gestão das informações.

Além disso, o sistema atende diferentes perfis de usuários, como Administrador da Frota, Operador/Despachante e Responsável Técnico pela Manutenção, possibilitando que cada profissional tenha acesso aos recursos necessários para desempenhar suas atividades de forma eficiente. Dessa maneira, a solução contribui

para melhorar o controle das operações, reduzir falhas administrativas e apoiar a tomada de decisões relacionadas à gestão da frota.

A empresa fictícia Amazon River Transportes Fluviais Ltda. está localizada na cidade de Manaus, Estado do Amazonas, atuando principalmente em rotas fluviais da região amazônica. Nesse contexto, o SISFROTA foi projetado para atender às necessidades de organizações que dependem do transporte hidroviário e necessitam de um controle eficiente de suas embarcações e processos de manutenção.

Dessa forma, a abrangência do projeto concentra-se na informatização dos processos de gestão de frotas náuticas, promovendo maior organização, segurança, confiabilidade das informações e eficiência operacional para empresas do setor de transporte fluvial.

4 OBJETIVOS

4.1. Objetivo Geral

Desenvolver o SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações, com o propósito de informatizar e centralizar o gerenciamento das operações relacionadas à frota, permitindo o controle de embarcações, tripulação, viagens, manutenções e documentação obrigatória, contribuindo para a melhoria da organização, segurança e eficiência operacional.

4.2. Objetivos Específicos

- a) Desenvolver funcionalidades para cadastro, consulta, atualização e gerenciamento das embarcações pertencentes à frota.
- b) Implementar mecanismos para cadastro e controle das informações da tripulação vinculada às embarcações.

- c) Permitir o registro e acompanhamento das manutenções preventivas e corretivas realizadas nas embarcações.
- d) Disponibilizar recursos para o agendamento de manutenções futuras, auxiliando no planejamento das atividades de manutenção.
- e) Registrar e armazenar informações referentes às viagens realizadas pelas embarcações, possibilitando consultas e acompanhamento histórico.
- f) Controlar a documentação obrigatória das embarcações, permitindo o acompanhamento de sua validade e regularidade.
- g) Implementar alertas para manutenções e documentos próximos do vencimento, contribuindo para a segurança e conformidade das operações.
- h) Gerar relatórios gerenciais relacionados às embarcações, viagens e manutenções, fornecendo informações para apoio à tomada de decisões.
- i) Garantir a integridade, organização e consistência dos dados armazenados por meio de validações e controles definidos pelo sistema.
- j) Aplicar os conhecimentos adquiridos durante o Curso Técnico em Desenvolvimento de Sistemas no desenvolvimento de uma solução prática voltada à gestão de frotas e manutenção de embarcações.

5 METAS

As metas estabelecidas para o desenvolvimento e validação do S – Sistema de Gestão Náutica e Manutenção de Embarcações visam quantificar os resultados esperados e definir os prazos para execução das atividades do projeto. O cronograma foi estruturado considerando a duração de três semanas do módulo de Prática Profissional Supervisionada (PPS), buscando atender às necessidades de controle operacional e manutenção de embarcações.

- **Na 1ª semana de projeto:** será realizado o levantamento e a análise dos requisitos do sistema, envolvendo o estudo dos processos de gerenciamento de embarcações, tripulantes, viagens e manutenções. Nesta etapa serão

identificadas e documentadas 100% das funcionalidades essenciais do sistema, servindo como base para a modelagem do banco de dados, elaboração dos diagramas e definição das regras de negócio.

- **Na 2ª semana de projeto:** será desenvolvido o protótipo funcional do SisFrota, incluindo a estrutura do banco de dados, interfaces do sistema e implementação das funcionalidades principais. Como meta, o sistema deverá permitir o cadastro e consulta de embarcações, tripulantes, viagens e manutenções, centralizando as informações operacionais em um único ambiente digital e reduzindo significativamente o uso de registros manuais.
- **Na 3ª semana de projeto:** serão realizados testes de funcionamento, validação das funcionalidades e correção de possíveis falhas identificadas. Os resultados obtidos servirão para avaliar a eficiência do sistema no gerenciamento das informações da frota. Ao final desta etapa, será entregue uma versão funcional do SisFrota, apta a auxiliar o controle operacional das embarcações, o acompanhamento das manutenções preventivas e corretivas e a organização das atividades relacionadas à gestão da frota náutica.

6 FASES DE EXECUÇÃO

Tabela 1 - Fases de Execução do Projeto

FASES	SEMANAS		
	1 ^a	2 ^a	3 ^a
I. Modelagem e Levantamento de Requisitos	X		
II. Design e Prototipação	X		
III. Gestão de Projeto(Kanban)	X	X	
IV. Desenvolvimento e Controle de Versão		X	
V. Documentação do Projeto		X	
VI. Relatório Técnico Final			X

6.1 Entrevista (questionário Google Forms)

Perguntas (em ordem e segmentos)

Questionário de Diagnóstico: SISFROTA — Sistema de Gestão Náutica e Manutenção de Embarcações.

Este questionário é rápido, anônimo e tem como objetivo identificar as principais dificuldades no controle de embarcações, manutenção e viagens realizado atualmente, contribuindo para o desenvolvimento do SISFROTA, uma solução tecnológica para tornar esses processos mais organizados, seguros e eficientes.

1. Você considera que o controle atual de embarcações, viagens e manutenções é organizado e eficiente? *(Tipo de pergunta no Forms: Múltipla escolha)*

- Eficiente
- Parcialmente eficiente
- Ineficiente
- Não sei informar

2. Quais são os principais problemas que podem ocorrer quando o controle de embarcações e manutenções é feito manualmente ou de forma descentralizada? *(Tipo de pergunta no Forms: Seleção)*

- Perda ou dificuldade de localizar informações
- Esquecimento de manutenções e revisões
- Falta de controle sobre documentos e vencimentos
- Não identifico problemas nesse tipo de controle

3. Você considera importante que um sistema envie alertas sobre documentos próximos do vencimento e manutenções que precisam ser realizadas? *(Tipo de pergunta no Forms: Múltipla escolha)*

- Muito importante
- Importante
- Moderadamente importante
- Não considero importante

4. Na sua opinião, qual seria o principal benefício de utilizar um sistema para gerenciar embarcações, viagens e manutenções? *(Tipo de pergunta no Forms: Seleção)*

- Maior controle das manutenções
- Maior segurança nas viagens e para os passageiros
- Redução do risco de esquecer documentos e revisões
- Facilidade para consultar históricos

5. Quais informações você considera mais importantes para acompanhar em um sistema de gestão de embarcações? *(Tipo de pergunta no Forms: Seleção)*

- Situação da embarcação
- Manutenções realizadas e futuras
- Validade dos documentos

- Tripulação responsável

6. Quem você acredita que seria mais beneficiado pelo uso de um sistema como o SISFROTA? *(Tipo de pergunta no Forms: Múltipla escolha)*

- Administradores de frota
- Responsáveis técnicos pela manutenção
- Proprietários de embarcações
- Todos os envolvidos na operação

7. Na sua opinião, qual seria o impacto da implantação de um sistema como o SISFROTA na gestão de embarcações? *(Tipo de pergunta no Forms: Múltipla escolha)*

- Muito positivo
- Positivo
- Pouco significativo
- Negativo

8. Você utilizaria um sistema como o SISFROTA para auxiliar no controle de embarcações, viagens e manutenções? *(Tipo de pergunta no Forms: Múltipla escolha)*

- Com certeza utilizaria
- Provavelmente utilizaria
- Provavelmente não utilizaria
- Não utilizaria

6.2 Requisitos Funcionais(RF)

Tabela 2 - Requisitos Funcionais

Requisitos Funcionais	
RF01 - Autenticação e Controle de Acesso	O sistema deve permitir o login de usuários e direcioná-los para a Dashboard correspondente ao seu perfil (Administrador, Operador/Despachante, Responsável Técnico).
RF02 - Cadastrar Embarcações	Permitir o cadastro, alteração, consulta e inativação de embarcações (nome, modelo, capacidade de carga/passageiros, ano, horímetro/odômetro e documentação associada).
RF03 - Cadastrar Tripulação	Gerenciar dados dos tripulantes (pilotos, condutores fluviais, marinheiros) incluindo nome, CPF, registro de habilitação (CIR/Arrais/Mestre) e validade da carteira.
RF04 - Registrar Manutenções	Registrar manutenções preventivas e corretivas por embarcação, detalhando serviços executados, peças trocadas e custos.
RF05 - Agendar Manutenções	Agendar as próximas revisões preventivas com base na data ou no horímetro/horas de uso da embarcação.
RF06 - Registrar Viagens Realizadas	Cadastrar saídas e chegadas (rota, data/horário de partida e término, quantidade de passageiros e comandante responsável).

RF07 - Alerta de Manutenções	Notificar e alertar no sistema manutenções que estejam vencidas ou próximas do vencimento estipulado.
RF08 - Relatório de Custos de Manutenção	Gerar relatórios com o somatório de custos de manutenção por embarcação em determinado período.
RF09 - Consultar Histórico de Manutenções	Permitir a busca do histórico completo de intervenções técnicas de uma embarcação específica.
RF10 - Controle de Validade de Documentação	Monitorar e alertar sobre prazos de vencimento de documentos obrigatórios (vistorias da Capitania dos Portos, seguros obrigatórios, licenças ambientais).
RF11 - Relatório de Viagens	Emitir relatórios analíticos de viagens realizadas por período, rota ou embarcação.
RF12 - Registrar Abastecimento	Registrar abastecimentos de combustível (data, litros abastecidos, valor total, posto/fornecedor e embarcação).
RF13 - Registrar Incidentes	Registrar ocorrências, avarias ou problemas identificados durante as viagens para análise da equipe técnica.
RF14 - Backup Manual do Banco de Dados	Permitir que o perfil Administrador gere uma cópia de segurança (.sql) do banco de dados local.
RF15 - Exportar Relatórios em PDF	Permitir a exportação dos relatórios gerados em formato PDF formatado para impressão ou envio.

Fonte: Autoria Própria

6.3 Requisitos Não Funcionais(RNF)

Tabela 3 - Requisitos Não Funcionais(RNF)

Requisitos Não Funcionais	
RNF01 - Facilidade de uso	O sistema deverá possuir interface simples e intuitiva.
RNF02 - Compatibilidade	O sistema deverá funcionar corretamente nos principais navegadores.
RNF03 - Desempenho	As consultas deverão ser exibidas rapidamente ao usuário.
RNF04 - Responsividade	O sistema deverá adaptar-se a diferentes tamanhos de tela.
RNF05 - Validação de dados	O sistema deverá validar os campos obrigatórios antes de salvar os registros.
RNF06 - Organização visual	O sistema deverá apresentar informações de forma clara e organizada.
RNF07 - Armazenamento seguro	Os dados cadastrados deverão ser armazenados de forma consistente.
RNF08 - Mensagens do sistema	O sistema deverá informar ao usuário quando uma operação for realizada com sucesso ou apresentar erro.
RNF09 - Manutenibilidade	O código deverá ser organizado para facilitar correções e melhorias futuras.

RNF10 - Disponibilidade

O sistema deverá permanecer funcional durante todo o período de utilização.

Fonte: Autoria Própria**6.4 Regras de Negócio(RN)****Tabela 4 - Regras de Negócio(RN)**

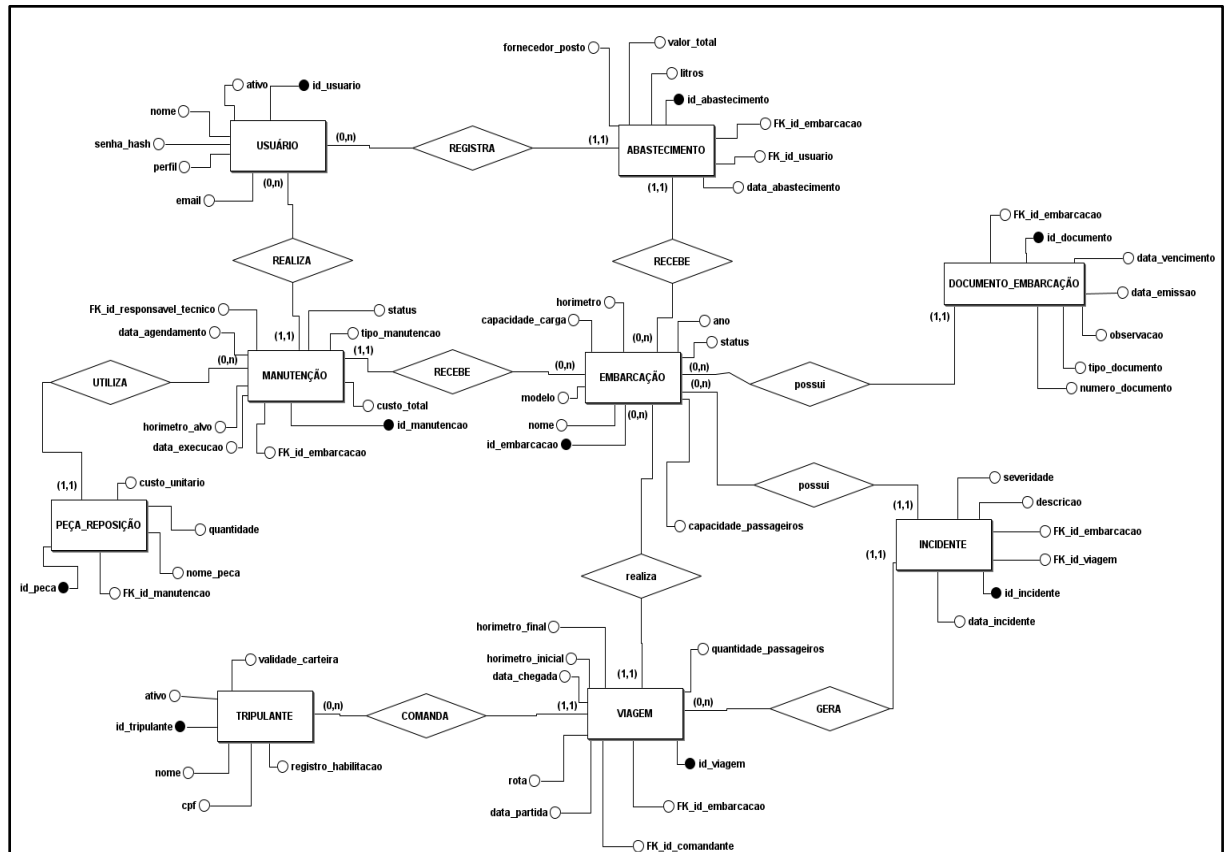
Regra de Negócio	
RN01 - Identificação Única da Embarcação	Cada embarcação cadastrada no sistema deverá possuir uma identificação única, não sendo permitido o cadastro de duas embarcações com a mesma identificação.
RN02 - Cadastro Obrigatório de Embarcação	Não será permitido cadastrar embarcações sem o preenchimento dos campos obrigatórios, como nome e identificação.
RN03 - Cadastro Obrigatório de Tripulante	Não será permitido cadastrar tripulantes sem informar nome completo e função exercida na embarcação.
RN04 - Vinculação de Manutenção à Embarcação	Toda manutenção registrada deverá estar associada a uma embarcação previamente cadastrada no sistema.
RN05 - Vinculação de Viagem à Embarcação	Toda viagem registrada deverá estar vinculada a uma embarcação cadastrada no sistema.
RN06 - Vinculação de Documentação à Embarcação	Os documentos cadastrados deverão estar obrigatoriamente associados a uma embarcação existente.
RN07 - Valor Válido para Manutenção	O valor informado para uma manutenção deverá ser maior que zero, garantindo a consistência dos registros financeiros.
RN08 - Quantidade Válida de Passageiros	A quantidade de passageiros informada em uma viagem deverá ser maior que zero.
RN09 - Confirmação de Exclusão	Antes da exclusão de qualquer registro, o sistema deverá solicitar a confirmação do usuário para evitar remoções acidentais.
RN10 - Validação de Campos Obrigatórios	O sistema não deverá permitir o salvamento de registros com campos obrigatórios em branco.
RN11 - Restrição de Registro de Viagens	Apenas embarcações ativas e cadastradas no sistema poderão receber novos registros de viagens.
RN12 - Restrição de Registro de Manutenções	Apenas embarcações ativas e cadastradas no sistema poderão receber novos registros de manutenções.

Fonte: Autoria Própria

6.5 Modelagem (Banco de Dados)

Modelagem no brModelo:

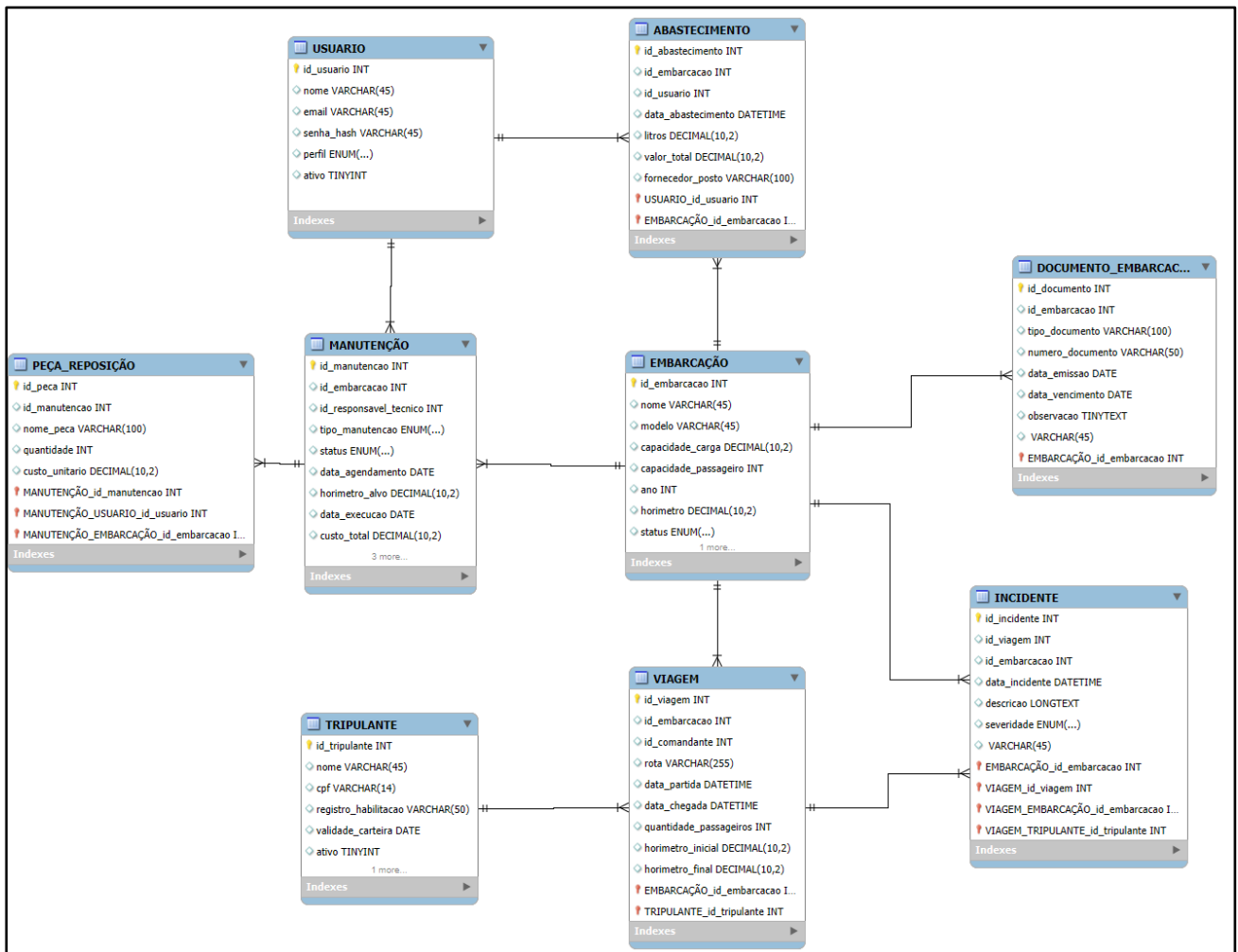
Figura 1 – Modelo Conceitual do Banco de Dados(DER)



Fonte: Autoria Própria

Modelagem no mySQL Workbench

Figura 2 - Modelo Lógico do Banco de Dados



Fonte: Autoria Própria

Modelo Físico: Script/Código SQL para criação do banco.

Tabela Usuários

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL,
  perfil ENUM('ADMINISTRADOR', 'OPERADOR', 'TECNICO') NOT NULL,
  status ENUM('ATIVO', 'INATIVO') DEFAULT 'ATIVO',
  data_criacao DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabela Embarcações

```
CREATE TABLE embarcacoes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  modelo VARCHAR(50) NOT NULL,
  capacidade_passageiros INT NOT NULL DEFAULT 0,
  capacidade_carga_ton DECIMAL(8,2) DEFAULT 0.00,
  ano_fabricacao INT NOT NULL,
  horimetro_horas INT NOT NULL DEFAULT 0,
  status ENUM('ATIVA', 'EM_MANUTENCAO', 'INATIVA') DEFAULT 'ATIVA',
  data_cadastro DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabela Tripulantes

```
CREATE TABLE tripulantes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cpf VARCHAR(14) NOT NULL UNIQUE,
  categoria_habilitacao ENUM('PILOTO_FLUVIAL', 'CONDUTOR_FLUVIAL', 'ARRAIS_AMADOR',
'MESTRE_AMADOR', 'CAPITAO_AMADOR') NOT NULL,
  numero_registro_cir VARCHAR(30) NOT NULL UNIQUE,
  data_vencimento_cir DATE NOT NULL,
  status ENUM('DISPONIVEL', 'EM_VIAGEM', 'INATIVO') DEFAULT 'DISPONIVEL'
) ENGINE=InnoDB;
```

Tabela Documentos_Embarcação

```
CREATE TABLE documentos_embarcacao (
  id INT AUTO_INCREMENT PRIMARY KEY,
  id_embarcacao INT NOT NULL,
  tipo_documento ENUM('VISTORIA_CAPITANIA', 'SEGURO_DPEM', 'LICENCA_AMBIENTAL',
'CERTIFICADO_NAVEGABILIDADE') NOT NULL,
  numero_documento VARCHAR(50) NOT NULL,
  data_emissao DATE NOT NULL,
  data_vencimento DATE NOT NULL,
  status ENUM('VALIDO', 'ALERTA_VENCIMENTO', 'VENCIDO') DEFAULT 'VALIDO',
  FOREIGN KEY (id_embarcacao) REFERENCES embarcacoes(id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Tabela Viagens

```
CREATE TABLE viagens (
  id INT AUTO_INCREMENT PRIMARY KEY,
  id_embarcacao INT NOT NULL,
  id_comandante INT NOT NULL,
  rota_destino VARCHAR(150) NOT NULL,
```

```

data_hora_partida DATETIME NOT NULL,
data_hora_chegada DATETIME NULL,
quantidade_passageiros INT NOT NULL DEFAULT 0,
status ENUM('EM_ANDAMENTO', 'CONCLUIDA', 'CANCELADA') DEFAULT 'EM_ANDAMENTO',
FOREIGN KEY (id_embarcacao) REFERENCES embarcacoes(id),
FOREIGN KEY (id_comandante) REFERENCES tripulantes(id)
) ENGINE=InnoDB;

```

Tabela Manutenções

```

CREATE TABLE manutencoes (
id INT AUTO_INCREMENT PRIMARY KEY,
id_embarcacao INT NOT NULL,
tipo_manutencao ENUM('PREVENTIVA', 'CORRETIVA') NOT NULL,
descricao_servico TEXT NOT NULL,
horimetro_agendado INT NULL, -- Horímetro limite para revisão
data_agendamento DATE NOT NULL,
data_execucao DATE NULL,
custo_total DECIMAL(10,2) DEFAULT 0.00,
status ENUM('AGENDADA', 'EM_ANDAMENTO', 'CONCLUIDA', 'CANCELADA') DEFAULT
'AGENDADA',
FOREIGN KEY (id_embarcacao) REFERENCES embarcacoes(id)
) ENGINE=InnoDB;

```

Tabela Abastecimentos

```

CREATE TABLE abastecimentos (
id INT AUTO_INCREMENT PRIMARY KEY,
id_embarcacao INT NOT NULL,
data_abastecimento DATETIME DEFAULT CURRENT_TIMESTAMP,
quantidade_litros DECIMAL(8,2) NOT NULL,
valor_total DECIMAL(10,2) NOT NULL,
fornecedor_posto VARCHAR(100) NOT NULL,
FOREIGN KEY (id_embarcacao) REFERENCES embarcacoes(id)
) ENGINE=InnoDB;

```

Tabela Incidentes

```

CREATE TABLE incidentes (
id INT AUTO_INCREMENT PRIMARY KEY,
id_embarcacao INT NOT NULL,
id_viagem INT NULL,
data_incidente DATETIME DEFAULT CURRENT_TIMESTAMP,
descricao TEXT NOT NULL,
gravidade ENUM('BAIXA', 'MEDIA', 'ALTA', 'CRITICA') NOT NULL,
status ENUM('PENDENTE', 'EM_ANALISE', 'RESOLVIDO') DEFAULT 'PENDENTE',
FOREIGN KEY (id_embarcacao) REFERENCES embarcacoes(id),
FOREIGN KEY (id_viagem) REFERENCES viagens(id) ON DELETE SET NULL
) ENGINE=InnoDB;

```

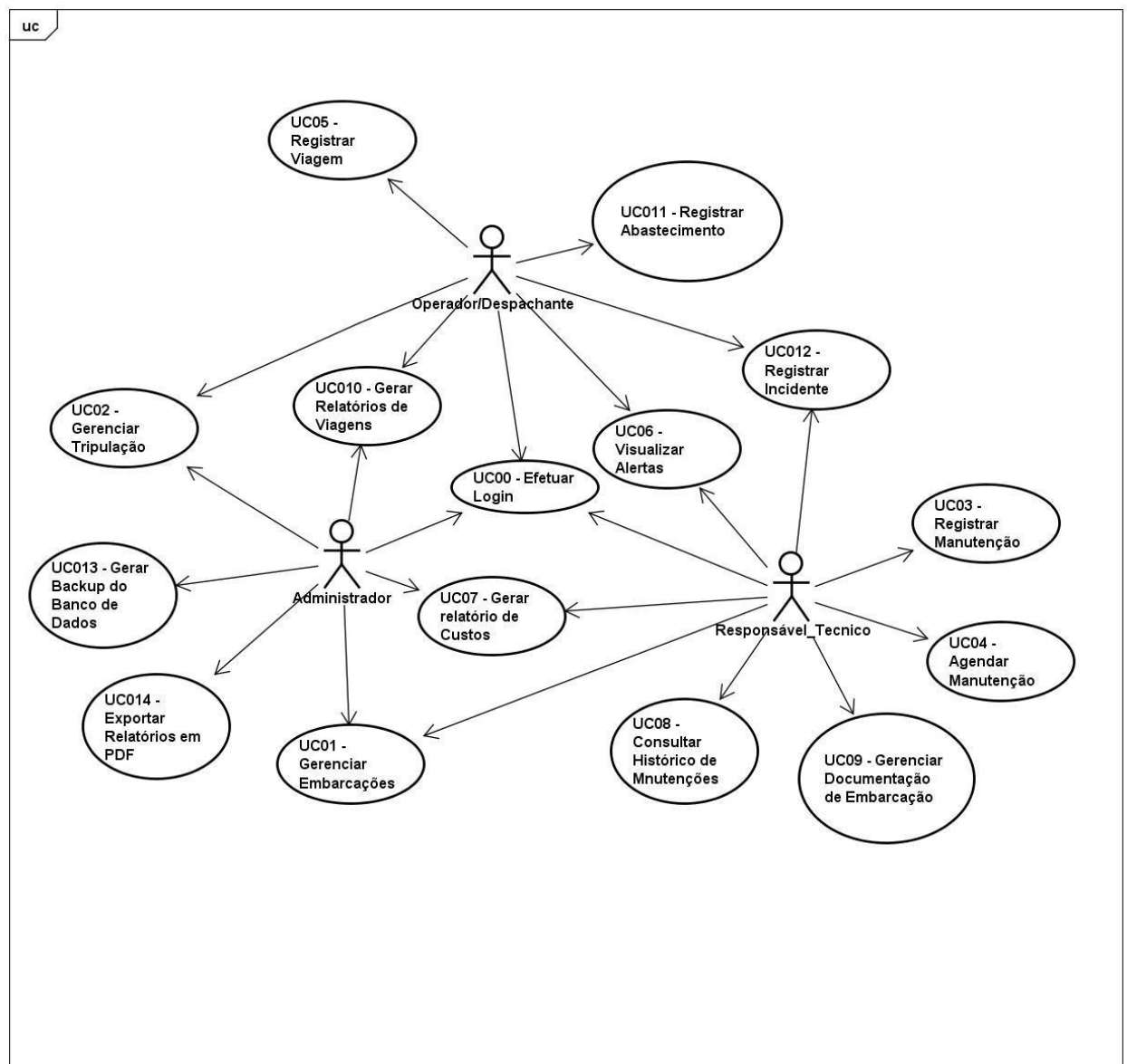
7 METODOLOGIA

Diagramas(Ferramenta: Astah)

7.1 Diagrama de Caso de Uso

Representação dos atores e das elipses (as ações/casos de uso que eles executam no sistema).

Figura 3 - Diagrama de Casos de Uso



powered by Astah

Fonte: Autoria Própria

Tabela 5 - Relacionamento entre Casos de Uso e Atores

Ordem Numeração	Casos de Uso	Relacionamentos
UC00	Efetuar Login	Todos os atores
UC01	Gerenciar Embarcações	Ator: Administrador
UC02	Gerenciar Tripulação	Ator: Administrador
UC03	Registrar Manutenção	Ator: Responsável Técnico
UC04	Agendar Manutenção	Ator: Responsável Técnico
UC05	Registrar Viagem	Ator: Operador/Despachante
UC06	Visualizar Alertas	Ator: Operador/Despachante, Responsável Técnico
UC07	Gerar Relatório de Custos	Ator: Administrador, Responsável Técnico
UC08	Consultar Histórico de Manutenções	Ator: Responsável Técnico
UC09	Gerenciar Documentação de Embarcação	Ator: Responsável Técnico
UC010	Gerar Relatórios de Viagens	Ator: Administrador, Operador/Despachante
UC011	Registrar Abastecimento	Ator: Operador/Despachante
UC012	Registrar Incidente	Ator: Operador/Despachante, Responsável Técnico
UC013	Gerar Backup do Banco de Dados	Ator: Administrador
UC014	Exportar Relatórios em PDF	Ator: Administrador

Fonte: Autoria Própria

Tabela 6 - Especificação do Caso de Uso: Ator Administrador

Atores:	Administrador
Pré-Condições:	Administrador autenticado no sistema por meio do UC00 – Efetuar Login .
Trigger:	Acessar uma funcionalidade administrativa do sistema e selecionar a operação desejada.
Fluxo Principal:	1. Administrador acessa o sistema e realiza o login. 2. Acessa o módulo correspondente à operação desejada. 3. Seleciona a operação que deseja realizar. 4. Informa ou altera os dados necessários. 5. O sistema valida os dados conforme as regras de negócio. 6. O sistema salva os dados ou executa a operação solicitada.
Fluxo Alternativo:	1. Cadastrar novas embarcações ou tripulantes. 2. Alterar dados de registros existentes. 3. Remover registros, mediante confirmação do usuário quando aplicável. 4. Gerar relatórios de custos e viagens. 5. Exportar relatórios em PDF. 6. Realizar backup do banco de dados.
Pós-condições:	Dados cadastrados ou atualizados com sucesso; operações realizadas conforme solicitado; relatórios gerados ou exportados quando solicitado; backup realizado quando solicitado; registros mantidos de acordo com as regras de negócio.
Regras de negócio:	RN01 – Cada embarcação cadastrada deve possuir uma identificação única. RN02 – Não é permitido cadastrar embarcações sem os campos obrigatórios, como nome e identificação. RN03 – Não é permitido cadastrar tripulantes sem informar nome completo e função exercida na embarcação. RN04 – Toda manutenção registrada deve estar associada a uma embarcação previamente cadastrada. RN05 – Toda viagem registrada deve estar vinculada a uma embarcação cadastrada no sistema. RN06 – Os documentos cadastrados devem estar obrigatoriamente associados a uma embarcação existente. RN07 – O valor informado para uma manutenção deve ser maior que zero. RN08 – A quantidade de passageiros informada em uma viagem deve ser maior que zero. RN09 – Antes da exclusão de qualquer registro, o sistema deve solicitar a confirmação do usuário. RN10 – O sistema não deve permitir o salvamento de registros com campos obrigatórios em branco. RN11 – Apenas embarcações ativas e cadastradas no sistema podem receber novos registros de viagens.

RN12 – Apenas embarcações ativas e cadastradas no sistema podem receber novos registros de manutenções.

Fonte: Autoria Própria

Tabela 7 - Especificação do Caso de Uso: Ator Operador/Despachante

Atores:	Operador/Despachante
Pré-Condições:	Usuário autenticado no sistema e dados necessários previamente cadastrados.
Trigger:	Acessar uma das funções disponíveis ao Operador/Despachante.
Fluxo Principal:	1. Acessar uma das funções disponíveis ao Operador/Despachante. 2. Operador acessa os alertas. 3. Operador seleciona os filtros.
Fluxo Alternativo:	1. Registrar viagens. 2. Visualizar alertas. 3. Gerar relatórios de viagens. 4. Registrar abastecimentos. 5. Registrar incidentes.
Pós-condições:	Viagens, abastecimentos e incidentes registrados com sucesso; alertas e relatórios apresentados conforme a solicitação do operador.
Regras de negócio:	RN05 – Vinculação de Viagem à Embarcação RN08 – Quantidade Válida de Passageiros RN10 – Validação de Campos Obrigatórios RN11 – Restrição de Registros de Viagens

Fonte : Autoria Própria

Tabela 8 - Especificação do Caso de Uso: Ator Responsável Técnico

Atores:	Responsável Técnico
Pré-Condições	Responsável Técnico autenticado
Trigger:	Apertar o botão para registrar manutenções, documentações ou incidentes.
Fluxo Principal	1. Responsável Técnico acessa o módulo técnico. 2. Seleciona a operação desejada. 3. Sistema salva as alterações.
Fluxo Alternativo:	1. Registrar e agendar manutenções. 2. Consultar histórico de manutenções. 3. Gerenciar documentações de embarcações. 4. Registrar incidentes e consultar alertas. 5. Gerar relatórios de custos de manutenção.

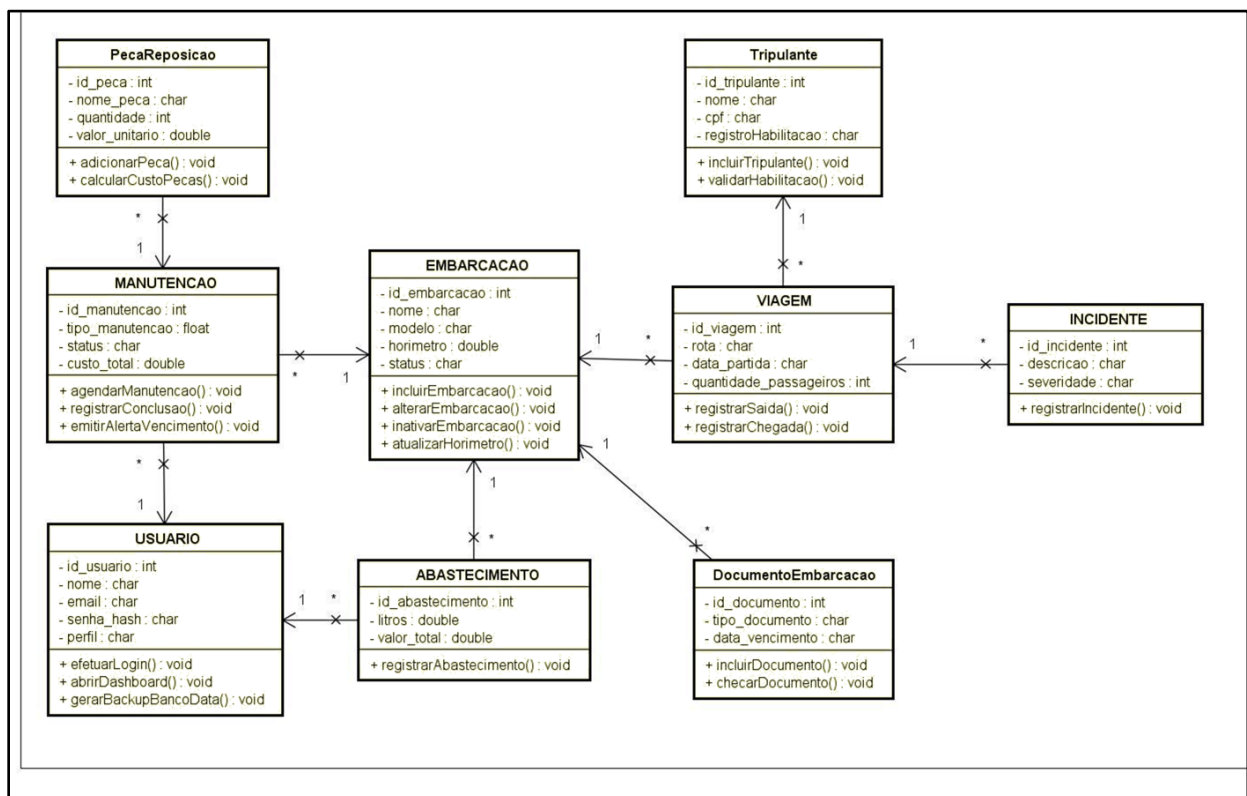
Pós-condições:	Históricos de manutenção e documentações atualizados.
Regras de negócio:	RN004 - Vinculação de Manutenção à Embarcação. RN006 - Vinculação de Documentação à Embarcação. RN007 - Valor Válido para Manutenção. RN010 - Validação de Campos Obrigatórios. RN012 - Restrição de Registro de Manutenções.

Fonte: Autoria Própria

7.2 Diagrama de Classes

Estrutura orientada a objetos contendo o nome das classes, os atributos e os métodos.

Figura 4 - Diagrama de Classes do Sistema



Fonte: Autoria Própria

Nome Tabela	usuarios
Volume esperado	Baixo — aproximadamente 10 a 50 registros.
Tempo de retenção	Enquanto houver necessidade de manter o histórico de acesso e usuários cadastrados.
Rotina de limpeza	Não se aplica

ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificador do usuário	id	INT	—	PK, AUTO_INCREMENT	Identificador único do usuário.
Nome do usuário	nome	VARCHAR	100	NOT NULL	Nome completo do usuário
E-mail do usuário	email	VARCHAR	100	NOT NULL, UNIQUE	E-mail utilizado para identificação e acesso ao sistema.
Senha do usuário	senha	VARCHAR	255	NOT NULL	Senha armazenada de forma criptografada/hash.
Perfil profissional	perfil	ENUM	—	NOT NULL	Define o nível de acesso: ADMINISTRADOR, OPERADOR ou TECNICO.
Ativo ou não no sistema	status	ENUM	—	DEFAULT 'ATIVO'	Indica se o usuário está ATIVO ou INATIVO.
Data da criação no sistema	data_criacao	DATETIME	—	DEFAULT CURRENT_TIMESTAMP	Data e hora em que o usuário foi cadastrado.

Fonte: Autoria Própria

Tabela 10 - Dicionário de Dados: Entidade Embarcações

ENTIDADE					
Entidade	Embarcações				
Descrição	Armazena os dados cadastrais e operacionais das embarcações pertencentes à frota.				
Nome Tabela	embarcações				
Volume esperado	Baixo — aproximadamente 5 a 50 registros.				
Tempo de retenção	Durante todo o período em que houver histórico da embarcação no sistema.				
Rotina de limpeza	Não se aplica				
ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificador da embarcação	id	INT	—	PK, AUTO_INCREMENT	Identificador único da embarcação.
Nome da embarcação	nome	VARCHAR	100	NOT NULL	Nome de identificação da embarcação.
Modelo da embarcação	modelo	VARCHAR	50	NOT NULL	Modelo ou tipo da embarcação.
Capacidade de passageiros	capacidade_passageiros	INT	—	NOT NULL, DEFAULT 0	Quantidade máxima de passageiros permitida.
Capacidade de carga	capacidade_carga_ton	DECIMAL	8,2	DEFAULT 0.00	Capacidade máxima de carga da embarcação em toneladas.
Ano de fabricação	ano_fabricação	INT	—	NOT NULL	Ano em que a embarcação foi fabricada.
Horímetro	horimetro_horas	INT	—	NOT NULL, DEFAULT 0	Total de horas de funcionamento/uso do motor.

Ativo ou não no sistema	status	ENUM	—	DEFAULT 'ATIVA'	Situação atual da embarcação: ATIVA, EM_MANUTENCAO ou
Data do cadastro no sistema	data_cadastro	DATETIME	—	DEFAULT CURRENT_TIMESTAMP	Data e hora do cadastro da embarcação.

Fonte: Autoria Própria

Tabela 11 - Dicionário de Dados: Entidade Tripulantes

ENTIDADE					
Entidade	Tripulantes				
Descrição	Armazena os dados cadastrais, habilitação e situação dos profissionais que compõem a tripulação das embarcações.				
Nome Tabela	tripulantes				
Volume esperado	Baixo — aproximadamente 10 a 100 registros.				
Tempo de retenção	Enquanto houver histórico do tripulante no sistema.				
Rotina de limpeza	Não se aplica				
ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificador do tripulante	id	INT	—	PK, AUTO_INCREMENT	Identificador único do tripulante.
Nome do tripulante	nome	VARCHAR	100	NOT NULL	Nome completo do tripulante.
CPF do tripulante	cpf	VARCHAR	14	NOT NULL	CPF do tripulante.

Categoria de habilitação	categoria_habilitação	ENUM	—	NOT NULL	Categoria da habilitação do tripulante.
Número do registro CIR	numero_registro_cir	VARCHAR	30	NOT NULL, UNIQUE	Número de registro da Carteira de Inscrição e Registro (CIR).
Data de vencimento do CIR	data_vencimento_cir	DATE	—	NOT NULL	Data de vencimento da habilitação/CIR.
Ativo ou não no sistema	staus	ENUM	—	DEFAULT 'DISPONÍVEL'	Situação do tripulante: DISPONIVEL, EM_VIAGEM ou INATIVO.

Fonte: Autoria Própria

Tabela 12 - Dicionário de Dados: Entidade Documentos da Embarcação

ENTIDADE					
Entidade	Documentos da Embarcação				
Descrição	Armazena os documentos obrigatórios relacionados às embarcações, incluindo emissão, vencimento e situação de validade.				
Nome Tabela	documentos_embarcacao				
Volume esperado	Médio — aproximadamente 20 a 200 registros				
Tempo de retenção	Durante todo o período de utilização do sistema, mantendo documentos vencidos para histórico.				
Rotina de limpeza	Não se aplica				
ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição

Identificador do documento	id	INT	—	PK, AUTO_INCREMENT	Identificador único do documento.
Identificador da embarcação	id_embarcacao	INT	—	NOT NULL, FK	Identifica a embarcação à qual o documento pertence.
Tipo de documento	tipo_documento	ENUM	—	NOT NULL	Tipo do documento obrigatório da embarcação.
Número do documento	numero_documento	VARCHAR	50	NOT NULL	Número ou código de identificação do documento.
Data de emissão do documento	data_emissao	DATE	—	NOT NULL	Data em que o documento foi emitido.
Data de vencimento do documento	data_vencimento	DATE	—	NOT NULL	Data limite de validade do documento.
Ativo ou não no sistema	status	ENUM	—	DEFAULT 'VALIDO'	Situação do documento: VALIDO, ALERTA_VENCIMENTO ou VENCIDO.

Fonte: Autoria Própria

Tabela 13 - Dicionário de Dados: Entidade Viagens

ENTIDADE	
Entidade	VIAGENS
Descrição	Identificador único da viagem. Gerado automaticamente pelo sistema para controlar cada viagem cadastrada.
Nome Tabela	viagens
Volume esperado	Aproximadamente 10.000 registros.
Tempo de retenção	5 anos

Rotina de limpeza Arquivamento anual de viagens concluídas

ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificação da Viagem	id	INT	Inteiro	PK, AUTO_INCREMENT	Identificador único da viagem gerado automaticamente pelo sistema.
Identificação da Embarcação	id_embarcacao	INT	Inteiro	FK, NOT NULL	Código da embarcação utilizada na viagem.
Identificação do Comandante	id_comandante	INT	Inteiro	FK, NOT NULL	Código do tripulante responsável pela condução da embarcação.
Destino da Viagem	rota_destino	VARCHAR	150	NOT NULL	Local de destino ou trajeto previsto para a viagem.
Data/Hora de Partida	data_hora_partida	DATETIME	Data e Hora	NOT NULL	Data e horário de início da viagem.
Data/Hora de Chegada	data_hora_chegada	DATETIME	Data e Hora	NOT NULL	Data e horário de término da viagem.
Quantidade de Passageiros	quantidade_passageiros	INT	Inteiro	NOT NULL	Número total de passageiros transportados.
Status da Viagem	status	ENUM	3 valores	DEFAULT	Situação atual da viagem (EM_ANDAMENTO, CONCLUIDA ou CANCELADA).

Fonte: Autoria Própria

Tabela 14 - Dicionário de Dados: Entidade Manutenções

ENTIDADE

Entidade	MANUTENÇÃO
Descrição	Armazena o histórico e o planejamento das manutenções preventivas e corretivas realizadas nas embarcações da frota
Nome Tabela	manutencoes
Volume esperado	Aproximadamente 5.000 registros
Tempo de retenção	10 anos
Rotina de limpeza	Arquivamento após conclusão dos serviços

ATRIBUTOS

Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificador da Manutenção	id	INT	Inteiro	PK, AUTO_INCREMENT	Identificador único da manutenção cadastrada.
Embarcação	id_embarcacao	INT	INTEIRO	FK, NOT NULL	Código da embarcação submetida à manutenção.
Tipo de Manutenção	tipo_manutencao	ENUM	2 valores	NOT NULL	Define se a manutenção é PREVENTIVA ou CORRETIVA.
Descrição do Serviço	descricao_servico	TEXT	Texto livre	NOT NULL	Descrição detalhada dos serviços executados ou programados.
Horímetro Programado	horimetro_agendado	INT	Inteiro	NULL	Quantidade de horas de funcionamento prevista para realização da manutenção
Data de Agendamento	data_agendamento	DATE	Data	NOT NULL	Data programada para execução da manutenção.

Data de Execução	data_execucao	DATE	Data	NULL	Data em que a manutenção foi efetivamente realizada.
Custo Total	custo_total	DECIMAL	10,2	DEFAULT 0.00	Valor total gasto na manutenção.
Status da Manutenção	status	ENUM	4 valores	DEFAULT	Situação da manutenção (AGENDADA, EM_ANDAMENTO, CONCLUIDA ou CANCELADA).

Fonte: Autoria Própria

Tabela 15 - Dicionário de Dados: Entidade Abastecimentos

ENTIDADE					
Entidade		ABASTECIMENTOS			
Descrição		Registra os abastecimentos realizados pelas embarcações, permitindo o controle do consumo de combustível e dos custos operacionais.			
Nome Tabela		abastecimentos			
Volume esperado		Aproximadamente 20.000 registros			
Tempo de retenção		5 anos			
Rotina de limpeza		Arquivamento anual dos registros mais antigos			
ATRIBUTOS					
Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificação do Abastecimento	id	INT	Inteiro	PK, AUTO_INCREMENT	Identificador único do abastecimento realizado.

Embarcação	id_embarcacao	INT	Inteiro	FK, NOT NULL	Código da embarcação abastecida
Data do Abastecimento	data_abastecimento	DATETIME	Data e Hora	DEFAULT CURRENT_TIMESTAMP	Data e horário em que o abastecimento foi registrado.
Quantidade de Litros	quantidade_litros	DECIMAL	8,2	NOT NULL	Quantidade de combustível abastecida em litros.
Valor Total	valor_total	DECIMAL	10,2	NOT NULL	Valor financeiro pago pelo abastecimento.
Fornecedor/Posto	fornecedor_posto	VARCHAR	100	NOT NULL	Nome do posto ou fornecedor responsável pelo abastecimento.

Fonte: Autoria Própria

Tabela 16 - Dicionário de Dados: Entidade Incidentes

ENTIDADE	
Entidade	INCIDENTES
Descrição	Armazena registros de incidentes, falhas operacionais, acidentes e demais ocorrências relacionadas às embarcações e viagens da frota.
Nome Tabela	incidentes
Volume esperado	Aproximadamente 2.000 registros
Tempo de retenção	10 anos
Rotina de limpeza	Não aplicável, devido à importância histórica das ocorrências
ATRIBUTOS	

Atributo	Nome do campo	Tipo De dado	Tamanho	Restrição	Descrição
Identificação do Incidente	id	INT	Inteiro	PK, AUTO_INCREMENT	Identificador único da ocorrência registrada no sistema.
Embarcação Envolvida	id_embarcacao	INT	Inteiro	FK, NOT NULL	Código da embarcação relacionada ao incidente.
Viagem Relacionada	id_viagem	INT	Inteiro	FK, NULL	Código da viagem na qual o incidente ocorreu, quando aplicável.
Data do Incidente	data_incidente	DATETIME	Data e Hora	DEFAULT CURRENT_TIMESTAMP	Data e horário do incidente ou de seu registro no sistema.
Descrição da Ocorrência	descricao	TEXT	Texto livre	NOT NULL	Relato detalhado contendo informações sobre o ocorrido, possíveis causas e consequências.
Grau de Gravidade	gravidade	ENUM	4 valores	NOT NULL	Nível de severidade da ocorrência: BAIXA, MEDIA, ALTA ou CRITICA.
Status do Incidente	status	ENUM	3 valores	DEFAULT	Situação atual da ocorrência: PENDENTE, EM_ANALISE ou RESOLVIDO.

Fonte: Autoria Própria

7.5 Uso de Ferramentas de Inteligência Artificial

Durante a elaboração deste relatório técnico, foram utilizadas ferramentas de Inteligência Artificial Generativa (IAG) como instrumentos de apoio à revisão gramatical, organização de ideias, aprimoramento da redação e verificação de referências bibliográficas.

As ferramentas utilizadas foram:

- **Gemini** (Google, versão 3.7 Flash, 2026): utilizado para apoio na estruturação de tópicos, revisão gramatical e organização de ideias.

- **ChatGPT** (OpenAI, versão GPT-5.6 Sol, 2026): utilizado para revisão gramatical, aprimoramento da redação e verificação de formatação ABNT.
- **Perplexity** (Perplexity AI, versão gratuita, 2026): utilizado para busca e verificação de referências bibliográficas e informações técnicas sobre ferramentas de desenvolvimento.

A utilização dessas ferramentas ocorreu de forma estritamente instrumental, não substituindo a análise crítica, a tomada de decisões técnicas, a elaboração dos modelos, a implementação do sistema ou a responsabilidade dos autores pelo conteúdo apresentado.

Todas as informações obtidas por meio das ferramentas de inteligência artificial foram revisadas e, quando relacionadas a conceitos técnicos, confrontadas com documentação oficial, artigos acadêmicos e outras fontes bibliográficas confiáveis. A definição dos requisitos, regras de negócio, modelos, diagramas, estrutura do banco de dados e demais decisões técnicas do SISFROTA permaneceram sob responsabilidade exclusiva da equipe de desenvolvimento.

Declara-se, portanto, que o uso de inteligência artificial neste trabalho teve caráter complementar e auxiliar, sendo os autores integralmente responsáveis pela análise, interpretação, conclusões apresentadas e pela precisão das informações técnicas contidas neste relatório.

7.6 Protótipo do Sistema SISFROTA(Telas)

Figura 6 - Tela de Login

A imagem mostra uma janela de software com o título "Gestão Náutica - Login". Dentro da janela, há dois campos de entrada de texto. O primeiro campo é rotulado "E-mail:" e o segundo "Senha:". Abaixo desses campos, há dois botões de ação: "Entrar" e "Sair". A interface é simples, com uma barra de título padrão de sistema operacional e botões de minimizar, maximizar e fechar.

Fonte: Autoria Própria

Figura 7 - Tela de Usuário - Administrador



Fonte: Autoria Própria

Figura 8 - Tela de Usuário - Operador/Despachante



Fonte: Autoria Própria

Figura 9 - Tela de Usuário - Responsável Técnico



Fonte: Autoria Própria

8 DESCRIÇÃO DAS ATIVIDADES PRÁTICAS NA ETAPA

Softwares ou ferramentas que foram utilizadas no projeto

8.1 brModelo

O brModelo é uma ferramenta de código aberto desenvolvida para fins educacionais e profissionais, voltada à modelagem de bancos de dados relacionais. Criada em 2005 no âmbito do Grupo de Banco de Dados da UFSC (GBD/UFSC), a ferramenta segue a metodologia proposta por Carlos A. Heuser em sua obra "Projeto de Banco de Dados".

Principais Funcionalidades:

- **Modelo Conceitual:** Permite a construção do Diagrama Entidade-Relacionamento (DER), definindo entidades, atributos, relacionamentos e chaves primárias de forma visual.
- **Modelo Lógico:** Converte automaticamente as entidades do modelo conceitual em tabelas, especificando tipos de dados, chaves primárias e chaves estrangeiras.

- Geração de Script SQL: Exporta a estrutura modelada diretamente para o modelo físico, gerando comandos SQL prontos para implementação em SGBDs relacionais. A ferramenta foi concebida como trabalho de conclusão de curso de especialização em banco de dados pelas universidades UFSC e UNIVAG, orientado pelo Professor Dr. Ronaldo dos Santos Mello, após constatar-se a inexistência de uma ferramenta nacional para essa finalidade.

8.2 Astah

O Astah é uma ferramenta de modelagem de sistemas baseada na Linguagem de Modelagem Unificada (UML), desenvolvida no Japão pela Change Vision e implementada na plataforma Java, o que garante sua portabilidade entre diferentes sistemas operacionais (Windows, macOS e Linux) . Anteriormente conhecido como JUDE (Java UML Development Environment), o software é amplamente utilizado para modelagem orientada a objetos .

Principais Características:

- Suporte UML Completo: Oferece suporte a diagramas de classe, caso de uso, sequência, atividade, estado, implantação e componentes, seguindo padrões UML 1.4 e parcialmente UML 2.x .
- Modelagem ERD: Inclui funcionalidades para criação de Diagramas Entidade-Relacionamento (ERD), integrando modelagem de dados ao fluxo de desenvolvimento de software .
- Recursos Avançados: Disponibiliza geração de código Java a partir dos modelos, exportação de documentação em HTML/RTF, merge de projetos e comparação entre versões de diagramas .
- Assistência Inteligente: Oferece funções de auto-layout, validação de consistência de modelos e sugestões de elementos durante a criação de diagramas .

A UML, criada por Grady Booch, Ivar Jacobson e James Rumbaugh, é o padrão mais adotado para modelagem de sistemas orientados a objetos, sendo mantida pela OMG (Object Management Group), organização internacional que aprova padrões abertos para aplicações orientadas a objetos .

8.3 Trello

O Trello é uma plataforma de gerenciamento de projetos baseada no método Kanban, que organiza tarefas em quadros (boards), listas e cartões (cards) . A ferramenta facilita o acompanhamento visual do fluxo de trabalho, permitindo que equipes colaborem em tempo real e monitorem o progresso das atividades .

Principais Funcionalidades:

- Boards (Quadros): Representam o espaço de trabalho onde o projeto é gerenciado, podendo ser personalizados para diferentes fluxos de trabalho .
- Cards (Cartões): Contêm informações detalhadas sobre cada tarefa, incluindo descrições, prazos, membros responsáveis, etiquetas e anexos .
- Lists (Listas): Agrupam cartões em colunas que representam diferentes estágios do fluxo de trabalho (ex: "A Fazer", "Em Progresso", "Concluído") .
- Power-Ups: Integrações com ferramentas externas como Slack, Google Drive, GitHub, Microsoft Teams e calendários, ampliando as funcionalidades nativas .
- Colaboração: Permite comentários, menções de membros, notificações em tempo real e atribuição de responsáveis, funcionando como um canal de comunicação integrado .

Estudos acadêmicos indicam que, apesar da facilidade de uso, a adoção efetiva do Trello depende do tempo disponível dos profissionais para explorar suas funcionalidades e integrar a ferramenta ao fluxo diário de trabalho . Profissionais com maior familiaridade tecnológica tendem a implementar rapidamente o uso da ferramenta, enquanto outros justificam falta de tempo hábil para exploração adequada .

8.4 MySQL Workbench

O MySQL Workbench é uma ferramenta oficial de desenvolvimento e administração de bancos de dados MySQL, desenvolvida e mantida pela Oracle Corporation. A plataforma oferece um ambiente integrado para modelagem de dados, desenvolvimento SQL e administração de servidores MySQL.

Principais Funcionalidades:

- **Modelagem de Dados (Data Modeling):** Permite a criação visual de Diagramas Entidade-Relacionamento (DER) com suporte a modelos conceituais e físicos, conversão automática entre modelos e engenharia reversa de esquemas existentes. **Editor SQL (SQL Development):** Oferece um ambiente completo para escrita, execução e depuração de consultas SQL, com autocompletar sensível ao contexto, syntax highlighting colorido, formatação automática de código, reutilização de snippets e histórico de execução.
- **Administração de Servidor (Server Administration):** Inclui ferramentas para configuração de instâncias de servidor, gerenciamento de usuários e privilégios, backup e restauração de dados, inspeção de logs de auditoria e visualização de saúde do banco de dados.
- **Engenharia Reversa e Forward Engineering:** Permite importar esquemas de bancos de dados existentes e gerar diagramas ERD automaticamente, além de converter modelos visuais em scripts SQL prontos para implantação.
- **Migração de Dados:** Suporta migração de bancos de dados de outros SGBDs (como Microsoft SQL Server, Microsoft Access, Sybase ASE e PostgreSQL) para MySQL, com conversão de tabelas, objetos e dados. **Ferramentas de Performance:** Disponibiliza dashboard de performance, relatórios de identificação de hotspots de I/O, análise de consultas SQL de alto custo e Visual Explain Plan para otimização de queries.

O MySQL Workbench está disponível em duas edições: Community Edition (gratuita) e Standard Edition (paga, com recursos enterprise adicionais como geração de documentação de banco de dados).

8.5 GitHub

O GitHub é a principal plataforma de hospedagem de repositórios Git e colaboração em desenvolvimento de software no mundo, adquirida pela Microsoft em 2018. A ferramenta funciona como uma rede social para desenvolvedores, permitindo versionamento, revisão de código e integração contínua.^{docs.oracle+1}

Principais Recursos:

- **Repositórios:** Armazenam todo o código-fonte, histórico completo de versões, documentação e issues de projetos, podendo ser públicos (visíveis a todos) ou privados (acesso restrito).
- **Commits:** Registram alterações específicas no código com mensagens descritivas, autor, timestamp e hash único, criando um histórico rastreável e imutável de modificações.
- **Branches:** Permitem criar ramificações isoladas do código principal para desenvolvimento de novas funcionalidades, experimentação ou correções de bugs sem afetar a branch main/master.
- **Pull Requests:** Mecanismo de revisão de código onde colaboradores solicitam a incorporação de suas alterações ao repositório principal, permitindo discussão, comentários linha a linha, aprovação e merge controlado.
- **GitHub Copilot:** Assistente de IA baseado em modelos de linguagem que sugere trechos de código, completa funções e gera documentação em tempo real, acelerando o desenvolvimento.
- **GitHub Actions:** Ferramenta de automação para CI/CD (Integração Contínua/Entrega Contínua), permitindo criar workflows personalizados para teste automatizado, build, deploy e outras tarefas baseadas em eventos.
- **Code Review:** Sistema integrado de revisão de código com comentários inline, sugestões de alteração, aprovação obrigatória e integração com ferramentas de análise estática.

9 ÓRGÃOS ENVOLVIDOS

Este projeto foi desenvolvido com o apoio das instituições e organizações participantes, sem as quais não seria possível sua realização. Os órgãos envolvidos são:

1. Centro de Educação Tecnológica do Amazonas – CETAM, instituição responsável pela formação acadêmica dos alunos do Curso Técnico em Desenvolvimento de Sistemas, fornecendo orientação metodológica, acompanhamento pedagógico e supervisão das atividades desenvolvidas durante a Prática Profissional Supervisionada (PPS).

2. Amazon River Transportes, empresa criada para representar uma organização do setor de transporte fluvial e gestão de embarcações. A empresa foi utilizada como base para o levantamento dos requisitos, modelagem dos processos operacionais e desenvolvimento do sistema SisFrota, fornecendo o contexto organizacional necessário para a elaboração do projeto.

3. Equipe de Desenvolvimento do Projeto SisFrota, composta pelos alunos responsáveis pela análise, modelagem, implementação, testes e documentação do sistema, atuando em todas as etapas de execução do projeto, desde o levantamento dos requisitos até a validação final da solução proposta.

4. Orientador/Instrutor da Prática Profissional Supervisionada (PPS), responsável pelo acompanhamento técnico e acadêmico do projeto, realizando a orientação das atividades, avaliação das entregas e validação dos resultados alcançados durante o desenvolvimento do sistema.

Essas instituições e participantes contribuíram de forma direta para a execução do projeto, possibilitando a elaboração de uma solução voltada ao gerenciamento de embarcações, tripulações, viagens e manutenções, atendendo aos objetivos propostos pelo SisFrota.

10 MECANISMOS E NORMAS DE EXECUÇÃO

Para assegurar o correto desenvolvimento, a conformidade legal e a operacionalização do SISFROTA – Sistema Desktop de Controle de Frota e Manutenção de Embarcações, foram estabelecidos mecanismos operacionais, convênios acadêmicos e normas regulatórias específicos para a execução do projeto:

10.1 Convênios e Parcerias Institucionais

- **Parceria Acadêmico-Operacional:** O projeto foi desenvolvido sob o amparo do convênio pedagógico entre o Centro de Educação Tecnológica do Amazonas (CETAM / Escola Galiléia) e a empresa fictícia *Amazon River Transportes Fluviais Ltda.* A parceria possibilitou o estudo de caso prático e a coleta de

dados operacionais reais para o levantamento dos Requisitos Funcionais (RF) e Não Funcionais (RNF) do sistema.

- Uso de Instalações e Laboratórios: A execução prática do projeto utilizou a infraestrutura física e tecnológica do Laboratório de Informática - 02 do CETAM (Escola Galiléia), situado na Rua Marginal Esquerda, nº 28, Bairro Galiléia, Manaus/AM.

10.2 Regulamentações Náuticas e Órgãos Normatizadores

A modelagem do sistema e as suas regras de negócio foram estruturadas para atender às normas dos seguintes órgãos reguladores:

- Capitania dos Portos / Marinha do Brasil: Conformidade com as exigências de documentação obrigatória das embarcações, incluindo o acompanhamento da validade da Caderneta de Inscrição e Registro (CIR) dos tripulantes habilitados (Mestre, Arrais, Piloto e Condutor Fluvial) e o Certificado de Trafegabilidade/Navegabilidade (RF03, RF10, RN03).
- Órgãos de Licenciamento Ambiental e de Segurança: Integração de alertas para o vencimento do seguro obrigatório DPEM e das licenças ambientais necessárias para o transporte hidroviário na Região Amazônica (RF10).

10.3 Normas Técnicas e Padrões de Desenvolvimento de Software

- Arquitetura e Modelagem de Software: Adoção dos padrões da *Object Management Group* (OMG) para a criação dos diagramas UML (Casos de Uso, Classes e Atividades) via ferramenta Astah.
- Modelagem de Banco de Dados Relacional: Estruturação das entidades do banco de dados seguindo a metodologia de Carlos A. Heuser, validada através das ferramentas brModelo e MySQL Workbench.
- Gestão Ágil e Versionamento: Utilização da metodologia Kanban para o gerenciamento visual de tarefas via Trello e aplicação de boas práticas de controle de versão e commits estruturados por meio do GitHub.
- Lei Geral de Proteção de Dados (LGPD - Lei nº 13.709/2018): Armazenamento de dados sensíveis de usuários e tripulantes (como senhas criptografadas via

Hash e CPF) garantindo restrição de acesso por perfis (Administrador, Operador e Técnico) e integridade das informações (RNF05, RNF07).

11 ORÇAMENTO

Tabela 17 - Orçamento

Ordem	Descrição dos Materiais/Serviços	Unidade	Quantidade	Valor Unitário(R\$)	Valor Parcial(R\$)
IMPRESSÃO DO RELATÓRIO					
1	Impressão do Relatório Técnico Final	Folha	162	0,20	32,40
Subtotal(R\$)					32,40
ENCADERNAÇÃO					
2	Encadernação espiral com capa transparente e contracapa	Unidade	1	15,00	10,00
Subtotal(R\$)					10,00
VALOR TOTAL DO PROJETO(R\$)					42,40

12 CRONOGRAMA

Tabela 18 - Cronograma

ATIVIDADES	SEMANA 1	SEMANA 2	SEMANA 3
Definição da temática, problema e justificativa técnica do SISFROTA	X		
Definição da área de abrangência, objetivos e metas do projeto	X		
Levantamento de Requisitos (RF e RNF) e aplicação do formulário de pesquisa	X		
Modelagem do Banco de Dados (brModelo e MySQL Workbench)	X		
Elaboração da Metodologia e Diagramas UML no Astah (Casos de Uso, Classes e Atividades)		X	
Construção do Dicionário de Dados e criação dos Scripts SQL		X	
Gestão das tarefas no Trello (Kanban) e versionamento no GitHub	X	X	
Descrição das ferramentas utilizadas e normas de execução do projeto		X	
Redação e consolidação do Relatório Técnico Final			X
Entrega do Relatório Técnico Final e Apresentação Oral do Projeto			X

Fonte: Autoria Própria

13 CONSIDERAÇÕES FINAIS

A realização deste projeto permitiu o desenvolvimento de uma solução tecnológica voltada para o gerenciamento de embarcações e atividades operacionais da empresa fictícia Amazon River Transportes, por meio da criação do sistema SISFROTA. O projeto surgiu da necessidade de demonstrar como a utilização de sistemas informatizados pode contribuir para a organização e o controle de informações relacionadas à gestão de frotas náuticas, substituindo processos manuais por procedimentos mais eficientes e seguros.

Durante o desenvolvimento do trabalho, foram aplicados os conhecimentos adquiridos ao longo do Curso Técnico em Desenvolvimento de Sistemas, abrangendo etapas como levantamento de requisitos, análise do problema, modelagem de banco de dados, elaboração de diagramas, desenvolvimento da interface e implementação das funcionalidades do sistema. Essas atividades permitiram transformar uma necessidade organizacional em uma solução tecnológica estruturada e documentada de acordo com as práticas estudadas durante a formação técnica.

Os resultados obtidos demonstram que o SISFROTA, atende aos objetivos inicialmente propostos, oferecendo recursos para o cadastro e gerenciamento de embarcações, controle de tripulantes, registro de viagens e acompanhamento das atividades de manutenção. A centralização dessas informações em um único sistema possibilita maior organização dos dados, reduzindo a dependência de registros físicos e contribuindo para a melhoria dos processos administrativos e operacionais da empresa.

Além dos benefícios relacionados ao controle das informações, o sistema desenvolvido apresenta potencial para auxiliar na tomada de decisões, uma vez que permite acesso mais rápido aos registros e ao histórico das operações realizadas. Dessa forma, a utilização de uma ferramenta informatizada pode favorecer o planejamento das atividades da empresa, contribuindo para a redução de falhas operacionais e para o aumento da eficiência na gestão da frota.

Outro aspecto importante observado durante a execução do projeto foi a relevância da fase de planejamento para o sucesso do desenvolvimento do sistema.

A elaboração dos diagramas e da documentação técnica permitiu compreender melhor os processos envolvidos e definir de forma adequada os requisitos necessários para a construção da solução. Esse processo evidenciou a importância da análise prévia dos problemas e da organização das informações antes da implementação das funcionalidades.

No âmbito acadêmico, a elaboração do SisFrota representou uma experiência significativa de aprendizagem, permitindo colocar em prática diversos conteúdos estudados ao longo do curso. O desenvolvimento do projeto exigiu pesquisa, planejamento, organização e aplicação de conhecimentos técnicos, contribuindo para o fortalecimento das competências relacionadas ao desenvolvimento de software e à resolução de problemas por meio da tecnologia.

A experiência adquirida durante a Prática Profissional Supervisionada também possibilitou uma visão mais próxima da realidade encontrada no mercado de trabalho, demonstrando a importância do desenvolvimento de sistemas como ferramenta de apoio à gestão organizacional. Além disso, reforçou a necessidade de atualização constante dos profissionais da área de tecnologia diante das demandas cada vez maiores por soluções digitais eficientes e inovadoras.

Embora os objetivos do projeto tenham sido alcançados, o sistema pode receber futuras melhorias visando ampliar suas funcionalidades e torná-lo ainda mais completo. Entre as possíveis evoluções destacam-se a implementação de relatórios gerenciais, controle de abastecimento das embarcações, notificações automáticas para manutenções preventivas, geração de indicadores de desempenho e integração com outras tecnologias de monitoramento. Essas melhorias poderiam aumentar ainda mais o valor agregado da solução para a organização.

Dessa forma, conclui-se que o desenvolvimento do SISFROTA, foi fundamental para demonstrar a aplicação prática dos conhecimentos obtidos durante o curso técnico, resultando em uma solução capaz de atender às necessidades propostas para o gerenciamento de uma frota náutica. O projeto alcançou seus objetivos, proporcionando uma ferramenta organizada, funcional e alinhada aos princípios de gestão da informação, evidenciando o papel da tecnologia como elemento estratégico para a modernização dos processos organizacionais.

Por fim, destaca-se que os códigos-fonte desenvolvidos durante a implementação do SISFROTA encontram-se disponibilizados nos anexos deste relatório, servindo como documentação complementar da solução proposta. Esses arquivos permitem a análise da estrutura do sistema, das funcionalidades implementadas e das tecnologias utilizadas no desenvolvimento, podendo também servir como base para futuras manutenções, atualizações ou aprimoramentos da aplicação

14 REFERÊNCIAS BIBLIOGRÁFICAS

- ASTAH. Astah Professional Features. 2026. Disponível em: <https://astah.net/products/astah-professional/>. Acesso em: 26 ago. 2026.
- ASTAH. Astah Professional User Guide. 2025. Disponível em: <https://astah.net/support/astah-pro/user-guide/>. Acesso em: 26 ago. 2026.
- ASTAH. Astah UML - High-performance UML diagramming. 2026. Disponível em: <https://astah.net/products/astah-uml/>. Acesso em: 26 ago. 2026.
- ASTAH. Astah UML Reference Manual. 2022. Disponível em: https://astah.net/wp-content/uploads/2022/03/ReferenceManual-astah-UML_professional.pdf. Acesso em: 26 ago. 2026.
- ASTAH. Astah UML Support. 2025. Disponível em: <https://astah.net/support/astah-uml/>. Acesso em: 26 ago. 2026.
- CLEVERENCE. MySQL Workbench: Complete Guide to Data Modeling, SQL Development. 2026. Disponível em: <https://www.cleverence.com/articles/oracle-documentation/mysql-workbench-5937/>. Acesso em: 26 ago. 2026.
- GITHUB. GitHub Docs. 2026. Disponível em: <https://docs.github.com/>. Acesso em: 26 ago. 2026.
- GITHUB. GitHub Features. 2026. Disponível em: <https://github.com/features>. Acesso em: 26 ago. 2026.
- JOURNAL UNIPDU. ISSN 2527-3671. Disponível em: <https://journal.unipdu.ac.id/index.php/teknologi/article/download/3115/1521/9726>. Acesso em: 26 ago. 2026.
- MENNA, Otávio Soares; RAMOS, Leonardo Antônio. Portabilização da ferramenta de modelagem de banco de dados brModelo. Repositório UFSC, 2007. Disponível em: <https://repositorio.ufsc.br/handle/123456789/184582>. Acesso em: 26 ago. 2026.
- ORACLE. MySQL Workbench Features. Disponível em: <https://www.mysql.com/products/workbench/features.html>. Acesso em: 26 ago. 2026.
- ORACLE. MySQL Workbench Introduction. Disponível em: <https://docs.oracle.com/cd/E19078-01/mysql/mysql-workbench/wb-intro.html>. Acesso em: 26 ago. 2026.
- ORACLE. MySQL Workbench Overview. Disponível em: <https://www.mysql.com/products/workbench/>. Acesso em: 26 ago. 2026.
- ORACLE. MySQL Workbench Reference Manual (PDF). Disponível em: https://docs.oracle.com/cd/E17952_01/workbench-en/workbench-en.pdf. Acesso em: 26 ago. 2026.

ORACLE. MySQL Workbench Tutorial (PDF). Disponível em: <https://www.oracle.com/technetwork/community/developer-day/mysql-hands-on-lab-403032.pdf>. Acesso em: 26 ago. 2026.

ORACLE. MySQL Workbench - What Is New. Disponível em: <https://dev.mysql.com/doc/workbench/en/wb-what-is-new.html>. Acesso em: 26 ago. 2026.

SBBd. brModelo: Quinze Anos! 2020. Disponível em: https://sbbd.org.br/2020/wp-content/uploads/sites/13/2020/09/brModelo___Distinguished_Demo_SBBd_2020-2-Ferramenta-brModelo-Quinze-Anos.pdf. Acesso em: 26 ago. 2026.

SIS4. Ferramenta de Modelagem de Bancos de Dados Relacionais brModelo (PDF). Disponível em: <https://sis4.com/brModelo.pdf>. Acesso em: 26 ago. 2026.

STAJANOVIC, de V. et al. Trello. PMC, 2021. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9327605/>. Acesso em: 26 ago. 2026.

UFSC. brModeloWeb: Ferramenta Web para Ensino e Modelagem de Banco de Dados. Repositório UFSC, 2016. Disponível em: <https://repositorio.ufsc.br/handle/123456789/171508>. Acesso em: 26 ago. 2026.

15 ANEXOS

Código-Fonte do SISFROTA desenvolvido em Linguagem Java

15.1 Tela de Login

```
package view;
import controller.UsuarioController;
import model.Usuario;
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.geom.Path2D;

public class TelaLogin extends JFrame {

    // Paleta de Cores (Design Tokens)
    private static final Color BG_APP      = new Color(241, 245, 249); // slate-100
    private static final Color CARD_BG     = Color.WHITE;
    private static final Color TEXT_TITLE  = new Color(15, 23, 42); // slate-900
    private static final Color TEXT_MUTED  = new Color(100, 116, 139); // slate-500
    private static final Color ACCENT      = new Color(99, 102, 241); // indigo-500
    private static final Color ACCENT_HOVER = new Color(79, 70, 229); // indigo-600
    private static final Color INPUT_BORDER = new Color(203, 213, 225); // slate-300
    private static final Font FONT_BASE    = new Font("Segoe UI", Font.PLAIN, 13);
    private static final Font FONT_BOLD    = new Font("Segoe UI", Font.BOLD, 13);

    private JTextField txtEmail;
    private JPasswordField txtSenha;
    private final UsuarioController controller;

    public TelaLogin() {
        controller = new UsuarioController();
        setTitle("Gestão Náutica - Autenticação");
        setSize(420, 530);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setResizable(false);

        initComponents();
    }

    private void initComponents() {
        JPanel pnlFundo = new JPanel(new GridBagLayout());
        pnlFundo.setBackground(BG_APP);

        // Card Principal
        JPanel pnlCard = new JPanel() {
            @Override
            protected void paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D) g.create();
                g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

                // Sombra suave
                g2.setColor(new Color(15, 23, 42, 15));
                g2.fillRoundRect(2, 4, getWidth() - 4, getHeight() - 4, 20, 20);
            }
        };
    }
}
```

```

        // Fundo Branco
        g2.setColor(CARD_BG);
        g2.fillRoundRect(0, 0, getWidth() - 2, getHeight() - 4, 20, 20);

        g2.dispose();
    }
};
pnlCard.setOpaque(false);
pnlCard.setPreferredSize(new Dimension(340, 420));
pnlCard.setLayout(new BoxLayout(pnlCard, BoxLayout.Y_AXIS));
pnlCard.setBorder(BorderFactory.createEmptyBorder(30, 35, 35, 35));

// Ícone da Âncora
JLabel lblIcone = new JLabel(" ⚓ ", SwingConstants.CENTER);
lblIcone.setFont(new Font("Segoe UI Emoji", Font.PLAIN, 42));
lblIcone.setForeground(ACCENT);
lblIcone.setAlignmentX(Component.CENTER_ALIGNMENT);
lblIcone.setBorder(BorderFactory.createEmptyBorder(10, 0, 5, 0));

JLabel lblTitulo = new JLabel("Gestão Náutica");
lblTitulo.setFont(new Font("Segoe UI", Font.BOLD, 22));
lblTitulo.setForeground(TEXT_TITLE);
lblTitulo.setAlignmentX(Component.CENTER_ALIGNMENT);

JLabel lblSubtitulo = new JLabel("Acesso restrito ao sistema");
lblSubtitulo.setFont(FONT_BASE);
lblSubtitulo.setForeground(TEXT_MUTED);
lblSubtitulo.setAlignmentX(Component.CENTER_ALIGNMENT);

// Campos de entrada
txtEmail = criarCampoTexto();
JPanel pnlSenhaContainer = criarCampoSenhaComOlho();

txtSenha.addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER) {
            efetuarLogin(null);
        }
    }
});

// Botões de ação
JButton btnEntrar = criarBotaoPrimario("Entrar no Sistema");
btnEntrar.addActionListener(this::efetuarLogin);

JButton btnSair = criarBotaoSecundario("Encerrar");
btnSair.addActionListener(e -> System.exit(0));

// Montagem do Card
pnlCard.add(lblIcone);
pnlCard.add(Box.createVerticalStrut(5));
pnlCard.add(lblTitulo);
pnlCard.add(Box.createVerticalStrut(4));
pnlCard.add(lblSubtitulo);
pnlCard.add(Box.createVerticalStrut(30));

pnlCard.add(criarLabelCampo("E-mail corporativo"));
pnlCard.add(Box.createVerticalStrut(5));

```

```

        pnlCard.add(txtEmail);
        pnlCard.add(Box.createVerticalStrut(15));

        pnlCard.add(criarLabelCampo("Senha de acesso"));
        pnlCard.add(Box.createVerticalStrut(5));
        pnlCard.add(pnlSenhaContainer);
        pnlCard.add(Box.createVerticalStrut(28));

        pnlCard.add(btnEntrar);
        pnlCard.add(Box.createVerticalStrut(10));
        pnlCard.add(btnSair);

        pnlFundo.add(pnlCard);
        add(pnlFundo);
    }

    private JLabel criarLabelCampo(String texto) {
        JLabel label = new JLabel(texto);
        label.setFont(new Font("Segoe UI", Font.BOLD, 11));
        label.setForeground(TEXT_MUTED);
        label.setAlignmentX(Component.CENTER_ALIGNMENT);
        label.setMaximumSize(new Dimension(Short.MAX_VALUE, label.getPreferredSize().height));
        return label;
    }

    private JTextField criarCampoTexto() {
        JTextField campo = new JTextField();
        estilizarCampoFormulario(campo);
        return campo;
    }

    private JPanel criarCampoSenhaComOlho() {
        JPanel painel = new JPanel(new BorderLayout());
        painel.setBackground(new Color(248, 250, 252));
        painel.setBorder(BorderFactory.createLineBorder(INPUT_BORDER, 1));
        painel.setMaximumSize(new Dimension(Short.MAX_VALUE, 38));
        painel.setPreferredSize(new Dimension(200, 38));

        txtSenha = new JPasswordField();
        txtSenha.setFont(FONT_BASE);
        txtSenha.setForeground(TEXT_TITLE);
        txtSenha.setBackground(new Color(248, 250, 252));
        txtSenha.setCaretColor(ACCENT);
        txtSenha.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 5));

        char echoCharPadrao = txtSenha.getEchoChar();

        JToggleButton btnOlho = new JToggleButton() {
            @Override
            protected void paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D) g.create();
                g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                    RenderingHints.VALUE_ANTIALIAS_ON);

                int w = getWidth();
                int h = getHeight();
                int cx = w / 2;
                int cy = h / 2;

                g2.setColor(isSelected() ? ACCENT : TEXT_MUTED);
            }
        };
    }

```

```

        g2.setStroke(new BasicStroke(1.8f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
        // Desenho do contorno do olho (amêndoa)
        Path2D eye = new Path2D.Float();
        eye.moveTo(cx - 9, cy);
        eye.quadTo(cx, cy - 6, cx + 9, cy);
        eye.quadTo(cx, cy + 6, cx - 9, cy);
        g2.draw(eye);

        // Pupila central
        g2.fillOval(cx - 2, cy - 2, 5, 5);

        // Risco diagonal quando a senha estiver ocultada (Estilo Nubank)
        if (!isSelected()) {
            g2.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
            g2.drawLine(cx - 9, cy + 7, cx + 9, cy - 7);
        }

        g2.dispose();
    }
};

btnOlho.setPreferredSize(new Dimension(38, 38));
btnOlho.setFocusPainted(false);
btnOlho.setContentAreaFilled(false);
btnOlho.setBorderPainted(false);
btnOlho.setCursor(new Cursor(Cursor.HAND_CURSOR));
btnOlho.setToolTipText("Mostrar/Ocultar Senha");

btnOlho.addActionListener(e -> {
    if (btnOlho.isSelected()) {
        txtSenha.setEchoChar((char) 0); // Exibe a senha
    } else {
        txtSenha.setEchoChar(echoCharPadrao); // Oculta a senha
    }
    btnOlho.repaint();
});
painel.add(txtSenha, BorderLayout.CENTER);
painel.add(btnOlho, BorderLayout.EAST);

return painel;
}

private void estilizarCampoFormulario(JTextField campo) {
    campo.setFont(FONT_BASE);
    campo.setForeground(TEXT_TITLE);
    campo.setBackground(new Color(248, 250, 252));
    campo.setCaretColor(ACCENT);
    campo.setMaximumSize(new Dimension(Short.MAX_VALUE, 38));
    campo.setPreferredSize(new Dimension(200, 38));
    campo.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(INPUT_BORDER, 1),
        BorderFactory.createEmptyBorder(5, 10, 5, 10)
    ));
}

private JButton criarBotaoPrimario(String texto) {
    JButton btn = new JButton(texto) {
        @Override

```



```

        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(getModel().isRollover() ? ACCENT_HOVER : ACCENT);
            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 10, 10);
            g2.dispose();
            super.paintComponent(g);
        }
    };
    btn.setFont(FONT_BOLD);
    btn.setForeground(Color.WHITE);
    btn.setContentAreaFilled(false);
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
    btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btn.setMaximumSize(new Dimension(Short.MAX_VALUE, 40));
    btn.setPreferredSize(new Dimension(200, 40));
    btn.setAlignmentX(Component.CENTER_ALIGNMENT);
    return btn;
}

private JButton criarBotaoSecundario(String texto) {
    JButton btn = new JButton(texto);
    btn.setFont(FONT_BASE);
    btn.setForeground(TEXT_MUTED);
    btn.setContentAreaFilled(false);
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
    btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btn.setMaximumSize(new Dimension(Short.MAX_VALUE, 35));
    btn.setAlignmentX(Component.CENTER_ALIGNMENT);
    return btn;
}

private void efetuarLogin(ActionEvent e) {
    String email = txtEmail.getText();
    String senha = new String(txtSenha.getPassword());

    if (email.trim().isEmpty() || senha.trim().isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Por favor, preencha todos os campos.",
            "Atenção", JOptionPane.WARNING_MESSAGE);
        return;
    }

    Usuario usuario = controller.autenticar(email, senha);

    if (usuario != null) {
        new TelaPrincipal(usuario).setVisible(true);
        this.dispose();
    } else {
        JOptionPane.showMessageDialog(this,
            "E-mail ou senha incorretos!",
            "Erro de Autenticação", JOptionPane.ERROR_MESSAGE);
        txtSenha.setText("");
        txtSenha.requestFocus();
    }
}
}

```

```

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new TelaLogin().setVisible(true));
    }
}

```

15.2 Tela Dashboard Operador

```

package view;
import controller.*;
import dao.*;
import dto.ConsultaHorarioDTO;
import model.*;

import javax.swing.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.Date;
import java.util.List;

public class TelaDashboardOperador extends JFrame {

    private final DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");

    // Controllers e DAOs
    private final ViagemController viagemController = new ViagemController();
    private final ViagemDAO viagemDAO = new ViagemDAO();
    private final RotaDAO rotaDAO = new RotaDAO();
    private final AbastecimentoController abastecimentoController = new AbastecimentoController();
    private final AbastecimentoDAO abastecimentoDAO = new AbastecimentoDAO();
    private final IncidenteController incidenteController = new IncidenteController();
    private final IncidenteDAO incidenteDAO = new IncidenteDAO();
    private final EmbarcacaoDAO embarcacaoDAO = new EmbarcacaoDAO();
    private final ConsultaHorariosController consultaController = new ConsultaHorariosController();

    // Componentes - Aba Viagens
    private JComboBox<Embarcacao> cbViagemEmbarcacao;
    private JComboBox<Tripulante> cbViagemComandante;
    private JComboBox<String> cbViagemDestino;
    private JTextField txtViagemPassageiros;
    private JSpinner spViagemData;
    private JSpinner spViagemHora;
    private JTable tblViagens;
    private DefaultTableModel modelViagens;

    // Componentes - Aba Abastecimento
    private JComboBox<Embarcacao> cbAbastEmbarcacao;
    private JSpinner spAbastData;
    private JTextField txtAbastLitros;
    private JTextField txtAbastValorTotal;
    private JComboBox<Fornecedor> cbAbastFornecedor;
    private JTable tblAbastecimentos;

```

```

private DefaultTableModel modelAbastecimentos;

// Componentes - Aba Incidente
private JComboBox<Embarcacao> cbIncidEmbarcacao;
private JSpinner spIncidDataHora;
private JTextField txtIncidViagemId;
private JComboBox<String> cbIncidGravidade;
private JTextArea txtIncidDescricao;
private JTable tblIncidentes;
private DefaultTableModel modelIncidentes;
private JTextArea txtDetalheIncidente;

// Componentes - Aba Consulta Horários
private JComboBox<String> cbConsultaEmbarcacao;
private JComboBox<String> cbConsultaComandante;
private JComboBox<String> cbConsultaDestino;
private JTable tblConsulta;
private DefaultTableModel modelConsulta;

public TelaDashboardOperador() {
    setTitle("Painel Integrado do Operador - Gestao Nautica");
    setSize(1150, 750);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(null);

    JTabbedPane tabbedPane = new JTabbedPane();
    tabbedPane.setFont(new Font("SansSerif", Font.BOLD, 12));

    tabbedPane.addTab("Registrar Viagem", criarPainelViagens());
    tabbedPane.addTab("Registrar Abastecimento", criarPainelAbastecimento());
    tabbedPane.addTab("Central de Incidentes", criarPainelIncidente());
    tabbedPane.addTab("Consultar Horários", criarPainelConsultaHorarios());

    add(tabbedPane);
    carregarTodosOsDados();
}

// =====
// ABA 1: REGISTRAR VIAGEM
// =====
private JPanel criarPainelViagens() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    JPanel pnlForm = new JPanel(new GridBagLayout());
    pnlForm.setBorder(BorderFactory.createTitledBorder(" Formulario de Nova Viagem "));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(6, 6, 6, 6);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
    gbc.gridx = 1; cbViagemEmbarcacao = new JComboBox<>();
    pnlForm.add(cbViagemEmbarcacao, gbc);

    gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Comandante:"), gbc);
    gbc.gridx = 3; cbViagemComandante = new JComboBox<>();
    pnlForm.add(cbViagemComandante, gbc);

    gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Rota / Destino:"), gbc);
    gbc.gridx = 1; cbViagemDestino = new JComboBox<>(); pnlForm.add(cbViagemDestino, gbc);
}

```

```

gbc.gridx = 2; gbc.gridy = 1; pnlForm.add(new JLabel("Qtd. Passageiros:"), gbc);
gbc.gridx = 3; txtViagemPassageiros = new JTextField(10); pnlForm.add(txtViagemPassageiros,
gbc);

gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Data Partida:"), gbc);
gbc.gridx = 1;
spViagemData = new JSpinner(new SpinnerDateModel());
spViagemData.setEditor(new JSpinner.DateEditor(spViagemData, "dd/MM/yyyy"));
pnlForm.add(spViagemData, gbc);

gbc.gridx = 2; gbc.gridy = 2; pnlForm.add(new JLabel("Horario Partida:"), gbc);
gbc.gridx = 3;
spViagemHora = new JSpinner(new SpinnerDateModel());
spViagemHora.setEditor(new JSpinner.DateEditor(spViagemHora, "HH:mm"));
pnlForm.add(spViagemHora, gbc);

gbc.gridx = 3; gbc.gridy = 3;
JButton btnSalvar = new JButton("Registrar Viagem");
btnSalvar.setFont(new Font("SansSerif", Font.BOLD, 12));
btnSalvar.addActionListener(e -> salvarViagem());
pnlForm.add(btnSalvar, gbc);

panel.add(pnlForm, BorderLayout.NORTH);

modelViagens = new DefaultTableModel(new String[]{"ID", "Embarcação", "Comandante",
"Destino", "Passageiros", "Partida", "Chegada", "Status"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }
};
tblViagens = new JTable(modelViagens);
ajustarLargurasViagens();

panel.add(new JScrollPane(tblViagens), BorderLayout.CENTER);

JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
JButton btnConcluir = new JButton("Concluir Viagem");
JButton btnCancelar = new JButton("Cancelar Viagem");
JButton btnExcluir = new JButton("Excluir Viagem");

btnConcluir.addActionListener(e -> alterarStatusViagem(true));
btnCancelar.addActionListener(e -> alterarStatusViagem(false));
btnExcluir.addActionListener(e -> excluirViagem());

pnlAcoes.add(btnConcluir);
pnlAcoes.add(btnCancelar);
pnlAcoes.add(btnExcluir);
panel.add(pnlAcoes, BorderLayout.SOUTH);

return panel;
}

private void ajustarLargurasViagens() {
    tblViagens.getColumnModel().getColumn(0).setPreferredWidth(50); // ID
    tblViagens.getColumnModel().getColumn(1).setPreferredWidth(160); // Embarcação
    tblViagens.getColumnModel().getColumn(2).setPreferredWidth(160); // Comandante
    tblViagens.getColumnModel().getColumn(3).setPreferredWidth(160); // Destino
    tblViagens.getColumnModel().getColumn(4).setPreferredWidth(90); // Passageiros
    tblViagens.getColumnModel().getColumn(5).setPreferredWidth(130); // Partida
    tblViagens.getColumnModel().getColumn(6).setPreferredWidth(130); // Chegada
    tblViagens.getColumnModel().getColumn(7).setPreferredWidth(110); // Status

```

```

}

private void salvarViagem() {
    try {
        Embarcacao emb = (Embarcacao) cbViagemEmbarcacao.getSelectedItemAt();
        Tripulante trip = (Tripulante) cbViagemComandante.getSelectedItemAt();
        String destino = (String) cbViagemDestino.getSelectedItemAt();

        if (emb == null || trip == null || destino == null || destino.trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Selecione todos os campos obrigatorios.", "Aviso",
                JOptionPane.WARNING_MESSAGE);
            return;
        }

        int passageiros = Integer.parseInt(txtViagemPassageiros.getText().trim());
        Date dPartida = (Date) spViagemData.getValue();
        Date dHora = (Date) spViagemHora.getValue();

        LocalDate lDate = dPartida.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();
        LocalTime lTime = dHora.toInstant().atZone(ZoneId.systemDefault()).toLocalTime();
        LocalDateTime partida = LocalDateTime.of(lDate, lTime);

        Viagem v = new Viagem(emb.getId(), trip.getId(), destino, partida, passageiros);
        String res = viagemController.registrarViagem(v, emb);

        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "Viagem registrada com sucesso!", "Sucesso",
                JOptionPane.INFORMATION_MESSAGE);
            txtViagemPassageiros.setText("");
            carregarTabelaViagens();
            carregarTabelaConsulta();
        } else {
            JOptionPane.showMessageDialog(this, res, "Alerta", JOptionPane.WARNING_MESSAGE);
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Informe um numero de passageiros valido.", "Erro",
            JOptionPane.ERROR_MESSAGE);
    }
}

private void alterarStatusViagem(boolean concluir) {
    int row = tblViagens.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma viagem na tabela.");
        return;
    }
    int idViagem = (int) modelViagens.getValueAt(row, 0);
    String status = (String) modelViagens.getValueAt(row, 7);

    if (!"EM_ANDAMENTO".equalsIgnoreCase(status)) {
        JOptionPane.showMessageDialog(this, "Apenas viagens 'EM_ANDAMENTO' podem ter seu
            status alterado.");
        return;
    }

    boolean ok = concluir ? viagemDAO.finalizarViagem(idViagem) :
        viagemDAO.cancelarViagem(idViagem);
    if (ok) {
        JOptionPane.showMessageDialog(this, "Status da viagem atualizado!");
        carregarTabelaViagens();
    }
}

```

```

        carregarTabelaConsulta();
    }
}

private void excluirViagem() {
    int row = tblViagens.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma viagem para excluir.");
        return;
    }
    int idViagem = (int) modelViagens.getValueAt(row, 0);
    if (JOptionPane.showConfirmDialog(this, "Confirma a exclusao da viagem ID " + idViagem + "?",
    "Atencao", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (viagemController.excluirViagem(idViagem)) {
            JOptionPane.showMessageDialog(this, "Viagem excluida!");
            carregarTabelaViagens();
            carregarTabelaConsulta();
        } else {
            JOptionPane.showMessageDialog(this, "Falha ao excluir a viagem.", "Erro",
            JOptionPane.ERROR_MESSAGE);
        }
    }
}

// =====
// ABA 2: REGISTRAR ABASTECIMENTO (COM EXCLUSÃO)
// =====
private JPanel criarPainelAbastecimento() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    JPanel pnlForm = new JPanel(new GridBagLayout());
    pnlForm.setBorder(BorderFactory.createTitledBorder(" Novo Lançamento de Abastecimento "));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(6, 6, 6, 6);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
    gbc.gridx = 1; cbAbastEmbarcacao = new JComboBox<>(); pnlForm.add(cbAbastEmbarcacao, gbc);

    gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Data Abastecimento:"), gbc);
    gbc.gridx = 3;
    spAbastData = new JSpinner(new SpinnerDateModel());
    spAbastData.setEditor(new JSpinner.DateEditor(spAbastData, "dd/MM/yyyy"));
    pnlForm.add(spAbastData, gbc);

    gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Litros (L):"), gbc);
    gbc.gridx = 1; txtAbastLitros = new JTextField(10); pnlForm.add(txtAbastLitros, gbc);

    gbc.gridx = 2; gbc.gridy = 1; pnlForm.add(new JLabel("Valor Total (R$):"), gbc);
    gbc.gridx = 3; txtAbastValorTotal = new JTextField(10); pnlForm.add(txtAbastValorTotal, gbc);

    gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Posto / Fornecedor:"), gbc);
    gbc.gridx = 1; cbAbastFornecedor = new JComboBox<>(); pnlForm.add(cbAbastFornecedor, gbc);

    gbc.gridx = 3; gbc.gridy = 2;
    JButton btnSalvar = new JButton("Salvar Abastecimento");
    btnSalvar.setFont(new Font("SansSerif", Font.BOLD, 12));
    btnSalvar.addActionListener(e -> salvarAbastecimento());
    pnlForm.add(btnSalvar, gbc);
}

```

```

panel.add(pnlForm, BorderLayout.NORTH);

modelAbastecimentos = new DefaultTableModel(new String[]{"ID", "Embarcação", "Data", "Litros",
"Valor Total (R$)", "Fornecedor"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }
};
tblAbastecimentos = new JTable(modelAbastecimentos);
ajustarLargurasAbastecimento();

panel.add(new JScrollPane(tblAbastecimentos), BorderLayout.CENTER);

// Paine de Ações de Abastecimento
JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
JButton btnExcluir = new JButton("Excluir Abastecimento");
btnExcluir.addActionListener(e -> excluirAbastecimento());
pnlAcoes.add(btnExcluir);

panel.add(pnlAcoes, BorderLayout.SOUTH);

return panel;
}

private void ajustarLargurasAbastecimento() {
    tblAbastecimentos.getColumnModel().getColumn(0).setPreferredWidth(60); // ID
    tblAbastecimentos.getColumnModel().getColumn(1).setPreferredWidth(200); // Embarcação
    tblAbastecimentos.getColumnModel().getColumn(2).setPreferredWidth(120); // Data
    tblAbastecimentos.getColumnModel().getColumn(3).setPreferredWidth(110); // Litros
    tblAbastecimentos.getColumnModel().getColumn(4).setPreferredWidth(130); // Valor Total
    tblAbastecimentos.getColumnModel().getColumn(5).setPreferredWidth(220); // Fornecedor
}

private void salvarAbastecimento() {
    try {
        Embarcacao emb = (Embarcacao) cbAbastEmbarcacao.getSelectedItem();
        Fornecedor forn = (Fornecedor) cbAbastFornecedor.getSelectedItem();

        if (emb == null || forn == null) {
            JOptionPane.showMessageDialog(this, "Selecione uma embarcação e um fornecedor.",
"Aviso", JOptionPane.WARNING_MESSAGE);
            return;
        }

        java.util.Date dateVal = (java.util.Date) spAbastData.getValue();
        java.time.LocalDate data = dateVal.toInstant().atZone(java.time.ZoneId.systemDefault()).toLocalDate();
        double litros = Double.parseDouble(txtAbastLitros.getText().trim().replace(",", "."));
        double valorTotal = Double.parseDouble(txtAbastValorTotal.getText().trim().replace(",", "."));

        Abastecimento a = new Abastecimento(emb.getId(), data, litros, valorTotal, forn.getNome());
        String res = abastecimentoController.registrar(a);

        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "Abastecimento registrado com sucesso!");
            txtAbastLitros.setText("");
            txtAbastValorTotal.setText("");
            carregarTabelaAbastecimentos();
        } else {
            JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
        }
    } catch (Exception ex) {

```

```

        JOptionPane.showMessageDialog(this, "Verifique os valores informados para Litros e Valor
Total.", "Erro de Digitação", JOptionPane.ERROR_MESSAGE);
    }
}

private void ajustarLargurasIncidentes() {
    tblIncidentes.getColumnModel().getColumn(0).setPreferredWidth(50); // ID
    tblIncidentes.getColumnModel().getColumn(1).setPreferredWidth(170); // Embarcação
    tblIncidentes.getColumnModel().getColumn(2).setPreferredWidth(140); // Data
    tblIncidentes.getColumnModel().getColumn(3).setPreferredWidth(380); // Descrição
    tblIncidentes.getColumnModel().getColumn(4).setPreferredWidth(90); // Gravidade
    tblIncidentes.getColumnModel().getColumn(5).setPreferredWidth(100); // Status
}

private void salvarIncidente() {
    Embarcacao emb = (Embarcacao) cbIncidEmbarcacao.getSelectedItem();
    Integer idEmbarcacao = (emb != null) ? emb.getId() : null;
    String viagemIdStr = txtIncidViagemId.getText();
    String gravidade = (String) cbIncidGravidade.getSelectedItem();
    String descricao = txtIncidDescricao.getText();

    if (descricao == null || descricao.trim().length() < 5) {
        JOptionPane.showMessageDialog(this, "Descreva detalhadamente o incidente antes de enviar.",
"Aviso", JOptionPane.WARNING_MESSAGE);
        return;
    }

    String res = incidenteController.registrarIncidente(idEmbarcacao, viagemIdStr, gravidade,
descricao);

    if ("OK".equals(res)) {
        JOptionPane.showMessageDialog(this, "Incidente reportado com sucesso!", "Enviado",
JOptionPane.INFORMATION_MESSAGE);
        txtIncidViagemId.setText("");
        txtIncidDescricao.setText("");
        spIncidDataHora.setValue(new java.util.Date());
        carregarTabelaIncidentes();
    } else {
        JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
    }
}

private void excluirAbastecimento() {
    int row = tblAbastecimentos.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione um abastecimento na tabela para excluir.");
        return;
    }
    int idAbast = (int) modelAbastecimentos.getValueAt(row, 0);

    if (JOptionPane.showConfirmDialog(this, "Confirma a exclusao do abastecimento ID " + idAbast +
"?", "Atenção", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (abastecimentoDAO.excluir(idAbast)) {
            JOptionPane.showMessageDialog(this, "Abastecimento excluído com sucesso!");
            carregarTabelaAbastecimentos();
        } else {
            JOptionPane.showMessageDialog(this, "Falha ao excluir abastecimento.", "Erro",
JOptionPane.ERROR_MESSAGE);
        }
    }
}

```



```

}

// =====
// ABA 3: REGISTRAR INCIDENTE (COM EXCLUSÃO)
// =====
private JPanel criarPainelIncidente() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    // Formulário
    JPanel pnlForm = new JPanel(new GridBagLayout());
    pnlForm.setBorder(BorderFactory.createTitledBorder(" Novo Incidente Operacional "));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
    gbc.gridx = 1; cbIncidEmbarcacao = new JComboBox<>(); pnlForm.add(cbIncidEmbarcacao, gbc);

    gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Data / Hora Ocorrência:"), gbc);
    gbc.gridx = 3;
    splIncidDataHora = new JSpinner(new SpinnerDateModel());
    splIncidDataHora.setEditor(new JSpinner.DateEditor(splIncidDataHora, "dd/MM/yyyy HH:mm"));
    pnlForm.add(splIncidDataHora, gbc);

    gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Gravidade:"), gbc);
    gbc.gridx = 1; cbIncidGravidade = new JComboBox<>(new String[]{"BAIXA", "MEDIA", "ALTA",
"CRITICA"}); pnlForm.add(cbIncidGravidade, gbc);

    gbc.gridx = 2; gbc.gridy = 1; pnlForm.add(new JLabel("Cod. Viagem (Opcional):"), gbc);
    gbc.gridx = 3; txtIncidViagemId = new JTextField(10); pnlForm.add(txtIncidViagemId, gbc);

    gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Descrição:"), gbc);
    gbc.gridx = 1; gbc.gridwidth = 3;
    txtIncidDescricao = new JTextArea(2, 40);
    txtIncidDescricao.setLineWrap(true);
    txtIncidDescricao.setWrapStyleWord(true);
    pnlForm.add(new JScrollPane(txtIncidDescricao), gbc);

    gbc.gridx = 3; gbc.gridy = 3; gbc.gridwidth = 1;
    JButton btnSalvar = new JButton("Reportar Incidente ao Adm");
    btnSalvar.setFont(new Font("SansSerif", Font.BOLD, 12));
    btnSalvar.addActionListener(e -> salvarIncidente());
    pnlForm.add(btnSalvar, gbc);

    panel.add(pnlForm, BorderLayout.NORTH);

    // Tabela
    modelIncidentes = new DefaultTableModel(new String[]{"ID", "Embarcação", "Data Incidente",
"Descrição", "Gravidade", "Status"}, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };
    tblIncidentes = new JTable(modelIncidentes);
    ajustarLargurasIncidentes();

    // Reativa o import estilizando a coluna de Gravidade por cor
    tblIncidentes.getColumnModel().getColumn(4).setCellRenderer(new DefaultTableCellRenderer() {
        @Override

```

```

    public Component getTableCellRendererComponent(JTable table, Object value, boolean isSelected,
        boolean hasFocus, int row, int column) {
        Component c = super.getTableCellRendererComponent(table, value, isSelected, hasFocus, row,
            column);
        String grav = (value != null) ? value.toString() : "";
        if ("CRITICA".equalsIgnoreCase(grav) || "ALTA".equalsIgnoreCase(grav)) {
            c.setForeground(Color.RED);
            c.setFont(c.getFont().deriveFont(Font.BOLD));
        } else if ("MEDIA".equalsIgnoreCase(grav)) {
            c.setForeground(new Color(200, 100, 0));
        } else {
            c.setForeground(new Color(0, 120, 0));
        }
        return c;
    }
};

```

```

JScrollPane scrollTabela = new JScrollPane(tblIncidentes);
scrollTabela.setPreferredSize(new Dimension(800, 180));

```

```

// Paine de Detalhes
JPanel pnlDetalhes = new JPanel(new BorderLayout());
pnlDetalhes.setBorder(BorderFactory.createTitledBorder("  Detalhes  Completos  do  Registro
Selecionado "));

```

```

txtDetalheIncidente = new JTextArea(4, 50);
txtDetalheIncidente.setEditable(false);
txtDetalheIncidente.setLineWrap(true);
txtDetalheIncidente.setWrapStyleWord(true);
txtDetalheIncidente.setBackground(new Color(245, 245, 245));
txtDetalheIncidente.setFont(new Font("Monospaced", Font.PLAIN, 12));
pnlDetalhes.add(new JScrollPane(txtDetalheIncidente), BorderLayout.CENTER);

```

```

tblIncidentes.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) {
        int row = tblIncidentes.getSelectedRow();
        if (row != -1) {
            txtDetalheIncidente.setText(
                "ID REGISTRO : " + modelIncidentes.getValueAt(row, 0) + "\n" +
                "EMBARCAÇÃO : " + modelIncidentes.getValueAt(row, 1) + "\n" +
                "DATA/HORA : " + modelIncidentes.getValueAt(row, 2) + "\n" +
                "GRAVIDADE : " + modelIncidentes.getValueAt(row, 4) + "\n" +
                "STATUS : " + modelIncidentes.getValueAt(row, 5) + "\n" +
                "-----\n" +
                "DESCRIÇÃO COMPLETA DO INCIDENTE:\n" + modelIncidentes.getValueAt(row, 3)
            );
            txtDetalheIncidente.setCaretPosition(0);
        }
    }
});

```

```

JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, scrollTabela, pnlDetalhes);
splitPane.setResizeWeight(0.6);
panel.add(splitPane, BorderLayout.CENTER);

```

```

// Paine de Ações de Incidentes
JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
JButton btnExcluir = new JButton("Excluir Incidente");
btnExcluir.addActionListener(e -> excluirIncidente());
pnlAcoes.add(btnExcluir);

```

```

panel.add(pnlAcoes, BorderLayout.SOUTH);

return panel;
}

private void excluirIncidente() {
    int row = tblIncidentes.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione um incidente na tabela para excluir.");
        return;
    }
    int idIncidente = (int) modelIncidentes.getValueAt(row, 0);

    if (JOptionPane.showConfirmDialog(this, "Confirma a exclusao do incidente ID " + idIncidente + "?",
    "Atencao", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (incidenteDAO.excluir(idIncidente)) {
            JOptionPane.showMessageDialog(this, "Incidente excluido com sucesso!");
            txtDetalheIncidente.setText("");
            carregarTabelaIncidentes();
        } else {
            JOptionPane.showMessageDialog(this, "Falha ao excluir incidente.", "Erro",
            JOptionPane.ERROR_MESSAGE);
        }
    }
}

// =====
// ABA 4: CONSULTAR HORÁRIOS
// =====
private JPanel criarPainelConsultaHorarios() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    JPanel pnlFiltros = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 5));
    pnlFiltros.setBorder(BorderFactory.createTitledBorder(" Filtros Rápidos "));

    pnlFiltros.add(new JLabel("Embarcação:"));
    cbConsultaEmbarcacao = new JComboBox<>();
    pnlFiltros.add(cbConsultaEmbarcacao);

    pnlFiltros.add(new JLabel("Comandante:"));
    cbConsultaComandante = new JComboBox<>();
    pnlFiltros.add(cbConsultaComandante);

    pnlFiltros.add(new JLabel("Destino:"));
    cbConsultaDestino = new JComboBox<>();
    pnlFiltros.add(cbConsultaDestino);

    JButton btnFiltrar = new JButton("Filtrar");
    btnFiltrar.addActionListener(e -> carregarTabelaConsulta());
    pnlFiltros.add(btnFiltrar);

    JButton btnLimpar = new JButton("Resetar Filtros");
    btnLimpar.addActionListener(e -> {
        cbConsultaEmbarcacao.setSelectedIndex(0);
        cbConsultaComandante.setSelectedIndex(0);
        cbConsultaDestino.setSelectedIndex(0);
        carregarTabelaConsulta();
    });
}

```

```

        pnlFiltros.add(btnLimpar);

        panel.add(pnlFiltros, BorderLayout.NORTH);

        modelConsulta = new DefaultTableModel(new String[]{"ID", "Embarcação", "Comandante",
"Destino", "Partida", "Chegada", "Passageiros", "Status"}, 0) {
            @Override public boolean isCellEditable(int r, int c) { return false; }
        };
        tblConsulta = new JTable(modelConsulta);
        ajustarLargurasConsulta();

        panel.add(new JScrollPane(tblConsulta), BorderLayout.CENTER);

        return panel;
    }

    private void ajustarLargurasConsulta() {
        tblConsulta.getColumnModel().getColumn(0).setPreferredWidth(50); // ID
        tblConsulta.getColumnModel().getColumn(1).setPreferredWidth(160); // Embarcação
        tblConsulta.getColumnModel().getColumn(2).setPreferredWidth(160); // Comandante
        tblConsulta.getColumnModel().getColumn(3).setPreferredWidth(160); // Destino
        tblConsulta.getColumnModel().getColumn(4).setPreferredWidth(130); // Partida
        tblConsulta.getColumnModel().getColumn(5).setPreferredWidth(130); // Chegada
        tblConsulta.getColumnModel().getColumn(6).setPreferredWidth(90); // Passageiros
        tblConsulta.getColumnModel().getColumn(7).setPreferredWidth(110); // Status
    }

    // =====
    // CARREGAMENTO DE DADOS E TABELAS
    // =====
    private void carregarTodosOsDados() {
        carregarCombos();
        carregarTabelaViagens();
        carregarTabelaAbastecimentos();
        carregarTabelaIncidentes();
        carregarTabelaConsulta();
    }

    private void carregarCombos() {
        List<Embarcacao> embarcacoes = embarcacaoDAO.listarTodas();
        List<Tripulante> tripulantes = new TripulanteDAO().listarTodos();
        List<String> rotas = rotaDAO.listarTodas();
        List<Fornecedor> fornecedores = new FornecedorDAO().listarTodos();

        cbViagemEmbarcacao.removeAllItems();
        cbAbastEmbarcacao.removeAllItems();
        cbIncidEmbarcacao.removeAllItems();
        cbViagemComandante.removeAllItems();
        cbViagemDestino.removeAllItems();
        cbAbastFornecedor.removeAllItems();

        embarcacoes.forEach(e -> {
            cbViagemEmbarcacao.addItem(e);
            cbAbastEmbarcacao.addItem(e);
            cbIncidEmbarcacao.addItem(e);
        });

        tripulantes.forEach(cbViagemComandante::addItem);
        rotas.forEach(cbViagemDestino::addItem);
        fornecedores.forEach(cbAbastFornecedor::addItem);
    }

```

```

// Combos da Consulta
cbConsultaEmbarcacao.removeAllItems();
cbConsultaComandante.removeAllItems();
cbConsultaDestino.removeAllItems();

consultaController.obterEmbarcacoes().forEach(cbConsultaEmbarcacao::addItem);
consultaController.obterComandantes().forEach(cbConsultaComandante::addItem);
consultaController.obterDestinos().forEach(cbConsultaDestino::addItem);
}

private void carregarTabelaViagens() {
    modelViagens.setRowCount(0);
    for (Viagem v : viagemDAO.listarTodas()) {
        modelViagens.addRow(new Object[]{
            v.getId(),
            v.getNomeEmbarcacao(),
            v.getNomeComandante(),
            v.getRotaDestino(),
            v.getQuantidadePassageiros(),
            v.getDataHoraPartida() != null ? v.getDataHoraPartida().format(dateTimeFormatter) : "-",
            v.getDataHoraChegada() != null ? v.getDataHoraChegada().format(dateTimeFormatter) : "-",
            v.getStatus()
        });
    }
}

private void carregarTabelaAbastecimentos() {
    modelAbastecimentos.setRowCount(0);
    for (AbastecimentoDAO.AbastecimentoDTO a : abastecimentoDAO.listarTodos()) {
        modelAbastecimentos.addRow(new Object[]{
            a.getId(),
            a.getNomeEmbarcacao(),
            a.getData().format(dateFormatter),
            String.format("%.2f L", a.getLitros()),
            String.format("R$ %.2f", a.getValorTotal()),
            a.getFornecedor()
        });
    }
}

private void carregarTabelaIncidentes() {
    modelIncidentes.setRowCount(0);
    List<Object[]> lista = incidenteDAO.listarTodosParaTabela();

    for (Object[] linha : lista) {
        java.time.LocalDateTime data = (java.time.LocalDateTime) linha[2];
        String dataStr = (data != null) ? data.format(dateTimeFormatter) : "-";

        modelIncidentes.addRow(new Object[]{
            linha[0], // ID
            linha[1], // Embarcação
            dataStr, // Data Incidente
            linha[3], // Descrição
            linha[4], // Gravidade
            linha[5] // Status
        });
    }
}

private void carregarTabelaConsulta() {

```

```

modelConsulta.setRowCount(0);
String selEmb = (String) cbConsultaEmbarcacao.getSelectedItem();
String selCom = (String) cbConsultaComandante.getSelectedItem();
String selDes = (String) cbConsultaDestino.getSelectedItem();

List<ConsultaHorarioDTO> lista = consultaController.buscarHorarios(selEmb, selCom, selDes);
for (ConsultaHorarioDTO dto : lista) {
    String partidaStr = dto.getDataHoraPartida() != null ?
dto.getDataHoraPartida().format(dateTimeFormatter) : "-";
    String chegadaStr = dto.getDataHoraChegada() != null ?
dto.getDataHoraChegada().format(dateTimeFormatter) : "Em Transito";

    modelConsulta.addRow(new Object[]{
        dto.getIdViagem(),
        dto.getEmbarcacao(),
        dto.getComandante(),
        dto.getRotaDestino(),
        partidaStr,
        chegadaStr,
        dto.getQuantidadePassageiros(),
        dto.getStatus()
    });
}
}
}

```

15.3 Tela Dashboard Responsável Técnico

```

package view;
import controller.TecnicoController;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.text.NumberFormat;
import java.util.List;
import java.util.Locale;

public class TelaDashboardTecnico extends JFrame {

    private final TecnicoController controller = new TecnicoController();

    private JTable tblOS, tblAlertas, tblHistorico;
    private DefaultTableModel modelOS, modelAlertas, modelHistorico;
    private JTextArea txtDetalhesOS, txtDetalhesAlertas, txtDetalhesHistorico;

    public TelaDashboardTecnico() {
        setTitle("Dashboard Técnico - Ordens de Serviço & Manutenção Fleet");
        setSize(1000, 680);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.addTab("Ordens de Serviço Ativas", criarPainelOS());
        tabbedPane.addTab("Alertas de Horímetro (Preventivas)", criarPainelAlertas());
        tabbedPane.addTab("Histórico de Motores & Peças", criarPainelHistorico());

        add(tabbedPane);
        carregarDados();
    }
}

```

```

}

private JPanel criarPainelOS() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    modelOS = new DefaultTableModel(new String[]{"ID OS", "Embarcação", "Tipo", "Descrição do
Serviço", "Data Agendada", "Custo Est. (R$)", "Status", "ID_EMB"}, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };

    tblOS = new JTable(modelOS);
    ajustarColunasTabelaOS();

    txtDetalhesOS = criarAreaTextoDetalhes();
    tblOS.getSelectionModel().addListSelectionListener(e -> {
        int row = tblOS.getSelectedRow();
        if (row != -1) {
            txtDetalhesOS.setText((String) modelOS.getValueAt(row, 3));
        } else {
            txtDetalhesOS.setText("");
        }
    });

    JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, new JScrollPane(tblOS),
criarPainelDetalhesContainer(" Detalhes do Serviço Selecionado (OS) ", txtDetalhesOS));
    splitPane.setResizeWeight(0.65);

    panel.add(splitPane, BorderLayout.CENTER);

    JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));

    JButton btnNovaOS = new JButton("+ Abrir Nova OS");
    btnNovaOS.setBackground(new Color(41, 128, 185));
    btnNovaOS.setForeground(Color.WHITE);
    btnNovaOS.addActionListener(e -> abrirDialogoNovaOS());

    JButton btnConcluir = new JButton("Concluir Serviço (Baixa de OS)");
    btnConcluir.setBackground(new Color(39, 174, 96));
    btnConcluir.setForeground(Color.WHITE);
    btnConcluir.addActionListener(e -> abrirDialogoConclusao());

    pnlAcoes.add(btnNovaOS);
    pnlAcoes.add(btnConcluir);
    panel.add(pnlAcoes, BorderLayout.SOUTH);

    return panel;
}

private JPanel criarPainelAlertas() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    modelAlertas = new DefaultTableModel(new String[]{"ID Emb.", "Embarcação", "Horímetro Atual
(h)", "Horímetro Alvo (h)", "Serviço Previsto"}, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };

    tblAlertas = new JTable(modelAlertas);
    tblAlertas.getColumnModel().getColumn(0).setPreferredWidth(60);

```

```

tblAlertas.getColumnModel().getColumn(1).setPreferredWidth(180);
tblAlertas.getColumnModel().getColumn(2).setPreferredWidth(130);
tblAlertas.getColumnModel().getColumn(3).setPreferredWidth(130);
tblAlertas.getColumnModel().getColumn(4).setPreferredWidth(400);

txtDetalhesAlertas = criarAreaTextoDetalhes();
tblAlertas.getSelectionModel().addListSelectionListener(e -> {
    int row = tblAlertas.getSelectedRow();
    if (row != -1) {
        txtDetalhesAlertas.setText((String) modelAlertas.getValueAt(row, 4));
    } else {
        txtDetalhesAlertas.setText("");
    }
});

JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, new
JScrollPane(tblAlertas), criarPainelDetalhesContainer(" Detalhes do Alerta Preventivo ",
txtDetalhesAlertas));
splitPane.setResizeWeight(0.65);

panel.add(splitPane, BorderLayout.CENTER);
return panel;
}

private JPanel criarPainelHistorico() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    modelHistorico = new DefaultTableModel(new String[]{"ID OS", "Embarcação", "Tipo", "Serviço
Realizado / Troca de Peças", "Data Execução", "Custo (R$)", 0} {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    });

    tblHistorico = new JTable(modelHistorico);
    tblHistorico.getColumnModel().getColumn(0).setPreferredWidth(50);
    tblHistorico.getColumnModel().getColumn(1).setPreferredWidth(160);
    tblHistorico.getColumnModel().getColumn(2).setPreferredWidth(100);
    tblHistorico.getColumnModel().getColumn(3).setPreferredWidth(380);
    tblHistorico.getColumnModel().getColumn(4).setPreferredWidth(110);
    tblHistorico.getColumnModel().getColumn(5).setPreferredWidth(100);

    txtDetalhesHistorico = criarAreaTextoDetalhes();
    tblHistorico.getSelectionModel().addListSelectionListener(e -> {
        int row = tblHistorico.getSelectedRow();
        if (row != -1) {
            txtDetalhesHistorico.setText((String) modelHistorico.getValueAt(row, 3));
        } else {
            txtDetalhesHistorico.setText("");
        }
    });

    JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, new
JScrollPane(tblHistorico), criarPainelDetalhesContainer(" Histórico Detalhado Mecânico ",
txtDetalhesHistorico));
    splitPane.setResizeWeight(0.65);

    panel.add(splitPane, BorderLayout.CENTER);
    return panel;
}

```



```

private JTextArea criarAreaTextoDetalhes() {
    JTextArea area = new JTextArea(4, 50);
    area.setEditable(false);
    area.setLineWrap(true);
    area.setWrapStyleWord(true);
    area.setFont(new Font("SansSerif", Font.PLAIN, 13));
    return area;
}

private JPanel criarPainelDetalhesContainer(String titulo, JTextArea areaTexto) {
    JPanel pnl = new JPanel(new BorderLayout());
    pnl.setBorder(BorderFactory.createTitledBorder(titulo));
    pnl.add(new JScrollPane(areaTexto), BorderLayout.CENTER);
    return pnl;
}

private void ajustarColunasTabelaOS() {
    tblOS.getColumnModel().getColumn(0).setPreferredWidth(50);
    tblOS.getColumnModel().getColumn(1).setPreferredWidth(150);
    tblOS.getColumnModel().getColumn(2).setPreferredWidth(90);
    tblOS.getColumnModel().getColumn(3).setPreferredWidth(320);
    tblOS.getColumnModel().getColumn(4).setPreferredWidth(100);
    tblOS.getColumnModel().getColumn(5).setPreferredWidth(100);
    tblOS.getColumnModel().getColumn(6).setPreferredWidth(110);

    tblOS.getColumnModel().getColumn(7).setMinWidth(0);
    tblOS.getColumnModel().getColumn(7).setMaxWidth(0);
    tblOS.getColumnModel().getColumn(7).setWidth(0);
}

private void carregarDados() {
    modelOS.setRowCount(0);
    NumberFormat nf = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));

    for (Object[] row : controller.obterOSAbertas()) {
        modelOS.addRow(new Object[]{
            row[0], row[1], row[2], row[3], row[4],
            nf.format((double) row[5]), row[6], row[7]
        });
    }

    modelAlertas.setRowCount(0);
    for (Object[] row : controller.obterAlertasHorimetro()) {
        modelAlertas.addRow(row);
    }

    modelHistorico.setRowCount(0);
    for (Object[] row : controller.obterHistoricoMotores()) {
        modelHistorico.addRow(new Object[]{
            row[0], row[1], row[2], row[3], row[4],
            nf.format((double) row[5])
        });
    }
}

private void abrirDialogoNovaOS() {
    List<Object[]> embarcacoes = controller.obterEmbarcacoes();
    if (embarcacoes.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Nenhuma embarcação cadastrada.");
        return;
    }
}

```

```

    }

    JComboBox<String> cbEmbarcacao = new JComboBox<>();
    for (Object[] emb : embarcacoes) {
        cbEmbarcacao.addItem(emb[0] + " - " + emb[1]);
    }

    JComboBox<String> cbTipo = new JComboBox<>(new String[]{"PREVENTIVA", "CORRETIVA"});
    JTextArea txtDescricao = new JTextArea(4, 25);
    txtDescricao.setLineWrap(true);
    txtDescricao.setWrapStyleWord(true);

    JTextField txtHorimetro = new JTextField();
    JSpinner spDataAgendamento = new JSpinner(new SpinnerDateModel());
    spDataAgendamento.setEditor(new JSpinner.DateEditor(spDataAgendamento, "dd/MM/yyyy"));

    JPanel panel = new JPanel(new GridLayout(0, 1, 5, 5));
    panel.add(new JLabel("Embarcação:"));
    panel.add(cbEmbarcacao);
    panel.add(new JLabel("Tipo de Manutenção:"));
    panel.add(cbTipo);
    panel.add(new JLabel("Descrição Detalhada do Serviço / Peças:"));
    panel.add(new JScrollPane(txtDescricao));
    panel.add(new JLabel("Horímetro Agendado (Gatilho Preventiva) [Opcional]:"));
    panel.add(txtHorimetro);
    panel.add(new JLabel("Data Prevista para Execução:"));
    panel.add(spDataAgendamento);

    int result = JOptionPane.showConfirmDialog(this, panel, "Abrir Nova Ordem de Serviço",
        JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

    if (result == JOptionPane.OK_OPTION) {
        int selectedIndex = cbEmbarcacao.getSelectedIndex();
        int idEmbarcacao = (int) embarcacoes.get(selectedIndex)[0];
        String tipo = (String) cbTipo.getSelectedItem();
        String desc = txtDescricao.getText();
        String horimetroStr = txtHorimetro.getText();
        java.util.Date dataAgendamento = (java.util.Date) spDataAgendamento.getValue();

        String res = controller.criarNovaOS(idEmbarcacao, tipo, desc, horimetroStr, dataAgendamento);
        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "Ordem de Serviço criada com sucesso!");
            carregarDados();
        } else {
            JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
        }
    }
}

private void abrirDialogoConclusao() {
    int row = tblOS.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma Ordem de Serviço na tabela para concluir.");
        return;
    }

    int idOS = (int) modelOS.getValueAt(row, 0);
    String embarcacao = (String) modelOS.getValueAt(row, 1);
    int idEmbarcacao = (int) modelOS.getValueAt(row, 7);

```

```

JTextField txtHorimetro = new JTextField();
JTextField txtCustoReal = new JTextField("0.00");

JPanel panel = new JPanel(new GridLayout(0, 1, 5, 5));
panel.add(new JLabel("Embarcação: " + embarcacao));
panel.add(new JLabel("Horímetro Atual do Motor (Horas Leitura):"));
panel.add(txtHorimetro);
panel.add(new JLabel("Custo Real Final do Serviço / Peças (R$):"));
panel.add(txtCustoReal);

int result = JOptionPane.showConfirmDialog(this, panel, "Encerrar e Dar Baixa na OS #" + idOS,
JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

if (result == JOptionPane.OK_OPTION) {
    String res = controller.finalizarManutencao(idOS, idEmbarcacao, txtHorimetro.getText(),
txtCustoReal.getText());
    if ("OK".equals(res)) {
        JOptionPane.showMessageDialog(this, "OS encerrada com sucesso e cadastrada no
histórico do motor!");
        carregarDados();
    } else {
        JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
    }
}
}
}

```

15.4 Tela Gerenciar Embarcações – Administrador

```

package view;
import controller.EmbarcacaoController;
import model.Embarcacao;

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*.*;
import java.util.List;

public class TelaGerenciarEmbarcacoes extends JFrame {

    private final EmbarcacaoController controller = new EmbarcacaoController();
    private JTable tblEmbarcacoes;
    private DefaultTableModel tableModel;

    private JTextField txtId, txtNome, txtModelo, txtCapPassageiros, txtCapCarga, txtAno, txtHorimetro;
    private JComboBox<String> cbStatus;
    private JButton btnSalvar, btnLimpar, btnExcluir;

    public TelaGerenciarEmbarcacoes() {
        setTitle("Gerenciamento de Embarcações e Frota (ADM)");
        setSize(950, 600);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        add(criarFormulario(), BorderLayout.NORTH);
        add(criarTabela(), BorderLayout.CENTER);
        add(criarBarraBotoes(), BorderLayout.SOUTH);
    }
}

```

```

    carregarTabela();
}

private JPanel criarFormulario() {
    JPanel panel = new JPanel(new GridBagLayout());
    panel.setBorder(BorderFactory.createTitledBorder(" Dados da Embarcação "));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    txtId = new JTextField(); txtId.setEditable(false);
    txtNome = new JTextField(15);
    txtModelo = new JTextField(15);
    txtCapPassageiros = new JTextField("0");
    txtCapCarga = new JTextField("0.00");
    txtAno = new JTextField("2024");
    txtHorimetro = new JTextField("0");
    cbStatus = new JComboBox<>(new String[]{"ATIVA", "EM_MANUTENCAO", "INATIVA"});

    // Linha 0
    gbc.gridx = 0; gbc.gridy = 0; panel.add(new JLabel("ID:"), gbc);
    gbc.gridx = 1; panel.add(txtId, gbc);
    gbc.gridx = 2; panel.add(new JLabel("Nome:"), gbc);
    gbc.gridx = 3; panel.add(txtNome, gbc);

    // Linha 1
    gbc.gridx = 0; gbc.gridy = 1; panel.add(new JLabel("Modelo:"), gbc);
    gbc.gridx = 1; panel.add(txtModelo, gbc);
    gbc.gridx = 2; panel.add(new JLabel("Cap. Passageiros:"), gbc);
    gbc.gridx = 3; panel.add(txtCapPassageiros, gbc);

    // Linha 2
    gbc.gridx = 0; gbc.gridy = 2; panel.add(new JLabel("Cap. Carga (Ton):"), gbc);
    gbc.gridx = 1; panel.add(txtCapCarga, gbc);
    gbc.gridx = 2; panel.add(new JLabel("Ano Fabricação:"), gbc);
    gbc.gridx = 3; panel.add(txtAno, gbc);

    // Linha 3
    gbc.gridx = 0; gbc.gridy = 3; panel.add(new JLabel("Horímetro (Horas):"), gbc);
    gbc.gridx = 1; panel.add(txtHorimetro, gbc);
    gbc.gridx = 2; panel.add(new JLabel("Status:"), gbc);
    gbc.gridx = 3; panel.add(cbStatus, gbc);

    return panel;
}

private JScrollPane criarTabela() {
    tableModel = new DefaultTableModel(new String[]{
        "ID", "Nome", "Modelo", "Passageiros", "Carga (Ton)", "Ano", "Horímetro (h)", "Status"
    }, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };

    tblEmbarcacoes = new JTable(tableModel);
    tblEmbarcacoes.getSelectionModel().addListSelectionListener(e
carregarFormularioDaTabela());
    return new JScrollPane(tblEmbarcacoes);
}

```

->

```

private JPanel criarBarraBotoes() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.RIGHT));

    btnSalvar = new JButton("Salvar");
    btnLimpar = new JButton("Limpar Campos");
    btnExcluir = new JButton("Excluir");

    btnSalvar.setBackground(new Color(46, 204, 113));
    btnSalvar.setForeground(Color.WHITE);
    btnExcluir.setBackground(new Color(231, 76, 60));
    btnExcluir.setForeground(Color.WHITE);

    btnSalvar.addActionListener(e -> salvar());
    btnLimpar.addActionListener(e -> limparCampos());
    btnExcluir.addActionListener(e -> excluir());

    panel.add(btnLimpar);
    panel.add(btnExcluir);
    panel.add(btnSalvar);

    return panel;
}

private void carregarTabela() {
    tableModel.setRowCount(0);
    List<Embarcacao> lista = controller.listarEmbarcacoes();
    for (Embarcacao e : lista) {
        tableModel.addRow(new Object[]{
            e.getId(), e.getNome(), e.getModelo(), e.getCapacidadePassageiros(),
            e.getCapacidadeCargaTon(), e.getAnoFabricacao(), e.getHorimetroHoras(), e.getStatus()
        });
    }
}

private void carregarFormularioDaTabela() {
    int row = tblEmbarcacoes.getSelectedRow();
    if (row != -1) {
        txtId.setText(tableModel.getValueAt(row, 0).toString());
        txtNome.setText(tableModel.getValueAt(row, 1).toString());
        txtModelo.setText(tableModel.getValueAt(row, 2).toString());
        txtCapPassageiros.setText(tableModel.getValueAt(row, 3).toString());
        txtCapCarga.setText(tableModel.getValueAt(row, 4).toString());
        txtAno.setText(tableModel.getValueAt(row, 5).toString());
        txtHorimetro.setText(tableModel.getValueAt(row, 6).toString());
        cbStatus.setSelectedItem(tableModel.getValueAt(row, 7).toString());
    }
}

private void salvar() {
    try {
        Integer id = txtId.getText().isEmpty() ? null : Integer.parseInt(txtId.getText());
        String nome = txtNome.getText();
        String modelo = txtModelo.getText();
        int capPass = Integer.parseInt(txtCapPassageiros.getText().trim());
        double capCarga = Double.parseDouble(txtCapCarga.getText().replace(",", "."));
        int ano = Integer.parseInt(txtAno.getText().trim());
        int horimetro = Integer.parseInt(txtHorimetro.getText().trim());
        String status = (String) cbStatus.getSelectedItem();
    }
}

```

```

        String res = controller.salvarOuAtualizar(id, nome, modelo, capPass, capCarga, ano, horimetro,
status);
        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "Embarcação salva com sucesso!");
            limparCampos();
            carregarTabela();
        } else {
            JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Verifique os valores numéricos digitados.", "Erro de
Validação", JOptionPane.ERROR_MESSAGE);
    }
}

private void excluir() {
    if (txtId.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Selecione uma embarcação na tabela.");
        return;
    }
    int confirm = JOptionPane.showConfirmDialog(this, "Confirma a exclusão da embarcação?",
"Atenção", JOptionPane.YES_NO_OPTION);
    if (confirm == JOptionPane.YES_OPTION) {
        int id = Integer.parseInt(txtId.getText());
        if (controller.excluirEmbarcacao(id)) {
            JOptionPane.showMessageDialog(this, "Embarcação excluída!");
            limparCampos();
            carregarTabela();
        } else {
            JOptionPane.showMessageDialog(this, "Não foi possível excluir (pode haver vínculos ativos
no banco).", "Erro", JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void limparCampos() {
    txtId.setText("");
    txtNome.setText("");
    txtModelo.setText("");
    txtCapPassageiros.setText("0");
    txtCapCarga.setText("0.00");
    txtAno.setText("2024");
    txtHorimetro.setText("0");
    cbStatus.setSelectedIndex(0);
    tblEmbarcacoes.clearSelection();
}
}

```

15.5 Tela de Gerenciamento de Tripulações

```

package view;
import controller.TripulanteController;
import model.Tripulante;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.Date;
import java.util.List;

```

```

public class TelaGerenciarTripulacao extends JFrame {

    private final TripulanteController controller = new TripulanteController();
    private JTable tblTripulantes;
    private DefaultTableModel tableModel;
    private JTextField txtId, txtNome, txtCpf, txtCir;
    private JComboBox<String> cbCategoria, cbStatus;
    private JSpinner spDataVencimento;
    private JButton btnSalvar, btnLimpar, btnExcluir;

    public TelaGerenciarTripulacao() {
        setTitle("Gerenciamento de Tripulação (ADM)");
        setSize(950, 600);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        add(criarFormulario(), BorderLayout.NORTH);
        add(criarTabela(), BorderLayout.CENTER);
        add(criarBarraBotoes(), BorderLayout.SOUTH);

        carregarTabela();
    }

    private JPanel criarFormulario() {
        JPanel panel = new JPanel(new GridBagLayout());
        panel.setBorder(BorderFactory.createTitledBorder(" Dados do Tripulante "));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        txtId = new JTextField(); txtId.setEditable(false);
        txtNome = new JTextField(15);
        txtCpf = new JTextField(12);
        txtCir = new JTextField(12);

        cbCategoria = new JComboBox<>(new String[]{
            "PILOTO_FLUVIAL", "CONDUTOR_FLUVIAL", "ARRAIS_AMADOR", "MESTRE_AMADOR",
            "CAPITAO_AMADOR"
        });
        cbStatus = new JComboBox<>(new String[]{"DISPONIVEL", "EM_VIAGEM", "INATIVO"});
        spDataVencimento = new JSpinner(new SpinnerDateModel());
        spDataVencimento.setEditor(new JSpinner.DateEditor(spDataVencimento, "dd/MM/yyyy"));

        // Linha 0
        gbc.gridx = 0; gbc.gridy = 0; panel.add(new JLabel("ID:"), gbc);
        gbc.gridx = 1; panel.add(txtId, gbc);
        gbc.gridx = 2; panel.add(new JLabel("Nome Completo:"), gbc);
        gbc.gridx = 3; panel.add(txtNome, gbc);

        // Linha 1
        gbc.gridx = 0; gbc.gridy = 1; panel.add(new JLabel("CPF:"), gbc);
        gbc.gridx = 1; panel.add(txtCpf, gbc);
        gbc.gridx = 2; panel.add(new JLabel("Categoria Habilitação:"), gbc);
        gbc.gridx = 3; panel.add(cbCategoria, gbc);

        // Linha 2
        gbc.gridx = 0; gbc.gridy = 2; panel.add(new JLabel("Nº Registro CIR:"), gbc);
        gbc.gridx = 1; panel.add(txtCir, gbc);
        gbc.gridx = 2; panel.add(new JLabel("Vencimento CIR:"), gbc);
    }

```

```

gbc.gridx = 3; panel.add(spDataVencimento, gbc);

// Linha 3
gbc.gridx = 0; gbc.gridy = 3; panel.add(new JLabel("Status:"), gbc);
gbc.gridx = 1; panel.add(cbStatus, gbc);

return panel;
}

private JScrollPane criarTabela() {
    tableModel = new DefaultTableModel(new String[]{
        "ID", "Nome", "CPF", "Categoria", "Nº CIR", "Vencimento CIR", "Status"
    }, 0) {
        @Override public boolean isCellEditable(int r, int c) { return false; }
    };

    tblTripulantes = new JTable(tableModel);
    tblTripulantes.getSelectionModel().addListSelectionListener(e -> carregarFormularioDaTabela());
    return new JScrollPane(tblTripulantes);
}

private JPanel criarBarraBotoes() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.RIGHT));

    btnSalvar = new JButton("Salvar");
    btnLimpar = new JButton("Limpar Campos");
    btnExcluir = new JButton("Excluir");

    btnSalvar.setBackground(new Color(46, 204, 113));
    btnSalvar.setForeground(Color.WHITE);
    btnExcluir.setBackground(new Color(231, 76, 60));
    btnExcluir.setForeground(Color.WHITE);

    btnSalvar.addActionListener(e -> salvar());
    btnLimpar.addActionListener(e -> limparCampos());
    btnExcluir.addActionListener(e -> excluir());

    panel.add(btnLimpar);
    panel.add(btnExcluir);
    panel.add(btnSalvar);

    return panel;
}

private void carregarTabela() {
    tableModel.setRowCount(0);
    List<Tripulante> lista = controller.listarTripulantes();
    for (Tripulante t : lista) {
        tableModel.addRow(new Object[]{
            t.getId(), t.getNome(), t.getCpf(), t.getCategoriaHabilitacao(),
            t.getNumeroRegistroCir(), t.getDataVencimentoCir(), t.getStatus()
        });
    }
}

private void carregarFormularioDaTabela() {
    int row = tblTripulantes.getSelectedRow();
    if (row != -1) {
        txtId.setText(tableModel.getValueAt(row, 0).toString());
        txtNome.setText(tableModel.getValueAt(row, 1).toString());
    }
}

```



```

        txtCpf.setText(tableModel.getValueAt(row, 2).toString());
        cbCategoria.setSelectedItem(tableModel.getValueAt(row, 3).toString());
        txtCir.setText(tableModel.getValueAt(row, 4).toString());

        java.util.Date dataVenc = (java.util.Date) tableModel.getValueAt(row, 5);
        spDataVencimento.setValue(dataVenc);

        cbStatus.setSelectedItem(tableModel.getValueAt(row, 6).toString());
    }
}

private void salvar() {
    try {
        Integer id = txtId.getText().isEmpty() ? null : Integer.parseInt(txtId.getText());
        String nome = txtNome.getText();
        String cpf = txtCpf.getText();
        String categoria = (String) cbCategoria.getSelectedItem();
        String cir = txtCir.getText();
        java.util.Date d = (java.util.Date) spDataVencimento.getValue();
        Date dataVencimento = new Date(d.getTime());
        String status = (String) cbStatus.getSelectedItem();

        String res = controller.salvarOuAtualizar(id, nome, cpf, categoria, cir, dataVencimento, status);
        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "Tripulante salvo com sucesso!");
            limparCampos();
            carregarTabela();
        } else {
            JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Verifique os dados informados.", "Erro",
            JOptionPane.ERROR_MESSAGE);
    }
}

private void excluir() {
    if (txtId.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Selecione um tripulante na tabela.");
        return;
    }

    int confirm = JOptionPane.showConfirmDialog(this, "Confirma a exclusão do tripulante?",
        "Atenção", JOptionPane.YES_NO_OPTION);
    if (confirm == JOptionPane.YES_OPTION) {
        int id = Integer.parseInt(txtId.getText());
        if (controller.excluirTripulante(id)) {
            JOptionPane.showMessageDialog(this, "Tripulante excluído!");
            limparCampos();
            carregarTabela();
        } else {
            JOptionPane.showMessageDialog(this, "Não foi possível excluir (o tripulante pode estar
                vinculado a viagens ativas).", "Erro", JOptionPane.ERROR_MESSAGE);
        }
    }
}

private void limparCampos() {
    txtId.setText("");
    txtNome.setText("");
    txtCpf.setText("");

```

```

        txtCir.setText("");
        cbCategoria.setSelectedIndex(0);
        cbStatus.setSelectedIndex(0);
        spDataVencimento.setValue(new java.util.Date());
        tblTripulantes.clearSelection();
    }
}

```

15.6 Tela de Manutenções

```

package view;
import controller.ManutencaoController;
import dao.EmbarcacaoDAO;
import model.Embarcacao;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.Date;

public class TelaManutencoes extends JFrame {

    private final ManutencaoController controller = new ManutencaoController();
    private JTable tblIncidentes;
    private JTable tblManutencoes;
    private DefaultTableModel modelIncidentes;
    private DefaultTableModel modelManutencoes;

    // Componentes para exibição de detalhes expansíveis
    private JTextArea txtDetalheIncidente;
    private JTextArea txtDetalheOS;

    private JComboBox<Embarcacao> cbEmbarcacoes;
    private JComboBox<String> cbTipo;
    private JTextArea txtDescricao;
    private JTextField txtHorimetro;
    private JSpinner spDataAgendamento;
    private JTextField txtCusto;

    public TelaManutencoes() {
        setTitle("Gestão de Manutenções, Preventivas e Incidentes (ADM)");
        setSize(1000, 700);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.addTab("Central de Incidentes (Operadores)", criarPainelIncidentes());
        tabbedPane.addTab("Ordens de Serviço e Agendamentos", criarPainelManutencoes());

        add(tabbedPane);
        carregarDados();
    }

    private JPanel criarPainelIncidentes() {
        JPanel panel = new JPanel(new BorderLayout(10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        modelIncidentes = new DefaultTableModel(new String[]{"ID", "Embarcação", "Data Incidente",
"Descrição", "Gravidade", "Status", "ID_EMB"}, 0) {
            @Override public boolean isCellEditable(int r, int c) { return false; }
        };
    }

```

```

tblIncidentes = new JTable(modelIncidentes);

// Esconde a coluna ID_EMB que guarda a chave estrangeira
tblIncidentes.getColumnModel().getColumn(6).setMinWidth(0);
tblIncidentes.getColumnModel().getColumn(6).setMaxWidth(0);
tblIncidentes.getColumnModel().getColumn(6).setWidth(0);

JScrollPane scrollTabela = new JScrollPane(tblIncidentes);

// Paine de Detalhes do Incidente
JPanel pnlDetalhes = new JPanel(new BorderLayout());
pnlDetalhes.setBorder(BorderFactory.createTitledBorder(" Detalhes do Incidente Selecionado "));

txtDetalheIncidente = new JTextArea(4, 50);
txtDetalheIncidente.setEditable(false);
txtDetalheIncidente.setLineWrap(true);
txtDetalheIncidente.setWrapStyleWord(true);
txtDetalheIncidente.setBackground(new Color(245, 245, 245));
txtDetalheIncidente.setFont(new Font("Monospaced", Font.PLAIN, 12));
pnlDetalhes.add(new JScrollPane(txtDetalheIncidente), BorderLayout.CENTER);

tblIncidentes.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) {
        int row = tblIncidentes.getSelectedRow();
        if (row != -1) {
            txtDetalheIncidente.setText(
                "ID INCIDENTE : " + modelIncidentes.getValueAt(row, 0) + "\n" +
                "EMBARCAÇÃO : " + modelIncidentes.getValueAt(row, 1) + "\n" +
                "DATA/HORA : " + modelIncidentes.getValueAt(row, 2) + "\n" +
                "GRAVIDADE : " + modelIncidentes.getValueAt(row, 4) + "\n" +
                "STATUS : " + modelIncidentes.getValueAt(row, 5) + "\n" +
                "-----\n" +
                "DESCRIÇÃO COMPLETA:\n" + modelIncidentes.getValueAt(row, 3)
            );
            txtDetalheIncidente.setCaretPosition(0);
        }
    }
});

JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, scrollTabela, pnlDetalhes);
splitPane.setResizeWeight(0.65);

panel.add(splitPane, BorderLayout.CENTER);

JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT));
JButton btnGerarOS = new JButton("Aprovar e Gerar Ordem de Serviço (OS)");
btnGerarOS.setBackground(new Color(41, 128, 185));
btnGerarOS.setForeground(Color.WHITE);

btnGerarOS.addActionListener(e -> aprovarEGerarOS());
pnlAcoes.add(btnGerarOS);
panel.add(pnlAcoes, BorderLayout.SOUTH);

return panel;
}

private JPanel criarPainelManutencoes() {
    JPanel panel = new JPanel(new BorderLayout(10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

```

```

JPanel pnlForm = new JPanel(new GridBagLayout());
pnlForm.setBorder(BorderFactory.createTitledBorder(" Nova Ordem de Serviço / Agendamento
"));
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(4, 4, 4, 4);
gbc.fill = GridBagConstraints.HORIZONTAL;

gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
gbc.gridx = 1; cbEmbarcacoes = new JComboBox<>(); pnlForm.add(cbEmbarcacoes, gbc);

gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Tipo:"), gbc);
gbc.gridx = 3; cbTipo = new JComboBox<>(new String[]{"PREVENTIVA", "CORRETIVA"});
pnlForm.add(cbTipo, gbc);

gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Descrição Serviço:"), gbc);
gbc.gridx = 1; gbc.gridwidth = 3;
txtDescricao = new JTextArea(2, 20);
pnlForm.add(new JScrollPane(txtDescricao), gbc);
gbc.gridwidth = 1;

gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Horímetro Meta:"), gbc);
gbc.gridx = 1; txtHorimetro = new JTextField(); pnlForm.add(txtHorimetro, gbc);

gbc.gridx = 2; gbc.gridy = 2; pnlForm.add(new JLabel("Data Agendada:"), gbc);
gbc.gridx = 3;
spDataAgendamento = new JSpinner(new SpinnerDateModel());
spDataAgendamento.setEditor(new JSpinner.DateEditor(spDataAgendamento, "dd/MM/yyyy"));
pnlForm.add(spDataAgendamento, gbc);

gbc.gridx = 0; gbc.gridy = 3; pnlForm.add(new JLabel("Custo Estimado (R$):"), gbc);
gbc.gridx = 1; txtCusto = new JTextField("0.00"); pnlForm.add(txtCusto, gbc);

gbc.gridx = 3; gbc.gridy = 3;
JButton btnSalvar = new JButton("Encaminhar para Técnico");
btnSalvar.addActionListener(e -> salvarManutencao());
pnlForm.add(btnSalvar, gbc);

panel.add(pnlForm, BorderLayout.NORTH);

modelManutencoes = new DefaultTableModel(new String[]{"ID", "Embarcação", "Tipo",
"Descrição", "Horímetro", "Data Agendada", "Custo (R$)", "Status"}, 0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }
};
tblManutencoes = new JTable(modelManutencoes);

JScrollPane scrollTabela = new JScrollPane(tblManutencoes);

// Paine de Detalhes da Ordem de Serviço
JPanel pnlDetalhesOS = new JPanel(new BorderLayout());
pnlDetalhesOS.setBorder(BorderFactory.createTitledBorder(" Detalhes da Ordem de Serviço
Selecionada "));

txtDetalheOS = new JTextArea(4, 50);
txtDetalheOS.setEditable(false);
txtDetalheOS.setLineWrap(true);
txtDetalheOS.setWrapStyleWord(true);
txtDetalheOS.setBackground(new Color(245, 245, 245));
txtDetalheOS.setFont(new Font("Monospaced", Font.PLAIN, 12));
pnlDetalhesOS.add(new JScrollPane(txtDetalheOS), BorderLayout.CENTER);

```

```

tblManutencoes.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()) {
        int row = tblManutencoes.getSelectedRow();
        if (row != -1) {
            txtDetalheOS.setText(
                "ID OS          : " + modelManutencoes.getValueAt(row, 0) + "\n" +
                "EMBARCAÇÃO       : " + modelManutencoes.getValueAt(row, 1) + "\n" +
                "TIPO MANUTENÇÃO    : " + modelManutencoes.getValueAt(row, 2) + "\n" +
                "HORÍMETRO META     : " + modelManutencoes.getValueAt(row, 4) + "\n" +
                "DATA AGENDADA      : " + modelManutencoes.getValueAt(row, 5) + "\n" +
                "CUSTO ESTIMADO (R$) : " + modelManutencoes.getValueAt(row, 6) + "\n" +
                "STATUS             : " + modelManutencoes.getValueAt(row, 7) + "\n" +
                "-----\n" +
                "DESCRIÇÃO DO SERVIÇO:\n" + modelManutencoes.getValueAt(row, 3)
            );
            txtDetalheOS.setCaretPosition(0);
        }
    }
});

JSplitPane splitPane = new JSplitPane(JSplitPane.VERTICAL_SPLIT, scrollTabela,
    pnlDetalhesOS);
splitPane.setResizeWeight(0.55);

panel.add(splitPane, BorderLayout.CENTER);

JPanel pnlStatus = new JPanel(new FlowLayout(FlowLayout.RIGHT));
JButton btnEmAndamento = new JButton("Marcar 'EM ANDAMENTO'");
JButton btnCancelar = new JButton("Cancelar OS");

btnEmAndamento.addActionListener(e -> alterarStatusOS("EM_ANDAMENTO"));
btnCancelar.addActionListener(e -> alterarStatusOS("CANCELADA"));

pnlStatus.add(btnEmAndamento);
pnlStatus.add(btnCancelar);
panel.add(pnlStatus, BorderLayout.SOUTH);

return panel;
}

private void carregarDados() {
    cbEmbarcacoes.removeAllItems();
    new EmbarcacaoDAO().listarTodas().forEach(cbEmbarcacoes::addItem);

    modelIncidentes.setRowCount(0);
    for (Object[] row : controller.obterIncidentesPendentes()) {
        modelIncidentes.addRow(row);
    }

    modelManutencoes.setRowCount(0);
    for (Object[] row : controller.obterManutencoes()) {
        modelManutencoes.addRow(row);
    }

    if (txtDetalheIncidente != null) txtDetalheIncidente.setText("");
    if (txtDetalheOS != null) txtDetalheOS.setText("");
}

private void aprovarEGerarOS() {
    int row = tblIncidentes.getSelectedRow();

```

```

if (row == -1) {
    JOptionPane.showMessageDialog(this, "Selecione um incidente na tabela para aprovação.");
    return;
}

int idIncidente = (int) modelIncidentes.getValueAt(row, 0);
String desc = (String) modelIncidentes.getValueAt(row, 3);
int idEmbarcacao = (int) modelIncidentes.getValueAt(row, 6);

Date dataHoje = new Date(System.currentTimeMillis());
boolean ok = controller.converterIncidenteEmOS(idIncidente, idEmbarcacao, desc, dataHoje);

if (ok) {
    JOptionPane.showMessageDialog(this, "Ordem de Serviço gerada e encaminhada ao Técnico!");
    carregarDados();
} else {
    JOptionPane.showMessageDialog(this, "Erro ao converter incidente em OS.", "Erro",
JOptionPane.ERROR_MESSAGE);
}
}

private void salvarManutencao() {
    try {
        Embarcacao emb = (Embarcacao) cbEmbarcacoes.getSelectedItem();
        String tipo = (String) cbTipo.getSelectedItem();
        String desc = txtDescricao.getText();
        Integer horimetro = txtHorimetro.getText().trim().isEmpty() ? null :
Integer.parseInt(txtHorimetro.getText().trim());
        java.util.Date d = (java.util.Date) spDataAgendamento.getValue();
        Date dataAgendada = new Date(d.getTime());
        double custo = Double.parseDouble(txtCusto.getText().replace(",", "."));

        String res = controller.criarOrdemServico(emb.getId(), tipo, desc, horimetro, dataAgendada,
custo);
        if ("OK".equals(res)) {
            JOptionPane.showMessageDialog(this, "OS registrada e enviada para o painel do Técnico!");
            txtDescricao.setText("");
            txtHorimetro.setText("");
            txtCusto.setText("0.00");
            carregarDados();
        } else {
            JOptionPane.showMessageDialog(this, res, "Aviso", JOptionPane.WARNING_MESSAGE);
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Verifique os campos numéricos e datas.", "Erro",
JOptionPane.ERROR_MESSAGE);
    }
}

private void alterarStatusOS(String novoStatus) {
    int row = tblManutencoes.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma manutenção na tabela.");
        return;
    }
    int id = (int) modelManutencoes.getValueAt(row, 0);
    if (controller.atualizarStatusOS(id, novoStatus)) {
        JOptionPane.showMessageDialog(this, "Status atualizado!");
        carregarDados();
    }
}

```

```

    }
}

```

15.7 Tela de Abastecimento

```

package view;
import controller.AbastecimentoController;
import dao.AbastecimentoDAO;
import dao.EmbarcacaoDAO;
import dao.FornecedorDAO;
import model.Abastecimento;
import model.Embarcacao;
import model.Fornecedor;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.Date;
import java.util.List;

public class TelaAbastecimento extends JFrame {

    private JComboBox<Embarcacao> cbEmbarcacoes;
    private JSpinner spData;
    private JTextField txtLitros;
    private JTextField txtValorTotal;
    private JComboBox<Fornecedor> cbFornecedores;
    private JTable tabelaAbastecimentos;
    private DefaultTableModel modelTabela;

    private final DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    private final AbastecimentoController controller = new AbastecimentoController();
    private final AbastecimentoDAO abastecimentoDAO = new AbastecimentoDAO();

    public TelaAbastecimento() {
        setTitle("Registrar Abastecimento (RF11)");
        setSize(850, 520);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        // Painei Superior (Formulário)
        JPanel pnlForm = new JPanel(new GridBagLayout());
        pnlForm.setBorder(BorderFactory.createTitledBorder("Novo Registro de Abastecimento"));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 6, 6, 6);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Linha 0: Embarcação e Data (JSpinner)
        gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
        gbc.gridx = 1; cbEmbarcacoes = new JComboBox<>(); pnlForm.add(cbEmbarcacoes, gbc);

        gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Data do Abastecimento:"), gbc);
        gbc.gridx = 3;
        spData = new JSpinner(new SpinnerDateModel());
        spData.setEditor(new JSpinner.DateEditor(spData, "dd/MM/yyyy"));
    }

```

```

pnlForm.add(spData, gbc);

// Linha 1: Litros e Valor Total
gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Litros Abastecidos:"), gbc);
gbc.gridx = 1; txtLitros = new JTextField(10); pnlForm.add(txtLitros, gbc);

gbc.gridx = 2; gbc.gridy = 1; pnlForm.add(new JLabel("Valor Total (R$):"), gbc);
gbc.gridx = 3; txtValorTotal = new JTextField(10); pnlForm.add(txtValorTotal, gbc);

// Linha 2: Posto / Fornecedor e Botão Salvar
gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Posto / Fornecedor:"), gbc);
gbc.gridx = 1; cbFornecedores = new JComboBox<>(); pnlForm.add(cbFornecedores, gbc);

gbc.gridx = 3; gbc.gridy = 2;
JButton btnSalvar = new JButton("Salvar Abastecimento");
btnSalvar.addActionListener(e -> salvar());
pnlForm.add(btnSalvar, gbc);

add(pnlForm, BorderLayout.NORTH);

// Tabela Central (Histórico)
modelTabela = new DefaultTableModel(new String[]{"ID", "Embarcação", "Data", "Litros", "Valor
Total (R$)", "Posto / Fornecedor"}, 0);
tabelaAbastecimentos = new JTable(modelTabela);
add(new JScrollPane(tabelaAbastecimentos), BorderLayout.CENTER);

carregarCombos();
atualizarTabela();
}

private void carregarCombos() {
    cbEmbarcacoes.removeAllItems();
    cbFornecedores.removeAllItems();

    new EmbarcacaoDAO().listarTodas().forEach(cbEmbarcacoes::addItem);
    new FornecedorDAO().listarTodos().forEach(cbFornecedores::addItem);
}

private void atualizarTabela() {
    modelTabela.setRowCount(0);
    List<AbastecimentoDAO.AbastecimentoDTO> lista = abastecimentoDAO.listarTodos();
    for (AbastecimentoDAO.AbastecimentoDTO a : lista) {
        modelTabela.addRow(new Object[]{
            a.getId(),
            a.getNomeEmbarcacao(),
            a.getData().format(dateFormatter),
            String.format("%.2f L", a.getLitros()),
            String.format("R$ %.2f", a.getValorTotal()),
            a.getFornecedor()
        });
    }
}

private void salvar() {
    try {
        Embarcacao emb = (Embarcacao) cbEmbarcacoes.getSelectedItem();
        Fornecedor forn = (Fornecedor) cbFornecedores.getSelectedItem();

        if (emb == null || forn == null) {
            JOptionPane.showMessageDialog(this, "Selecione uma embarcação e um fornecedor.");
        }
    }
}

```



```

        return;
    }

    Date dateVal = (Date) spData.getValue();
    LocalDate data = dateVal.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();

    double litros = Double.parseDouble(txtLitros.getText().trim().replace(",", "."));
    double valorTotal = Double.parseDouble(txtValorTotal.getText().trim().replace(",", "."));

    Abastecimento abastecimento = new Abastecimento(emb.getId(), data, litros, valorTotal,
forn.getNome());
    String resultado = controller.registrar(abastecimento);

    if ("OK".equals(resultado)) {
        JOptionPane.showMessageDialog(this, "Abastecimento registrado com sucesso!");
        txtLitros.setText("");
        txtValorTotal.setText("");
        spData.setValue(new Date());
        atualizarTabela();
    } else {
        JOptionPane.showMessageDialog(this, resultado, "Alerta",
JOptionPane.WARNING_MESSAGE);
    }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Informe valores numéricos válidos para Litros e Valor
Total.", "Erro de Validação", JOptionPane.ERROR_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Erro ao processar dados: " + ex.getMessage(), "Erro",
JOptionPane.ERROR_MESSAGE);
    }
    }
}
}

```

15.8 Tela de Consultas de Horários

```

package view;
import controller.ConsultaHorariosController;
import dto.ConsultaHorarioDTO;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.format.DateTimeFormatter;
import java.util.List;

public class TelaConsultaHorarios extends JFrame {

    private JTable tabela;
    private DefaultTableModel model;

    private JComboBox<String> cbEmbarcacao;
    private JComboBox<String> cbComandante;
    private JComboBox<String> cbDestino;

    private final ConsultaHorariosController controller = new ConsultaHorariosController();
    private final DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");

    public TelaConsultaHorarios() {
        setTitle("Consulta de Horários e Saídas de Viagens");
        setSize(980, 520);
    }
}

```

```

setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
setLocationRelativeTo(null);
setLayout(new BorderLayout(10, 10));

// Painel de Filtros Selecionáveis
JPanel panelTopo = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10));
panelTopo.setBorder(BorderFactory.createTitledBorder(" Selecionar Filtros "));

panelTopo.add(new JLabel("Embarcação:"));
cbEmbarcacao = new JComboBox<>();
panelTopo.add(cbEmbarcacao);

panelTopo.add(new JLabel("Comandante:"));
cbComandante = new JComboBox<>();
panelTopo.add(cbComandante);

panelTopo.add(new JLabel("Destino:"));
cbDestino = new JComboBox<>();
panelTopo.add(cbDestino);

JButton btnFiltrar = new JButton("Filtrar");
btnFiltrar.addActionListener(e -> carregarTabela());
panelTopo.add(btnFiltrar);

JButton btnLimpar = new JButton("Limpar Filtros");
btnLimpar.addActionListener(e -> limparEFiltrar());
panelTopo.add(btnLimpar);

add(panelTopo, BorderLayout.NORTH);

// Tabela
String[] colunas = {"ID", "Embarcação", "Comandante", "Destino", "Partida", "Chegada",
"Passageiros", "Status"};
model = new DefaultTableModel(colunas, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};

tabela = new JTable(model);
tabela.setRowHeight(22);

JScrollPane scrollPane = new JScrollPane(tabela);
scrollPane.setBorder(BorderFactory.createEmptyBorder(0, 10, 10, 10));
add(scrollPane, BorderLayout.CENTER);

carregarOpcoesFiltros();
carregarTabela();
}

private void carregarOpcoesFiltros() {
    cbEmbarcacao.removeAllItems();
    cbComandante.removeAllItems();
    cbDestino.removeAllItems();

    controller.obterEmbarcacoes().forEach(cbEmbarcacao::addItem);
    controller.obterComandantes().forEach(cbComandante::addItem);
    controller.obterDestinos().forEach(cbDestino::addItem);
}

```

```

private void limparEFiltrar() {
    carregarOpcoesFiltros();
    cbEmbarcacao.setSelectedIndex(0);
    cbComandante.setSelectedIndex(0);
    cbDestino.setSelectedIndex(0);
    carregarTabela();
}

private void carregarTabela() {
    model.setRowCount(0);

    String selEmbarcacao = (String) cbEmbarcacao.getSelectedItem();
    String selComandante = (String) cbComandante.getSelectedItem();
    String selDestino = (String) cbDestino.getSelectedItem();

    List<ConsultaHorarioDTO> viagens = controller.buscarHorarios(selEmbarcacao, selComandante,
selDestino);

    for (ConsultaHorarioDTO v : viagens) {
        String partidaStr = v.getDataHoraPartida() != null ? v.getDataHoraPartida().format(formatter) : "-";
        String chegadaStr = v.getDataHoraChegada() != null ? v.getDataHoraChegada().format(formatter) : "Em Trânsito";

        model.addRow(new Object[]{
            v.getIdViagem(),
            v.getEmbarcacao(),
            v.getComandante(),
            v.getRotaDestino(),
            partidaStr,
            chegadaStr,
            v.getQuantidadePassageiros(),
            v.getStatus()
        });
    }
}

```

15.9 Tela de Incidentes

```

package view;
import controller.IncidenteController;
import dao.EmbarcacaoDAO;
import model.Embarcacao;

import javax.swing.*;
import java.awt.*;

public class TelaIncidente extends JFrame {

    private JComboBox<Embarcacao> cbEmbarcacoes;
    private JComboBox<String> cbGravidade;
    private JTextArea txtDescricao;
    private JTextField txtViagemId;

    private final IncidenteController incidenteController = new IncidenteController();
    private final EmbarcacaoDAO embarcacaoDAO = new EmbarcacaoDAO();

```

```

public TelaIncidente() {
    setTitle("Registrar Incidente Operacional (RF12)");
    setSize(500, 420);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(10, 10));

    JPanel formPanel = new JPanel(new GridBagLayout());
    formPanel.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.fill = GridBagConstraints.HORIZONTAL;
    gbc.insets = new Insets(6, 6, 6, 6);

    // Embarcação
    gbc.gridx = 0; gbc.gridy = 0;
    formPanel.add(new JLabel("Embarcação:"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    cbEmbarcacoes = new JComboBox<>();
    embarcacaoDAO.listarTodas().forEach(cbEmbarcacoes::addItem);
    formPanel.add(cbEmbarcacoes, gbc);

    // ID Viagem (Opcional)
    gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0;
    formPanel.add(new JLabel("Cód. Viagem (Opcional):"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    txtViagemId = new JTextField();
    formPanel.add(txtViagemId, gbc);

    // Gravidade
    gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0;
    formPanel.add(new JLabel("Gravidade:"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    cbGravidade = new JComboBox<>(new String[]{"BAIXA", "MEDIA", "ALTA", "CRITICA"});
    formPanel.add(cbGravidade, gbc);

    // Descrição
    gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0;
    formPanel.add(new JLabel("Descrição do Ocorrido:"), gbc);
    gbc.gridx = 1; gbc.weightx = 1.0;
    txtDescricao = new JTextArea(5, 20);
    txtDescricao.setLineWrap(true);
    txtDescricao.setWrapStyleWord(true);
    formPanel.add(new JScrollPane(txtDescricao), gbc);

    add(formPanel, BorderLayout.CENTER);

    JButton btnSalvar = new JButton("Registrar Incidente");
    btnSalvar.setFont(new Font("SansSerif", Font.BOLD, 12));
    btnSalvar.addActionListener(e -> salvar());

    JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    btnPanel.setBorder(BorderFactory.createEmptyBorder(0, 0, 10, 15));
    btnPanel.add(btnSalvar);
    add(btnPanel, BorderLayout.SOUTH);
}

private void salvar() {
    Embarcacao emb = (Embarcacao) cbEmbarcacoes.getSelectedItem();
    Integer idEmbarcacao = (emb != null) ? emb.getId() : null;
    String viagemIdStr = txtViagemId.getText();

```

```

String gravidade = (String) cbGravidade.getSelectedItem();
String descricao = txtDescricao.getText();

String resultado = incidenteController.registrarIncidente(idEmbarcacao, viagemIdStr, gravidade,
descricao);

if ("OK".equals(resultado)) {
    JOptionPane.showMessageDialog(this, "Incidente registrado com sucesso!", "Sucesso",
JOptionPane.INFORMATION_MESSAGE);
    dispose();
} else {
    JOptionPane.showMessageDialog(this, resultado, "Aviso",
JOptionPane.WARNING_MESSAGE);
}
}
}

```

15.10 Tela de Viagens

```

package view;
import controller.ViagemController;
import dao.EmbarcacaoDAO;
import dao.RotaDAO;
import dao.TripulanteDAO;
import dao.ViagemDAO;
import model.Embarcacao;
import model.Tripulante;
import model.Viagem;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.Date;
import java.util.List;

public class TelaViagens extends JFrame {

    private JComboBox<Embarcacao> cbEmbarcacoes;
    private JComboBox<Tripulante> cbComandantes;
    private JComboBox<String> cbDestinos;
    private JTextField txtPassageiros;
    private JSpinner spDataPartida;
    private JSpinner spHorarioPartida;
    private JTable tabelaViagens;
    private DefaultTableModel modelTabela;

    private final DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    private final ViagemController controller = new ViagemController();
    private final ViagemDAO viagemDAO = new ViagemDAO();
    private final RotaDAO rotaDAO = new RotaDAO();

    public TelaViagens() {
        setTitle("Registro e Controle de Viagens (RF05)");
        setSize(920, 580);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    }
}

```

```

setLocationRelativeTo(null);
setLayout(new BorderLayout(10, 10));

JPanel pnlForm = new JPanel(new GridBagLayout());
pnlForm.setBorder(BorderFactory.createTitledBorder("Nova Viagem"));
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(6, 6, 6, 6);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Linha 0: Embarcação e Comandante
gbc.gridx = 0; gbc.gridy = 0; pnlForm.add(new JLabel("Embarcação:"), gbc);
gbc.gridx = 1; cbEmbarcacoes = new JComboBox<>(); pnlForm.add(cbEmbarcacoes, gbc);

gbc.gridx = 2; gbc.gridy = 0; pnlForm.add(new JLabel("Comandante:"), gbc);
gbc.gridx = 3; cbComandantes = new JComboBox<>(); pnlForm.add(cbComandantes, gbc);

// Linha 1: Rota e Passageiros
gbc.gridx = 0; gbc.gridy = 1; pnlForm.add(new JLabel("Rota / Destino:"), gbc);
gbc.gridx = 1; cbDestinos = new JComboBox<>(); pnlForm.add(cbDestinos, gbc);

gbc.gridx = 2; gbc.gridy = 1; pnlForm.add(new JLabel("Passageiros:"), gbc);
gbc.gridx = 3; txtPassageiros = new JTextField(10); pnlForm.add(txtPassageiros, gbc);

// Linha 2: Data da Partida e Horário com seletores numéricos/data
gbc.gridx = 0; gbc.gridy = 2; pnlForm.add(new JLabel("Data da Partida:"), gbc);
gbc.gridx = 1;
spDataPartida = new JSpinner(new SpinnerDateModel());
spDataPartida.setEditor(new JSpinner.DateEditor(spDataPartida, "dd/MM/yyyy"));
pnlForm.add(spDataPartida, gbc);

gbc.gridx = 2; gbc.gridy = 2; pnlForm.add(new JLabel("Horário:"), gbc);
gbc.gridx = 3;
spHorarioPartida = new JSpinner(new SpinnerDateModel());
spHorarioPartida.setEditor(new JSpinner.DateEditor(spHorarioPartida, "HH:mm"));
pnlForm.add(spHorarioPartida, gbc);

// Linha 3: Botão Salvar
gbc.gridx = 3; gbc.gridy = 3;
JButton btnSalvar = new JButton("Registrar Viagem");
btnSalvar.addActionListener(e -> salvarViagem());
pnlForm.add(btnSalvar, gbc);

add(pnlForm, BorderLayout.NORTH);

// Tabela Central
modelTabela = new DefaultTableModel(new String[]{"ID", "Embarcação", "Comandante",
"Destino", "Passageiros", "Partida", "Chegada", "Status"}, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false;
    }
};
tabelaViagens = new JTable(modelTabela);
add(new JScrollPane(tabelaViagens), BorderLayout.CENTER);

// Ações da Tabela
JPanel pnlAcoes = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 10));
JButton btnConcluir = new JButton("Concluir Viagem Seleccionada");
JButton btnCancelar = new JButton("Cancelar Viagem");
JButton btnExcluir = new JButton("Excluir Viagem");

```

```

        btnConcluir.addActionListener(e -> alterarStatusViagem(true));
        btnCancelar.addActionListener(e -> alterarStatusViagem(false));
        btnExcluir.addActionListener(e -> excluirViagem());

        pnlAcoes.add(btnConcluir);
        pnlAcoes.add(btnCancelar);
        pnlAcoes.add(btnExcluir);
        add(pnlAcoes, BorderLayout.SOUTH);

        carregarCombos();
        atualizarTabela();
    }

    private void carregarCombos() {
        cbEmbarcacoes.removeAllItems();
        cbComandantes.removeAllItems();
        cbDestinos.removeAllItems();

        new EmbarcacaoDAO().listarTodas().forEach(cbEmbarcacoes::addItem);
        new TripulanteDAO().listarTodos().forEach(cbComandantes::addItem);
        rotaDAO.listarTodas().forEach(cbDestinos::addItem);
    }

    private void atualizarTabela() {
        modelTabela.setRowCount(0);
        List<Viagem> lista = viagemDAO.listarTodas();
        for (Viagem v : lista) {
            modelTabela.addRow(new Object[]{
                v.getId(),
                v.getNomeEmbarcacao(),
                v.getNomeComandante(),
                v.getRotaDestino(),
                v.getQuantidadePassageiros(),
                v.getDataHoraPartida() != null ? v.getDataHoraPartida().format(formatter) : "-",
                v.getDataHoraChegada() != null ? v.getDataHoraChegada().format(formatter) : "-",
                v.getStatus()
            });
        }
    }

    private void salvarViagem() {
        try {
            Embarcacao emb = (Embarcacao) cbEmbarcacoes.getSelectedItem();
            Tripulante trip = (Tripulante) cbComandantes.getSelectedItem();
            String destino = (String) cbDestinos.getSelectedItem();

            if (emb == null || trip == null || destino == null || destino.trim().isEmpty()) {
                JOptionPane.showMessageDialog(this, "Preencha todos os campos obrigatórios.");
                return;
            }

            int passageiros = Integer.parseInt(txtPassageiros.getText().trim());

            Date dateData = (Date) spDataPartida.getValue();
            Date dateHora = (Date) spHorarioPartida.getValue();

            LocalDate localDate = dateData.toInstant().atZone(ZoneId.systemDefault()).toLocalDate();
            LocalTime localTime = dateHora.toInstant().atZone(ZoneId.systemDefault()).toLocalTime();
            LocalDateTime partida = LocalDateTime.of(localDate, localTime);

```

```

Viagem v = new Viagem(emb.getId(), trip.getId(), destino, partida, passageiros);

String resultado = controller.registrarViagem(v, emb);

if ("OK".equals(resultado)) {
    JOptionPane.showMessageDialog(this, "Viagem registrada com sucesso!");
    txtPassageiros.setText("");
    spDataPartida.setValue(new Date());
    spHorarioPartida.setValue(new Date());
    atualizarTabela();
} else {
    JOptionPane.showMessageDialog(this, resultado, "Alerta de Regra de Negócio",
JOptionPane.WARNING_MESSAGE);
}
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Informe um número inteiro válido para os
passageiros.", "Erro de Entrada", JOptionPane.ERROR_MESSAGE);
} catch (Exception ex) {
    JOptionPane.showMessageDialog(this, "Erro ao processar os campos da viagem.", "Erro de
Entrada", JOptionPane.ERROR_MESSAGE);
}
}

private void alterarStatusViagem(boolean concluir) {
    int linha = tabelaViagens.getSelectedRow();
    if (linha == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma viagem na tabela.");
        return;
    }

    int idViagem = (int) modelTabela.getValueAt(linha, 0);
    String statusAtual = (String) modelTabela.getValueAt(linha, 7);

    if (!"EM_ANDAMENTO".equalsIgnoreCase(statusAtual)) {
        JOptionPane.showMessageDialog(this, "Apenas viagens 'EM_ANDAMENTO' podem ser
alteradas.");
        return;
    }

    boolean ok = concluir ? viagemDAO.finalizarViagem(idViagem) :
viagemDAO.cancelarViagem(idViagem);

    if (ok) {
        JOptionPane.showMessageDialog(this, "Status da viagem atualizado!");
        atualizarTabela();
    } else {
        JOptionPane.showMessageDialog(this, "Erro ao atualizar a viagem no banco.");
    }
}

// MÉTODO NOVO PARA EXCLUIR REGISTRO
private void excluirViagem() {
    int linha = tabelaViagens.getSelectedRow();
    if (linha == -1) {
        JOptionPane.showMessageDialog(this, "Selecione uma viagem na tabela para excluir.",
"Aviso", JOptionPane.WARNING_MESSAGE);
        return;
    }
}

```



```

int idViagem = (int) modelTabela.getValueAt(linha, 0);

int confirmacao = JOptionPane.showConfirmDialog(
    this,
    "Tem certeza que deseja excluir permanentemente a viagem ID " + idViagem + "?",
    "Confirmar Exclusão",
    JOptionPane.YES_NO_OPTION,
    JOptionPane.WARNING_MESSAGE
);

if (confirmacao == JOptionPane.YES_OPTION) {
    boolean ok = controller.excluirViagem(idViagem);
    if (ok) {
        JOptionPane.showMessageDialog(this, "Viagem excluída com sucesso!", "Sucesso",
JOptionPane.INFORMATION_MESSAGE);
        atualizarTabela();
    } else {
        JOptionPane.showMessageDialog(this,
            "Erro ao excluir a viagem.\nVerifique se existem incidentes ou outros registros vinculados
a esta viagem.",
            "Erro de Integridade",
            JOptionPane.ERROR_MESSAGE);
    }
}
}
}
}

```

15.11 Tela de Relatório de Custos

```

package view;
import dao.RelatorioDAO;
import service.EmailService;
import service.GeradorPDFService;

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*.*;
import java.io.File;
import java.text.NumberFormat;
import java.util.List;
import java.util.Locale;

public class TelaRelatorioCustos extends JFrame {

    private final RelatorioDAO dao = new RelatorioDAO();
    private JTable tblCustos;
    private DefaultTableModel tableModel;
    private List<Object[]> dadosCarregados;

    public TelaRelatorioCustos() {
        setTitle("Relatório e Indicadores de Custos (ADM)");
        setSize(900, 550);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        add(criarTabela(), BorderLayout.CENTER);
        add(criarBarraAcoes(), BorderLayout.SOUTH);

        carregarDados();
    }

```

```

    }

    private JScrollPane criarTabela() {
        tableModel = new DefaultTableModel(new String[]{"ID", "Embarcação", "Manutenção (R$)",
"Abastecimento (R$)", "Custo Total (R$)"}, 0) {
            @Override public boolean isCellEditable(int r, int c) { return false; }
        };
        tblCustos = new JTable(tableModel);
        return new JScrollPane(tblCustos);
    }

    private JPanel criarBarraAcoes() {
        JPanel panel = new JPanel(new FlowLayout(FlowLayout.RIGHT, 15, 10));

        JButton btnBaixarPDF = new JButton("Gerar e Baixar PDF (Downloads)");
        btnBaixarPDF.setBackground(new Color(41, 128, 185));
        btnBaixarPDF.setForeground(Color.WHITE);

        JButton btnEnviarEmail = new JButton("Enviar por E-mail");
        btnEnviarEmail.setBackground(new Color(39, 174, 96));
        btnEnviarEmail.setForeground(Color.WHITE);

        btnBaixarPDF.addActionListener(e -> baixarPDF());
        btnEnviarEmail.addActionListener(e -> enviarEmail());

        panel.add(btnBaixarPDF);
        panel.add(btnEnviarEmail);
        return panel;
    }

    private void carregarDados() {
        tableModel.setRowCount(0);
        dadosCarregados = dao.obterResumoCustosPorEmbarcacao();
        NumberFormat nf = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));

        for (Object[] row : dadosCarregados) {
            tableModel.addRow(new Object[]{
                row[0], row[1],
                nf.format((double) row[2]),
                nf.format((double) row[3]),
                nf.format((double) row[4])
            });
        }
    }

    private void baixarPDF() {
        try {
            String caminhoPDF = GeradorPDFService.gerarRelatorioCustos(dadosCarregados);
            JOptionPane.showMessageDialog(this, "PDF gerado com sucesso na pasta
Downloads!\nCaminho: " + caminhoPDF);
            Desktop.getDesktop().open(new File(caminhoPDF));
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Erro ao gerar PDF: " + ex.getMessage(), "Erro",
JOptionPane.ERROR_MESSAGE);
        }
    }

    private void enviarEmail() {
        String emailDestino = JOptionPane.showInputDialog(this, "Informe o e-mail do destinatário:",
"Enviar Relatório", JOptionPane.QUESTION_MESSAGE);
    }

```

```

        if (emailDestino != null && !emailDestino.trim().isEmpty()) {
            try {
                String caminhoPDF = GeradorPDFService.gerarRelatorioCustos(dadosCarregados);
                boolean enviado = EmailService.enviarRelatorioPorEmail(emailDestino.trim(), caminhoPDF);
                if (enviado) {
                    JOptionPane.showMessageDialog(this, "E-mail enviado com sucesso para " +
emailDestino);
                } else {
                    JOptionPane.showMessageDialog(this, "Falha ao enviar e-mail. Verifique as credenciais
SMTP no EmailService.", "Aviso", JOptionPane.WARNING_MESSAGE);
                }
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Erro na operação: " + ex.getMessage(), "Erro",
JOptionPane.ERROR_MESSAGE);
            }
        }
    }
}

```

15.12 Tela Principal

```

package view;
import controller.AlertaController;
import model.Usuario;
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.geom.Path2D;
import java.awt.geom.RoundRectangle2D;
import java.util.ArrayList;
import java.util.List;

public class TelaPrincipal extends JFrame {
    // -----
    // PALETA DE CORES (Design Tokens)
    // -----
    private static final Color BG_APP          = new Color(241, 245, 249); // slate-100
    private static final Color HEADER_DARK     = new Color(15, 23, 42);    // slate-900
    private static final Color HEADER_DARK_2   = new Color(30, 41, 59);    // slate-800
    private static final Color CARD_BG         = Color.WHITE;
    private static final Color CARD_BORDER     = new Color(226, 232, 240); // slate-200
    private static final Color CARD_BORDER_HOVER = new Color(99, 102, 241); // indigo-500
    private static final Color CARD_BG_HOVER   = new Color(248, 250, 252); // slate-50
    private static final Color TEXT_TITLE      = new Color(15, 23, 42);    // slate-900
    private static final Color TEXT_MUTED      = new Color(100, 116, 139); // slate-500
    private static final Color TEXT_ON_DARK    = new Color(226, 232, 240); // slate-200
    private static final Color ACCENT          = new Color(99, 102, 241); // indigo-500
    private static final Color ACCENT_CYAN     = new Color(56, 189, 248); // sky-400
    private static final Color DANGER          = new Color(239, 68, 68);   // red-500

    private static final Font FONT_BOLD        = new Font("Segoe UI", Font.BOLD, 12);

    private final Usuario usuarioLogado;
    private List<String> alertasAtivos = new ArrayList<>();
    private BotaoNotificacao btnNotificacoes;

    public enum Tipolcone { EMBARCACOES, MANUTENCOES, TRIPULACAO, RELATORIOS,
OPERACIONAL, SERVICOS }

    public TelaPrincipal(Usuario usuario) {

```

```

        this.usuarioLogado = usuario;
        setTitle("Gestão Náutica - Painel Principal");
        setSize(1040, 720);
        setMinimumSize(new Dimension(850, 600));
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        initComponents();
        verificarEExibirAlertas();
    }

    private void verificarEExibirAlertas() {
        SwingUtilities.invokeLater(() -> {
            AlertaController alertaController = new AlertaController();
            // Passando o usuário logado para filtrar os alertas por perfil
            alertasAtivos = alertaController.verificarAlertasVencimento(usuarioLogado);

            if (!alertasAtivos.isEmpty()) {
                btnNotificacoes.setNotificacao(true, alertasAtivos.size());
                exibirDialogoAlertas();
            } else {
                btnNotificacoes.setNotificacao(false, 0);
            }
        });
    }

    private void exibirDialogoAlertas() {
        if (alertasAtivos.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Nenhuma pendência ou alerta no momento.",
                "Central de Notificações",
                JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        StringBuilder mensagem = new StringBuilder("Atenção! Existem pendências com vencimento próximo (15 dias):\n\n");
        for (String alerta : alertasAtivos) {
            mensagem.append("• ").append(alerta).append("\n");
        }
        JOptionPane.showMessageDialog(this,
            mensagem.toString(),
            "Alertas do Sistema (RN03)",
            JOptionPane.WARNING_MESSAGE);
    }

    private void initComponents() {
        setLayout(new BorderLayout());
        getContentPane().setBackground(BG_APP);

        JPanel pnlTopo = new JPanel();
        pnlTopo.setLayout(new BoxLayout(pnlTopo, BoxLayout.Y_AXIS));
        pnlTopo.add(criarHeaderRefinado());
        pnlTopo.add(criarPainelGraficosCustomizados());

        add(pnlTopo, BorderLayout.NORTH);
        add(criarPainelMenu(), BorderLayout.CENTER);
    }

    // -----

```

```

// CABEÇALHO REFINADO
// -----
private JPanel criarHeaderRefinado() {
    JPanel panelHeader = new JPanel(new BorderLayout(20, 0)) {
        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            GradientPaint gp = new GradientPaint(0, 0, HEADER_DARK, getWidth(), getHeight(),
HEADER_DARK_2);
            g2.setPaint(gp);
            g2.fillRect(0, 0, getWidth(), getHeight());
            g2.dispose();
            super.paintComponent(g);
        }
    };
    panelHeader.setOpaque(false);
    panelHeader.setBorder(BorderFactory.createEmptyBorder(18, 28, 18, 28));

    // LADO ESQUERDO: Marca Náutica + Título + Usuário
    JPanel pnlInfo = new JPanel(new FlowLayout(FlowLayout.LEFT, 16, 0));
    pnlInfo.setOpaque(false);

    // Ícone do Leme/Âncora Vetorial
    JPanel pnlLogo = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setColor(new Color(56, 189, 248, 40));
            g2.fillOval(0, 0, 42, 42);
            g2.setColor(ACCENT_CYAN);
            g2.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));
            int cx = 21, cy = 21;
            g2.drawOval(cx - 10, cy - 10, 20, 20);
            g2.drawOval(cx - 4, cy - 4, 8, 8);
            for (int i = 0; i < 8; i++) {
                g2.drawLine(cx, cy - 10, cx, cy - 16);
                g2.rotate(Math.toRadians(45), cx, cy);
            }
            g2.dispose();
        }
    };
    pnlLogo.setPreferredSize(new Dimension(42, 42));
    pnlLogo.setOpaque(false);

    String nome = usuarioLogado.getNome();
    String perfilRaw = usuarioLogado.getPerfil() != null ? usuarioLogado.getPerfil().toUpperCase() :
"USUÁRIO";

    JPanel pnlTitulos = new JPanel();
    pnlTitulos.setLayout(new BoxLayout(pnlTitulos, BoxLayout.Y_AXIS));
    pnlTitulos.setOpaque(false);

    JLabel lblSistema = new JLabel("SISTEMA DE GESTÃO NÁUTICA");
    lblSistema.setFont(new Font("Segoe UI", Font.BOLD, 10));
    lblSistema.setForeground(ACCENT_CYAN);

```

```

JLabel lblUsuario = new JLabel("<html><span style='color:#F8FAFC; font-size:14pt; font-family:Segoe UI;'><b>" + nome + "</b></span></html>");

pnlTitulos.add(lblSistema);
pnlTitulos.add(Box.createVerticalStrut(2));
pnlTitulos.add(lblUsuario);

// Badge de Perfil Customizado
JLabel lblBadge = new JLabel(perfilRaw) {
    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setColor(new Color(99, 102, 241, 50));
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 12, 12);
        g2.setColor(ACCENT);
        g2.setStroke(new BasicStroke(1f));
        g2.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 12, 12);
        g2.dispose();
        super.paintComponent(g);
    }
};
lblBadge.setFont(new Font("Segoe UI", Font.BOLD, 9));
lblBadge.setForeground(new Color(224, 231, 255));
lblBadge.setBorder(BorderFactory.createEmptyBorder(4, 10, 4, 10));

pnlInfo.add(pnlLogo);
pnlInfo.add(pnlTitulos);
pnlInfo.add(Box.createHorizontalStrut(8));
pnlInfo.add(lblBadge);

// LADO DIREITO: Notificações + Sair
JPanel pnlAcoesTopo = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 0));
pnlAcoesTopo.setOpaque(false);

btnNotificacoes = new BotaoNotificacao("Notificações");
btnNotificacoes.setToolTipText("Clique para ver os alertas pendentes");
btnNotificacoes.addActionListener(e -> exibirDialogoAlertas());

JButton btnSair = criarBotaoSair();

pnlAcoesTopo.add(btnNotificacoes);
pnlAcoesTopo.add(btnSair);

panelHeader.add(pnlInfo, BorderLayout.WEST);
panelHeader.add(pnlAcoesTopo, BorderLayout.EAST);
return panelHeader;
}
// -----
// PAINEL DE GRÁFICOS DINÂMICOS / PERSONALIZADOS POR PERFIL
// -----
private JPanel criarPainelGraficosCustomizados() {
    JPanel pnlContainer = new JPanel(new GridLayout(1, 2, 20, 0));
    pnlContainer.setBackground(BG_APP);
    pnlContainer.setBorder(BorderFactory.createEmptyBorder(20, 32, 10, 32));

    String perfil = usuarioLogado.getPerfil() != null ? usuarioLogado.getPerfil().toUpperCase() : "";
    switch (perfil) {

```

```

case "ADMINISTRADOR":
    pnlContainer.add(new GraficoRoscaPanel(
        "Status da Frota Náutica",
        new String[]{"Operacional", "Manutenção", "Inativo"},
        new double[]{65, 25, 10},
        new Color[]{new Color(16, 185, 129), new Color(245, 158, 11), new Color(239, 68, 68)},
        "10 Embs"
    ));
    pnlContainer.add(new GraficoBarrasPanel(
        "Despesas Operacionais ($)",
        new String[]{"Jan", "Fev", "Mar", "Abr", "Mai"},
        new double[]{12, 19, 15, 22, 18},
        ACCENT
    ));
    break;

case "OPERADOR":
    pnlContainer.add(new GraficoRoscaPanel(
        "Status dos Incidentes",
        new String[]{"Resolvidos", "Em Aberto"},
        new double[]{80, 20},
        new Color[]{new Color(16, 185, 129), new Color(239, 68, 68)},
        "80% OK"
    ));
    pnlContainer.add(new GraficoBarrasPanel(
        "Horas de Navegação (Semana)",
        new String[]{"Seg", "Ter", "Qua", "Qui", "Sex"},
        new double[]{8, 10, 6, 12, 9},
        new Color(14, 165, 233)
    ));
    break;

case "TECNICO":
    pnlContainer.add(new GraficoRoscaPanel(
        "Ordens de Serviço (OS)",
        new String[]{"Concluídas", "Em Progresso", "Urgentes"},
        new double[]{55, 30, 15},
        new Color[]{new Color(16, 185, 129), new Color(99, 102, 241), new Color(239, 68, 68)},
        "12 OS"
    ));
    pnlContainer.add(new GraficoBarrasPanel(
        "Tempo Médio de Reparo (Horas)",
        new String[]{"Sem 1", "Sem 2", "Sem 3", "Sem 4"},
        new double[]{4.5, 3.2, 5.0, 2.8},
        new Color(168, 85, 247)
    ));
    break;

default:
    pnlContainer.add(new GraficoRoscaPanel("Atividades", new String[]{"Ativas"}, new
double[]{100}, new Color[]{ACCENT}, "100%"));
    pnlContainer.add(new GraficoBarrasPanel("Registro", new String[]{"Dias"}, new double[]{5,
ACCENT}));
    break;
}

return pnlContainer;
}

private JButton criarBotaoSair() {

```

```

        JButton btn = new JButton("Sair") {
            @Override
            protected void paintComponent(Graphics g) {
                Graphics2D g2 = (Graphics2D) g.create();
                g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                boolean hover = getModel().isRollover();
                g2.setColor(hover ? DANGER : Color.WHITE);
                g2.fillRoundRect(0, 0, getWidth(), getHeight(), 10, 10);
                g2.dispose();
                setForeground(hover ? Color.WHITE : DANGER);
                super.paintComponent(g);
            }
        };
        btn.setFont(FONT_BOLD);
        btn.setForeground(DANGER);
        btn.setContentAreaFilled(false);
        btn.setFocusPainted(false);
        btn.setBorderPainted(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        btn.setBorder(BorderFactory.createEmptyBorder(8, 18, 8, 18));
        btn.addActionListener(e -> {
            new TelaLogin().setVisible(true);
            dispose();
        });
        return btn;
    }

    // -----
    // PAINEL CENTRAL / GRID DE CARDS
    // -----
    private JComponent criarPainelMenu() {
        String perfil = usuarioLogado.getPerfil() != null ? usuarioLogado.getPerfil().toUpperCase() : "";

        int colunas = perfil.equals("ADMINISTRADOR") ? 2 : 1;
        JPanel panelMenu = new JPanel(new GridLayout(0, colunas, 20, 20));
        panelMenu.setBackground(BG_APP);
        panelMenu.setBorder(BorderFactory.createEmptyBorder(15, 32, 28, 32));

        switch (perfil) {
            case "ADMINISTRADOR":
                panelMenu.add(criarCardMenu(Tipolcone.EMBARCACOES, "Gerenciar Embarcações",
"Cadastro, frotas e especificações técnicas",
                e -> new TelaGerenciarEmbarcacoes().setVisible(true)));
                panelMenu.add(criarCardMenu(Tipolcone.MANUTENCOES, "Manutenções e Preventivas",
"Ordens de serviço, reparos e chamados",
                e -> new TelaManutencoes().setVisible(true)));
                panelMenu.add(criarCardMenu(Tipolcone.TRIPULACAO, "Gerenciar Tripulação",
"Tripulantes, habilitações e certidões",
                e -> new TelaGerenciarTripulacao().setVisible(true)));
                panelMenu.add(criarCardMenu(Tipolcone.RELATORIOS, "Relatórios de Custos (PDF)",
"Exportação de despesas operacionais",
                e -> new TelaRelatorioCustos().setVisible(true)));
                break;

            case "OPERADOR":
                panelMenu.add(criarCardMenu(Tipolcone.OPERACIONAL, "Painel Operacional Integrado",
"Lançamento de incidentes e diário de bordo",
                e -> new TelaDashboardOperador().setVisible(true)));
                break;
        }
    }

```



```

        case "TECNICO":
            panelMenu.add(criarCardMenu(Tipolcone.SERVICOS, "Serviços Gerais", "Execução de
ordens de serviço atribuídas",
                e -> new TelaDashboardTecnico().setVisible(true)));
            break;

        default:
            JOptionPane.showMessageDialog(this, "Perfil de usuário não reconhecido.", "Erro de
Permissão", JOptionPane.ERROR_MESSAGE);
            break;
    }

    JPanel pnlWrapper = new JPanel(new BorderLayout());
    pnlWrapper.setBackground(BG_APP);
    pnlWrapper.add(panelMenu, BorderLayout.NORTH);

    JScrollPane scroll = new JScrollPane(pnlWrapper);
    scroll.setBorder(BorderFactory.createEmptyBorder());
    scroll.getViewport().setBackground(BG_APP);
    scroll.getVerticalScrollBar().setUnitIncrement(16);
    return scroll;
}

private JButton criarCardMenu(Tipolcone icone, String titulo, String subtítulo,
java.awt.event.ActionListener acao) {
    JButton btn = new JButton() {
        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            boolean hover = getModel().isRollover();
            int w = getWidth(), h = getHeight();
            RoundedRectangle2D shape = new RoundedRectangle2D.Float(0, 0, w - 2, h - 2, 16, 16);

            g2.setColor(new Color(15, 23, 42, hover ? 25 : 10));
            g2.fill(new RoundedRectangle2D.Float(2, hover ? 5 : 3, w - 2, h - 2, 16, 16));

            g2.setColor(hover ? CARD_BG_HOVER : CARD_BG);
            g2.fill(shape);

            g2.setColor(hover ? CARD_BORDER_HOVER : CARD_BORDER);
            g2.setStroke(new BasicStroke(hover ? 1.8f : 1.0f));
            g2.draw(shape);

            g2.dispose();
            super.paintComponent(g);
        }
    };

    btn.setLayout(new BorderLayout());
    btn.setOpaque(false);
    btn.setContentAreaFilled(false);
    btn.setFocusPainted(false);
    btn.setBorderPainted(false);
    btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btn.setPreferredSize(new Dimension(340, 95));

```

```

JPanel conteudo = new JPanel(new BorderLayout(16, 0));
conteudo.setOpaque(false);
conteudo.setBorder(BorderFactory.createEmptyBorder(14, 20, 14, 20));

PainellconeVetorial pnlcone = new PainellconeVetorial(icones);

JLabel lblTexto = new JLabel("<html><body>"
    + "<span style='color:#0F172A; font-size:11.5pt; font-family:Segoe UI;'><b>" + titulo +
"</b></span><br>"
    + "<span style='color:#64748B; font-size:8.5pt;'>" + subtítulo + "</span>"
    + "</body></html>");

JLabel lblSeta = new JLabel(">");
lblSeta.setFont(new Font("Segoe UI", Font.BOLD, 22));
lblSeta.setForeground(new Color(148, 163, 184));

conteudo.add(pnlcone, BorderLayout.WEST);
conteudo.add(lblTexto, BorderLayout.CENTER);
conteudo.add(lblSeta, BorderLayout.EAST);

btn.add(conteudo, BorderLayout.CENTER);
btn.addActionListener(acao);

return btn;
}
// -----
// DESENHO VETORIAL DE ÍCONES
// -----
private static class PainellconeVetorial extends JPanel {
    private final Tipolcone tipo;
    public PainellconeVetorial(Tipolcone tipo) {
        this.tipo = tipo;
        setPreferredSize(new Dimension(48, 48));
        setOpaque(false);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        Color bg, fg;
        switch (tipo) {
            case EMBARCACOES: bg = new Color(224, 231, 255); fg = new Color(79, 70, 229); break;
            case MANUTENCOES: bg = new Color(254, 243, 199); fg = new Color(217, 119, 6); break;
            case TRIPULACAO: bg = new Color(204, 251, 241); fg = new Color(13, 148, 136); break;
            case RELATORIOS: bg = new Color(254, 226, 226); fg = new Color(225, 29, 72); break;
            case OPERACIONAL: bg = new Color(209, 250, 229); fg = new Color(5, 150, 105); break;
            case SERVICOS:
                default: bg = new Color(243, 232, 255); fg = new Color(147, 51, 234); break;
        }

        g2.setColor(bg);
        g2.fillOval(0, 0, 48, 48);

        g2.setColor(fg);
        g2.setStroke(new BasicStroke(2.0f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));

```

```

int cx = 24, cy = 24;

switch (tipo) {
    case EMBARCACOES:
        Path2D boat = new Path2D.Float();
        boat.moveTo(cx - 10, cy + 3); boat.lineTo(cx + 10, cy + 3);
        boat.lineTo(cx + 6, cy + 9); boat.lineTo(cx - 6, cy + 9);
        boat.closePath();
        g2.draw(boat);
        g2.drawLine(cx, cy + 3, cx, cy - 8);
        Path2D sail = new Path2D.Float();
        sail.moveTo(cx, cy - 8); sail.lineTo(cx + 6, cy - 2); sail.lineTo(cx, cy - 2);
        sail.closePath();
        g2.fill(sail);
        break;
    case MANUTENCOES:
        g2.rotate(Math.toRadians(45), cx, cy);
        g2.drawOval(cx - 4, cy - 10, 8, 8);
        g2.fillRect(cx - 2, cy - 2, 4, 11);
        break;
    case TRIPULACAO:
        g2.drawOval(cx - 4, cy - 9, 8, 8);
        g2.drawArc(cx - 8, cy, 16, 12, 0, 180);
        break;
    case RELATORIOS:
        g2.drawRoundRect(cx - 7, cy - 10, 14, 20, 3, 3);
        g2.drawLine(cx - 4, cy - 4, cx + 4, cy - 4);
        g2.drawLine(cx - 4, cy, cx + 4, cy);
        g2.drawLine(cx - 4, cy + 4, cx + 1, cy + 4);
        break;
    case OPERACIONAL:
        g2.drawRoundRect(cx - 8, cy - 7, 16, 18, 3, 3);
        g2.drawRoundRect(cx - 3, cy - 10, 6, 4, 2, 2);
        g2.drawLine(cx - 4, cy, cx + 4, cy);
        break;
    case SERVICOS:
        g2.drawOval(cx - 5, cy - 5, 10, 10);
        for (int i = 0; i < 8; i++) {
            g2.drawLine(cx, cy - 5, cx, cy - 9);
            g2.rotate(Math.toRadians(45), cx, cy);
        }
        break;
}

g2.dispose();
}
}
// -----
// GRÁFICO 1: GRÁFICO DE ROSCA 2D NATIVO
// -----
private static class GraficoRoscaPanel extends JPanel {
    private final String titulo;
    private final String[] categorias;
    private final double[] valores;
    private final Color[] cores;
    private final String centroTexto;

    public GraficoRoscaPanel(String titulo, String[] categorias, double[] valores, Color[] cores, String
centroTexto) {
        this.titulo = titulo;

```

```

        this.categorias = categorias;
        this.valores = valores;
        this.cores = cores;
        this.centroTexto = centroTexto;
        setOpaque(false);
        setPreferredSize(new Dimension(300, 130));
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        int w = getWidth(), h = getHeight();

        // Card Fundo
        g2.setColor(CARD_BG);
        g2.fillRoundRect(0, 0, w - 1, h - 1, 14, 14);
        g2.setColor(CARD_BORDER);
        g2.setStroke(new BasicStroke(1f));
        g2.drawRoundRect(0, 0, w - 1, h - 1, 14, 14);

        // Título
        g2.setFont(new Font("Segoe UI", Font.BOLD, 11));
        g2.setColor(TEXT_MUTED);
        g2.drawString(titulo.toUpperCase(), 14, 22);

        // Cálculo dos Arcos
        double total = 0;
        for (double v : valores) total += v;

        int diameter = 80;
        int x = 14;
        int y = 34;

        int startAngle = 90;
        for (int i = 0; i < valores.length; i++) {
            int angle = (int) Math.round((valores[i] / total) * 360);
            g2.setColor(cores[i]);
            g2.fillArc(x, y, diameter, diameter, startAngle, angle);
            startAngle += angle;
        }

        // Furo Central da Rosca
        int holeSize = 50;
        int hx = x + (diameter - holeSize) / 2;
        int hy = y + (diameter - holeSize) / 2;
        g2.setColor(CARD_BG);
        g2.fillOval(hx, hy, holeSize, holeSize);

        // Texto Central
        g2.setFont(new Font("Segoe UI", Font.BOLD, 10));
        g2.setColor(TEXT_TITLE);
        FontMetrics fm = g2.getFontMetrics();
        int tx = hx + (holeSize - fm.stringWidth(centroTexto)) / 2;
        int ty = hy + ((holeSize - fm.getHeight()) / 2) + fm.getAscent();
        g2.drawString(centroTexto, tx, ty);

        // Legendas à Direita

```

```

        int lx = x + diameter + 18;
        int ly = y + 14;
        g2.setFont(new Font("Segoe UI", Font.PLAIN, 10));

        for (int i = 0; i < categorias.length; i++) {
            g2.setColor(cores[i]);
            g2.fillRoundRect(lx, ly, 8, 8, 2, 2);
            g2.setColor(TEXT_TITLE);
            g2.drawString(categorias[i] + " (" + (int) valores[i] + "%)", lx + 12, ly + 8);
            ly += 18;
        }

        g2.dispose();
    }
}
// -----
// GRÁFICO 2: GRÁFICO DE BARRAS 2D NATIVO
// -----
private static class GraficoBarrasPanel extends JPanel {
    private final String titulo;
    private final String[] labels;
    private final double[] valores;
    private final Color corBarra;

    public GraficoBarrasPanel(String titulo, String[] labels, double[] valores, Color corBarra) {
        this.titulo = titulo;
        this.labels = labels;
        this.valores = valores;
        this.corBarra = corBarra;
        setOpaque(false);
        setPreferredSize(new Dimension(300, 130));
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int w = getWidth(), h = getHeight();

        // Card Fundo
        g2.setColor(CARD_BG);
        g2.fillRoundRect(0, 0, w - 1, h - 1, 14, 14);
        g2.setColor(CARD_BORDER);
        g2.setStroke(new BasicStroke(1f));
        g2.drawRoundRect(0, 0, w - 1, h - 1, 14, 14);

        // Título
        g2.setFont(new Font("Segoe UI", Font.BOLD, 11));
        g2.setColor(TEXT_MUTED);
        g2.drawString(titulo.toUpperCase(), 14, 22);

        // Maior valor para escala
        double maxVal = 0;
        for (double v : valores) if (v > maxVal) maxVal = v;

        int barWidth = 18;
        int chartHeight = 55;
        int startX = 24;
        int startY = 100;

```

```

int gap = (w - 40 - (labels.length * barWidth)) / (labels.length);

g2.setFont(new Font("Segoe UI", Font.PLAIN, 9));

for (int i = 0; i < valores.length; i++) {
    int barH = (int) ((valores[i] / maxVal) * chartHeight);
    int bx = startX + i * (barWidth + gap);
    int by = startY - barH;

    // Barra Arredondada
    g2.setColor(corBarra);
    g2.fillRoundRect(bx, by, barWidth, barH, 4, 4);

    // Label X
    g2.setColor(TEXT_MUTED);
    g2.drawString(labels[i], bx + (barWidth / 2) - 8, startY + 14);
}

g2.dispose();
}
}
// -----
// BOTÃO DE NOTIFICAÇÃO
// -----
private static class BotaoNotificacao extends JButton {
    private boolean temNotificacao = false;
    private int quantidade = 0;

    public BotaoNotificacao(String texto) {
        super(texto);
        setFont(FONT_BOLD);
        setForeground(TEXT_ON_DARK);
        setContentAreaFilled(false);
        setFocusPainted(false);
        setBorderPainted(false);
        setBorder(BorderFactory.createEmptyBorder(8, 36, 8, 20));
        setCursor(new Cursor(Cursor.HAND_CURSOR));
    }

    public void setNotificacao(boolean ativa, int qtd) {
        this.temNotificacao = ativa;
        this.quantidade = qtd;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);

        boolean hover = getModel().isRollover();
        g2.setColor(hover ? new Color(71, 85, 105) : new Color(51, 65, 85));
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 10, 10);

        int bx = 14;
        int by = (getHeight() - 16) / 2;
        g2.setColor(TEXT_ON_DARK);
        g2.setStroke(new BasicStroke(1.5f));
        g2.drawArc(bx + 4, by, 4, 4, 0, 180);
    }
}

```

```

Path2D bell = new Path2D.Float();
bell.moveTo(bx + 1, by + 11);
bell.curveTo(bx + 1, by + 6, bx + 3, by + 3, bx + 6, by + 3);
bell.curveTo(bx + 9, by + 3, bx + 11, by + 6, bx + 11, by + 11);
bell.lineTo(bx + 13, by + 12);
bell.lineTo(bx - 1, by + 12);
bell.closePath();
g2.fill(bell);
g2.fillOval(bx + 4, by + 13, 4, 3);

super.paintComponent(g2);

if (temNotificacao) {
    int raio = 16;
    int x = getWidth() - raio - 4;
    int y = 4;

    g2.setColor(DANGER);
    g2.fillOval(x, y, raio, raio);

    g2.setColor(new Color(30, 41, 59));
    g2.setStroke(new BasicStroke(2f));
    g2.drawOval(x, y, raio, raio);

    if (quantidade > 0) {
        g2.setColor(Color.WHITE);
        g2.setFont(new Font("Segoe UI", Font.BOLD, 9));
        FontMetrics fm = g2.getFontMetrics();
        String txt = quantidade > 9 ? "9+" : String.valueOf(quantidade);
        int txtX = x + (raio - fm.stringWidth(txt)) / 2;
        int txtY = y + ((raio - fm.getHeight()) / 2) + fm.getAscent();
        g2.drawString(txt, txtX, txtY);
    }
}

g2.dispose();
}
}
}

```

Camada Controller

15.13 UsuarioController.java

package controller; / import ...; Declara o pacote e importa as dependências de acesso a dados e modelos.

private final UsuarioDAO usuarioDAO;; Mantém a instância do DAO de usuários.

UsuarioController(): Construtor que inicializa a instância do UsuarioDAO.

autenticar(String email, String senha): Valida se o e-mail ou a senha fornecidos são nulos/vazios; se forem válidos, remove os espaços nas pontas (trim()) e consulta a autenticação no banco via usuarioDAO.autenticar().

```

package controller;
import dao.UsuarioDAO;
import model.Usuario;

```

```

public class UsuarioController {

    private final UsuarioDAO usuarioDAO;

    public UsuarioController() {
        this.usuarioDAO = new UsuarioDAO();
    }

    public Usuario autenticar(String email, String senha) {
        if (email == null || email.trim().isEmpty() || senha == null || senha.trim().isEmpty()) {
            return null;
        }
        return usuarioDAO.autenticar(email.trim(), senha.trim());
    }
}

```

15.14 EmbarcacaoController.java

listarEmbarcacoes(): Retorna a lista de todas as embarcações gravadas no banco.

salvarOuAtualizar(...): Verifica a presença obrigatória de nome e modelo; valida se o ano de fabricação está entre 1900 e 2030; instancia e preenche a entidade Embarcacao; e aciona dao.salvar(emb) caso seja um novo cadastro (ID nulo/0) ou dao.atualizar(emb) para edições.

excluirEmbarcacao(int id): Executa a remoção do registro pelo ID informado.

```

package controller;
import dao.EmbarcacaoDAO;
import model.Embarcacao;
import java.util.List;

public class EmbarcacaoController {

    private final EmbarcacaoDAO dao = new EmbarcacaoDAO();

    public List<Embarcacao> listarEmbarcacoes() {
        return dao.listarTodas();
    }

    public String salvarOuAtualizar(Integer id, String nome, String modelo, int capPassageiros, double capCarga, int ano, int horimetro, String status) {
        if (nome == null || nome.trim().isEmpty()) {
            return "O nome da embarcação é obrigatório.";
        }
        if (modelo == null || modelo.trim().isEmpty()) {
            return "O modelo é obrigatório.";
        }
        if (ano < 1900 || ano > 2030) {
            return "Ano de fabricação inválido.";
        }

        Embarcacao emb = new Embarcacao();
        emb.setNome(nome.trim());
        emb.setModelo(modelo.trim());
        emb.setCapacidadePassageiros(capPassageiros);
        emb.setCapacidadeCargaTon(capCarga);
        emb.setAnoFabricacao(ano);
        emb.setHorimetroHoras(horimetro);
        emb.setStatus(status);
    }
}

```



```

        boolean sucesso;
        if (id == null || id == 0) {
            sucesso = dao.salvar(emb);
        } else {
            emb.setId(id);
            sucesso = dao.atualizar(emb);
        }

        return sucesso ? "OK" : "Erro ao salvar embarcação no banco de dados.";
    }

    public boolean excluirEmbarcacao(int id) {
        return dao.excluir(id);
    }
}

```

15.15 ManutencaoController.java

obterIncidentesPendentes() e obterManutencoes(): Solicitam as listagens consolidadas ao IncidenteDAO e ManutencaoDAO.

criarOrdemServico(...): Valida o preenchimento da descrição e data, salvando a ordem de serviço no banco.

converterIncidenteEmOS(...): Gera automaticamente uma Ordem de Serviço do tipo "CORRETIVA" vinculada a um incidente e altera o status do incidente para "EM_ANALISE" em caso de sucesso.

atualizarStatusOS(...): Modifica a etapa de status da ordem de serviço.

```

package controller;
import dao.IncidenteDAO;
import dao.ManutencaoDAO;

```

```

import java.sql.Date;
import java.util.List;

```

```

public class ManutencaoController {

```

```

    private final ManutencaoDAO manutencaoDAO = new ManutencaoDAO();
    private final IncidenteDAO incidenteDAO = new IncidenteDAO();

```

```

    public List<Object[]> obterIncidentesPendentes() {
        return incidenteDAO.listarIncidentesPendentes();
    }

```

```

    public List<Object[]> obterManutencoes() {
        return manutencaoDAO.listarTodasManutencoes();
    }

```

```

    public String criarOrdemServico(int idEmbarcacao, String tipo, String descricao, Integer horimetro,
Date dataAgendada, double custo) {
        if (descricao == null || descricao.trim().isEmpty()) {
            return "A descrição do serviço é obrigatória.";
        }
        if (dataAgendada == null) {
            return "A data de agendamento é obrigatória.";
        }

```

```

        boolean ok = manutencaoDAO.salvar(idEmbarcacao, tipo, descricao, horimetro, dataAgendada,
custo);

```

```

        return ok ? "OK" : "Erro ao registrar ordem de serviço no banco de dados.";
    }

    public boolean converterIncidenteEmOS(int idIncidente, int idEmbarcacao, String descricao, Date
dataAgendada) {
        boolean osCriada = manutencaoDAO.salvar(idEmbarcacao, "CORRETIVA", "OS Originada do
Incidente #" + idIncidente + ": " + descricao, null, dataAgendada, 0.0);
        if (osCriada) {
            incidenteDAO.atualizarStatus(idIncidente, "EM_ANALISE");
            return true;
        }
        return false;
    }

    public boolean atualizarStatusOS(int idManutencao, String status) {
        return manutencaoDAO.alterarStatus(idManutencao, status);
    }
}

```

15.16 TripulanteController.java

listarTripulantes(): Recupera a lista de tripulantes.

salvarOuAtualizar(...): Valida nome, CPF, CIR e data de vencimento do registro da CIR; monta o modelo Tripulante; e alterna entre inclusão (salvar) e edição (atualizar).

excluirTripulante(int id): Deleta o tripulante pelo ID.

```

package controller;
import dao.TripulanteDAO;
import model.Tripulante;
import java.sql.Date;
import java.util.List;

public class TripulanteController {
    private final TripulanteDAO dao = new TripulanteDAO();
    public List<Tripulante> listarTripulantes() {
        return dao.listarTodos();
    }
    public String salvarOuAtualizar(Integer id, String nome, String cpf, String categoria, String cir, Date
dataVencimento, String status) {
        if (nome == null || nome.trim().isEmpty()) {
            return "O nome do tripulante é obrigatório.";
        }
        if (cpf == null || cpf.trim().isEmpty()) {
            return "O CPF é obrigatório.";
        }
        if (cir == null || cir.trim().isEmpty()) {
            return "O registro CIR é obrigatório.";
        }
        if (dataVencimento == null) {
            return "A data de vencimento da CIR é obrigatória.";
        }
        Tripulante t = new Tripulante();
        t.setNome(nome.trim());
        t.setCpf(cpf.trim());
        t.setCategoriaHabilitacao(categoria);
        t.setNumeroRegistroCir(cir.trim());
        t.setDataVencimentoCir(dataVencimento);
        t.setStatus(status);
    }
}

```

```

        boolean sucesso;
        if (id == null || id == 0) {
            sucesso = dao.salvar(t);
        } else {
            t.setId(id);
            sucesso = dao.atualizar(t);
        }
        return sucesso ? "OK" : "Erro ao salvar tripulante no banco de dados (CPF ou CIR podem estar duplicados).";
    }
    public boolean excluirTripulante(int id) {
        return dao.excluir(id);
    }
}

```

15.17 ViagemController.java

registrarViagem(...): Aplica as regras de negócio RN01 e RN02, bloqueando a viagem se a embarcação estiver em manutenção/inativa, se a embarcação ou o comandante já possuírem uma viagem "EM_ANDAMENTO", ou se a quantidade de passageiros ultrapassar a capacidade.

excluirViagem(int idViagem): Valida se o ID é maior que zero e envia a requisição de exclusão para o banco.

```

package controller;
import dao.ViagemDAO;
import model.Embarcacao;
import model.Viagem;

public class ViagemController {

    private final ViagemDAO viagemDAO = new ViagemDAO();

    public String registrarViagem(Viagem viagem, Embarcacao embarcacao) {
        // Regra RN01: Validação da Embarcação
        if ("EM_MANUTENCAO".equalsIgnoreCase(embarcacao.getStatus()) ||
            "INATIVA".equalsIgnoreCase(embarcacao.getStatus())) {
            return "A embarcação selecionada está em manutenção ou inativa.";
        }
        if (viagemDAO.embarcacaoEmViagem(viagem.getIdEmbarcacao())) {
            return "A embarcação selecionada já possui uma viagem em andamento.";
        }
        // Regra RN02: Validação da Tripulação / Comandante
        if (viagemDAO.comandanteEmViagem(viagem.getIdComandante())) {
            return "O comandante selecionado já está escalado em outra viagem em andamento.";
        }
        // Validação de Capacidade
        if (viagem.getQuantidadePassageiros() > embarcacao.getCapacidadePassageiros()) {
            return "A quantidade de passageiros (" + viagem.getQuantidadePassageiros() +
                ") excede a capacidade máxima (" + embarcacao.getCapacidadePassageiros() + ").";
        }
        // Persistência
        boolean sucesso = viagemDAO.salvar(viagem);
        return sucesso ? "OK" : "Erro ao salvar a viagem no banco de dados.";
    }
    // MÉTODO NOVO NO CONTROLLER
    public boolean excluirViagem(int idViagem) {
        if (idViagem <= 0) {
            return false;
        }
    }
}

```

```

    }
    return viagemDAO.excluirViagem(idViagem);
}
}

```

15.18 RelatorioController.java

CustoEmbarcacaoDTO: DTO interno para transportar o nome da embarcação, contagem de manutenções e o custo somado.

obterConsolidadoCustos(): Consulta via JDBC o agrupamento de custos das manutenções concluídas por embarcação.

```

package controller;
import dao.ConexaoDAO;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class RelatorioController {

    public static class CustoEmbarcacaoDTO {
        private String nomeEmbarcacao;
        private int totalManutencoes;
        private double custoTotal;

        public CustoEmbarcacaoDTO(String nomeEmbarcacao, int totalManutencoes, double custoTotal)
        {
            this.nomeEmbarcacao = nomeEmbarcacao;
            this.totalManutencoes = totalManutencoes;
            this.custoTotal = custoTotal;
        }

        public String getNomeEmbarcacao() { return nomeEmbarcacao; }
        public int getTotalManutencoes() { return totalManutencoes; }
        public double getCustoTotal() { return custoTotal; }
    }

    public List<CustoEmbarcacaoDTO> obterConsolidadoCustos() {
        List<CustoEmbarcacaoDTO> lista = new ArrayList<>();
        String sql = "SELECT e.nome AS embarcacao, COUNT(m.id) AS qtd_manutencoes,
        COALESCE(SUM(m.custo_total), 0) AS custo_total " +
            "FROM embarcacoes e " +
            "LEFT JOIN manutencoes m ON e.id = m.id_embarcacao AND m.status = 'CONCLUIDA'
        " +
            "GROUP BY e.id, e.nome " +
            "ORDER BY custo_total DESC";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                lista.add(new CustoEmbarcacaoDTO(
                    rs.getString("embarcacao"),
                    rs.getInt("qtd_manutencoes"),
                    rs.getDouble("custo_total")
                ));
            }
        }
    }
}

```

```

    }
} catch (SQLException e) {
    System.err.println("Erro ao gerar relatório de custos: " + e.getMessage());
}
return lista;
}
}

```

15.19 AlertaController.java

verificarAlertasVencimento(): Executa consultas no banco de dados para levantar tripulantes com habilitação a vencer nos próximos 15 dias e manutenções agendadas para o mesmo período.

```

package controller;
import dao.ConexaoDAO;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class AlertaController {

    public List<String> verificarAlertasVencimento() {
        List<String> alertas = new ArrayList<>();

        // Query 1: Validação de CIR dos tripulantes vencidas ou a vencer em 15 dias
        String sqlTripulantes = "SELECT nome, data_vencimento_cir FROM tripulantes " +
            "WHERE data_vencimento_cir <= DATE_ADD(CURDATE(), INTERVAL 15 DAY) " +
            "AND status != 'INATIVO'";

        // Query 2: Manutenções pendentes/agendadas para os próximos 15 dias
        String sqlManutencoes = "SELECT e.nome AS embarcacao, m.descricao_servico, " +
            m.data_agendamento " +
            "FROM manutencoes m " +
            "JOIN embarcacoes e ON m.id_embarcacao = e.id " +
            "WHERE m.data_agendamento <= DATE_ADD(CURDATE(), INTERVAL 15 " +
            DAY) " +
            "AND m.status = 'AGENDADA'";

        try (Connection conn = ConexaoDAO.obterConexao()) {
            // Consulta Tripulantes
            try (PreparedStatement stmt = conn.prepareStatement(sqlTripulantes);
                ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    alertas.add(" Tripulante: " + rs.getString("nome") +
                        " | CIR vence em: " + rs.getDate("data_vencimento_cir"));
                }
            }
            // Consulta Manutenções
            try (PreparedStatement stmt = conn.prepareStatement(sqlManutencoes);
                ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    alertas.add(" Manutenção: " + rs.getString("embarcacao") +
                        " (" + rs.getString("descricao_servico") + ") | Data: " +
                        rs.getDate("data_agendamento"));
                }
            }
        } catch (SQLException e) {

```

```

        System.err.println("Erro ao verificar alertas de vencimento: " + e.getMessage());
    }

    return alertas;
}
}

```

15.20 AbastecimentoController.java

registrar(...): Valida os atributos de abastecimento (identificação da embarcação, valores positivos para litros e custo total, e fornecedor), chamando o DAO para persistir.

```

package controller;
import dao.AbastecimentoDAO;
import model.Abastecimento;

public class AbastecimentoController {

    private final AbastecimentoDAO abastecimentoDAO = new AbastecimentoDAO();
    public String registrar(Abastecimento abastecimento) {
        if (abastecimento == null) {
            return "Dados de abastecimento inválidos.";
        }
        if (abastecimento.getEmbarcacaoId() <= 0) {
            return "Selecione uma embarcação válida.";
        }
        if (abastecimento.getLitros() <= 0) {
            return "A quantidade de litros deve ser maior que zero.";
        }
        if (abastecimento.getValorTotal() <= 0) {
            return "O valor total deve ser maior que zero.";
        }
        if (abastecimento.getFornecedor() == null || abastecimento.getFornecedor().trim().isEmpty()) {
            return "Selecione um posto / fornecedor válido.";
        }

        boolean sucesso = abastecimentoDAO.salvar(abastecimento);
        return sucesso ? "OK" : "Erro ao registrar o abastecimento no banco de dados.";
    }
}

```

15.21 IncidenteController.java

registrarIncidente(...): Valida dados da ocorrência, realiza a conversão segura do código da viagem e cria o registro com a data/hora atual.

```

package controller;
import dao.IncidenteDAO;
import model.Incidente;
import java.time.LocalDateTime;

public class IncidenteController {

    private final IncidenteDAO incidenteDAO = new IncidenteDAO();

    public String registrarIncidente(Integer idEmbarcacao, String viagemIdStr, String gravidade, String descricao) {
        if (idEmbarcacao == null || idEmbarcacao <= 0) {
            return "Selecione uma embarcação válida.";
        }
    }
}

```

```

    if (descricao == null || descricao.trim().isEmpty()) {
        return "A descrição do incidente não pode ficar em branco.";
    }
    Integer idViagem = null;
    if (viagemIdStr != null && !viagemIdStr.trim().isEmpty()) {
        try {
            idViagem = Integer.parseInt(viagemIdStr.trim());
        } catch (NumberFormatException e) {
            return "O código da viagem deve ser um número inteiro válido.";
        }
    }
    Incidente incidente = new Incidente(
        idEmbarcacao,
        idViagem,
        LocalDateTime.now(),
        descricao.trim(),
        gravidade
    );
    boolean sucesso = incidenteDAO.salvar(incidente);
    return sucesso ? "OK" : "Erro ao salvar o incidente no banco de dados.";
}
}

```

15.22 ConsultaHorariosController.java

buscarHorarios(...): Carrega todas as saídas e aplica filtros em memória através do Java Streams baseados em embarcação, comandante e destino.

obterEmbarcacoes(), obterComandantes(), obterDestinos(): Retornam as listas para filtros incluindo o item "Todos" no início.

```

package controller;
import dao.ConsultaHorariosDAO;
import dto.ConsultaHorarioDTO;
import java.util.List;
import java.util.stream.Collectors;

public class ConsultaHorariosController {

    private final ConsultaHorariosDAO dao = new ConsultaHorariosDAO();

    public List<ConsultaHorarioDTO> buscarHorarios(String embarcacao, String comandante, String destino) {
        List<ConsultaHorarioDTO> todos = dao.listarHorariosSaida();

        return todos.stream().filter(item -> {
            boolean matchEmbarcacao = (embarcacao == null || embarcacao.equals("Todos")) ||
            item.getEmbarcacao().equalsIgnoreCase(embarcacao);
            boolean matchComandante = (comandante == null || comandante.equals("Todos")) ||
            item.getComandante().equalsIgnoreCase(comandante);
            boolean matchDestino = (destino == null || destino.equals("Todos")) ||
            item.getRotaDestino().equalsIgnoreCase(destino);

            return matchEmbarcacao && matchComandante && matchDestino;
        }).collect(Collectors.toList());
    }

    public List<String> obterEmbarcacoes() {
        List<String> lista = dao.listarTodasEmbarcacoes();
        lista.add(0, "Todos");
    }
}

```

```

        return lista;
    }

    public List<String> obterComandantes() {
        List<String> lista = dao.listarTodosComandantes();
        lista.add(0, "Todos");
        return lista;
    }

    public List<String> obterDestinos() {
        List<String> lista = dao.listarTodosDestinos();
        lista.add(0, "Todos");
        return lista;
    }
}

```

15.23 TecnicoController.java

obterOSAbertas(), obterAlertasHorimetro(), obterHistoricoMotores(), obterEmbarcacoes(): Buscam os dados operacionais da oficina.

criarNovaOS(...) e finalizarManutencao(...): Validam entrada de horímetros, custos e datas, convertendo tipos de dados antes de persistir o status e horímetro final.

```

package controller;
import dao.TecnicoDAO;
import java.sql.Date;
import java.util.List;

public class TecnicoController {

    private final TecnicoDAO dao = new TecnicoDAO();

    public List<Object[]> obterOSAbertas() {
        return dao.listarOSAbertas();
    }

    public List<Object[]> obterAlertasHorimetro() {
        return dao.listarAlertasHorimetro();
    }

    public List<Object[]> obterHistoricoMotores() {
        return dao.listarHistoricoCompleto();
    }

    public List<Object[]> obterEmbarcacoes() {
        return dao.obterEmbarcacoesSimplificado();
    }

    public String criarNovaOS(int idEmbarcacao, String tipo, String descricao, String horimetroStr,
        java.util.Date dataAgendamento) {
        if (descricao == null || descricao.trim().isEmpty()) {
            return "A descrição detalhada do serviço é obrigatória.";
        }
        if (dataAgendamento == null) {
            return "A data de agendamento é obrigatória.";
        }
        Integer horimetro = null;
        if (horimetroStr != null && !horimetroStr.trim().isEmpty()) {
            try {
                horimetro = Integer.parseInt(horimetroStr.trim());
            } catch (NumberFormatException e) {
                return "Horímetro deve ser um número inteiro válido.";
            }
        }
    }
}

```



```

    }
}
Date dataSQL = new Date(dataAgendamento.getTime());
boolean ok = dao.salvarNovaOS(idEmbarcacao, tipo, descricao.trim(), horimetro, dataSQL);
return ok ? "OK" : "Erro ao abrir Ordem de Serviço no banco de dados.";
}
public String finalizarManutencao(int idManutencao, int idEmbarcacao, String horimetroStr, String
custoStr) {
    try {
        int horimetro = Integer.parseInt(horimetroStr.trim());
        double custo = Double.parseDouble(custoStr.replace(",", ".").trim());

        if (horimetro < 0 || custo < 0) {
            return "Valores não podem ser negativos.";
        }
        boolean ok = dao.concluirOS(idManutencao, idEmbarcacao, horimetro, custo);
        return ok ? "OK" : "Erro ao concluir Ordem de Serviço no banco de dados.";
    } catch (NumberFormatException e) {
        return "Por favor, insira valores numéricos válidos para Horímetro e Custo.";
    }
}
}
}

```

Camada DAO(Data Access Object)

15.24 ConexaoDAO.java

Define as credenciais de acesso ao MySQL (localhost, porta 3306, banco gestao_nautica_db).

obterConexao(): Carrega o driver JDBC com `mysql.cj.jdbc.Driver` e retorna a conexão ativa.

```

package dao;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
/**
 * Classe responsável por gerenciar a conexão JDBC com o MySQL (XAMPP).
 */
public class ConexaoDAO {

    private static final String URL =
"jdbc:mysql://localhost:3306/gestao_nautica_db?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = ""; // Senha padrão do XAMPP (em branco)
    public static Connection obterConexao() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("Driver MySQL JDBC não foi encontrado na biblioteca do projeto!",
e);
        }
    }
}
}

```

15.25 UsuarioDAO.java

autenticar(...): Busca o usuário ativo pelo e-mail e valida a senha utilizando o algoritmo criptográfico BCrypt (com suporte de fallback para texto puro).

cadastrarUsuario(...): Aplica o hash BCrypt na senha informada e insere o novo usuário no banco.

```
package dao;
import model.Usuario;
import org.mindrot.jbcrypt.BCrypt;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UsuarioDAO {
    /**
     * Valida o e-mail e verifica a senha criptografada com BCrypt.
     */
    public Usuario autenticar(String email, String senhaDigitada) {
        // Busca o usuário apenas pelo email e status ativo
        String sql = "SELECT * FROM usuarios WHERE email = ? AND status = 'ATIVO'";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, email);

            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    String hashSenhaBanco = rs.getString("senha");

                    // Verifica se a senha digitada bate com o hash criptografado do banco
                    // Suporta fallback para senhas em texto puro durante transição
                    boolean senhaValida = false;
                    if (hashSenhaBanco.startsWith("$2a$") || hashSenhaBanco.startsWith("$2b$")) {
                        senhaValida = BCrypt.checkpw(senhaDigitada, hashSenhaBanco);
                    } else {
                        // Compara direto se o banco ainda tiver senhas antigas não criptografadas
                        senhaValida = senhaDigitada.equals(hashSenhaBanco);
                    }

                    if (senhaValida) {
                        Usuario usuario = new Usuario();
                        usuario.setId(rs.getInt("id"));
                        usuario.setNome(rs.getString("nome"));
                        usuario.setEmail(rs.getString("email"));
                        usuario.setSenha(hashSenhaBanco);
                        usuario.setPerfil(rs.getString("perfil"));
                        usuario.setStatus(rs.getString("status"));
                        return usuario;
                    }
                }
            }
        } catch (SQLException e) {
            System.err.println("Erro ao autenticar usuário: " + e.getMessage());
        }
        return null;
    }
    /**
     * Método utilitário para cadastrar ou atualizar a senha de um usuário com hash BCrypt.
     */
    public boolean cadastrarUsuario(Usuario usuario) {
        String sql = "INSERT INTO usuarios (nome, email, senha, perfil, status) VALUES (?, ?, ?, ?, ?)";
        String senhaHash = BCrypt.hashpw(usuario.getSenha(), BCrypt.gensalt());
```

```

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, usuario.getNome());
        stmt.setString(2, usuario.getEmail());
        stmt.setString(3, senhaHash);
        stmt.setString(4, usuario.getPerfil());
        stmt.setString(5, usuario.getStatus());

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Erro ao cadastrar usuário: " + e.getMessage());
        return false;
    }
}
}

```

15.26 EmbarcacaoDAO.java

Realiza as operações SQL diretas na tabela embarcacoes: salvar (INSERT), atualizar (UPDATE), listarTodas (SELECT) e excluir (DELETE).

```

package dao;
import model.Embarcacao;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class EmbarcacaoDAO {

    public boolean salvar(Embarcacao emb) {
        String sql = "INSERT INTO embarcacoes (nome, modelo, capacidade_passageiros,
capacidade_carga_ton, ano_fabricacao, horimetro_horas, status) " +
            "VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, emb.getNome());
            stmt.setString(2, emb.getModelo());
            stmt.setInt(3, emb.getCapacidadePassageiros());
            stmt.setDouble(4, emb.getCapacidadeCargaTon());
            stmt.setInt(5, emb.getAnoFabricacao());
            stmt.setInt(6, emb.getHorimetroHoras());
            stmt.setString(7, emb.getStatus() != null ? emb.getStatus() : "ATIVA");
            return stmt.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean atualizar(Embarcacao emb) {
        String sql = "UPDATE embarcacoes SET nome = ?, modelo = ?, capacidade_passageiros = ?, "
+
            "capacidade_carga_ton = ?, ano_fabricacao = ?, horimetro_horas = ?, status = ?
WHERE id = ?";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, emb.getNome());
            stmt.setString(2, emb.getModelo());
            stmt.setInt(3, emb.getCapacidadePassageiros());

```

```

        stmt.setDouble(4, emb.getCapacidadeCargaTon());
        stmt.setInt(5, emb.getAnoFabricacao());
        stmt.setInt(6, emb.getHorimetroHoras());
        stmt.setString(7, emb.getStatus());
        stmt.setInt(8, emb.getId());

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public List<Embarcacao> listarTodas() {
    List<Embarcacao> lista = new ArrayList<>();
    String sql = "SELECT * FROM embarcacoes ORDER BY nome ASC";

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            Embarcacao emb = new Embarcacao();
            emb.setId(rs.getInt("id"));
            emb.setNome(rs.getString("nome"));
            emb.setModelo(rs.getString("modelo"));
            emb.setCapacidadePassageiros(rs.getInt("capacidade_passageiros"));
            emb.setCapacidadeCargaTon(rs.getDouble("capacidade_carga_ton"));
            emb.setAnoFabricacao(rs.getInt("ano_fabricacao"));
            emb.setHorimetroHoras(rs.getInt("horimetro_horas"));
            emb.setStatus(rs.getString("status"));
            lista.add(emb);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

public boolean excluir(int id) {
    String sql = "DELETE FROM embarcacoes WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, id);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}

```

15.27 ManutencaoDAO.java

salvar(...): Insere ordens de serviço com status padrão 'AGENDADA'.

listarTodasManutencoes(): Executa um JOIN entre as tabelas de manutenções e embarcações para montar a exibição em tabelas.

alterarStatus(...): Altera o estado do ciclo de vida da manutenção.

```
package dao;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ManutencaoDAO {

    public boolean salvar(int idEmbarcacao, String tipo, String descricao, Integer horimetro, Date
dataAgendada, double custoEstimado) {
        String sql = "INSERT INTO manutencoes (id_embarcacao, tipo_manutencao, descricao_servico,
horimetro_agendado, data_agendamento, custo_total, status) " +
            "VALUES (?, ?, ?, ?, ?, ?, 'AGENDADA)";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, idEmbarcacao);
            stmt.setString(2, tipo);
            stmt.setString(3, descricao);

            if (horimetro != null && horimetro > 0) {
                stmt.setInt(4, horimetro);
            } else {
                stmt.setNull(4, Types.INTEGER);
            }

            stmt.setDate(5, dataAgendada);
            stmt.setDouble(6, custoEstimado);

            return stmt.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public List<Object[]> listarTodasManutencoes() {
        List<Object[]> lista = new ArrayList<>();
        String sql = "SELECT m.id, e.nome AS embarcacao, m.tipo_manutencao, m.descricao_servico, "
+
            "m.horimetro_agendado, m.data_agendamento, m.custo_total, m.status " +
            "FROM manutencoes m " +
            "JOIN embarcacoes e ON m.id_embarcacao = e.id " +
            "ORDER BY m.data_agendamento DESC";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                lista.add(new Object[]{
                    rs.getInt("id"),
                    rs.getString("embarcacao"),
                    rs.getString("tipo_manutencao"),
                    rs.getString("descricao_servico"),
                    rs.getObject("horimetro_agendado") != null ? rs.getInt("horimetro_agendado") : "-",
                    rs.getDate("data_agendamento"),
                    rs.getDouble("custo_total"),
                    rs.getString("status")
                });
            }
        }
    }
}
```

```

    });
}
} catch (SQLException e) {
    e.printStackTrace();
}
return lista;
}

public boolean alterarStatus(int idManutencao, String novoStatus) {
    String sql = "UPDATE manutencoes SET status = ? WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, novoStatus);
        stmt.setInt(2, idManutencao);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}

```

15.28 TripulanteDAO.java

Oferece persistência completa (métodos salvar, atualizar, listarTodos, excluir) para os tripulantes e suas habilitações marítimas.

```

package dao;
import model.Tripulante;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class TripulanteDAO {

    public boolean salvar(Tripulante t) {
        String sql = "INSERT INTO tripulantes (nome, cpf, categoria_habilitacao, numero_registro_cir,
data_vencimento_cir, status) " +
            "VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setString(1, t.getNome());
            stmt.setString(2, t.getCpf());
            stmt.setString(3, t.getCategoriaHabilitacao());
            stmt.setString(4, t.getNumeroRegistroCir());
            stmt.setDate(5, t.getDataVencimentoCir());
            stmt.setString(6, t.getStatus() != null ? t.getStatus() : "DISPONIVEL");

            return stmt.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean atualizar(Tripulante t) {
        String sql = "UPDATE tripulantes SET nome = ?, cpf = ?, categoria_habilitacao = ?, " +
            "numero_registro_cir = ?, data_vencimento_cir = ?, status = ? WHERE id = ?";
    }
}

```

```

try (Connection conn = ConexaoDAO.obterConexao();
    PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setString(1, t.getNome());
    stmt.setString(2, t.getCpf());
    stmt.setString(3, t.getCategoriaHabilitacao());
    stmt.setString(4, t.getNumeroRegistroCir());
    stmt.setDate(5, t.getDataVencimentoCir());
    stmt.setString(6, t.getStatus());
    stmt.setInt(7, t.getId());

    return stmt.executeUpdate() > 0;
} catch (SQLException e) {
    e.printStackTrace();
    return false;
}
}

public List<Tripulante> listarTodos() {
    List<Tripulante> lista = new ArrayList<>();
    String sql = "SELECT * FROM tripulantes ORDER BY nome ASC";

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            Tripulante t = new Tripulante();
            t.setId(rs.getInt("id"));
            t.setNome(rs.getString("nome"));
            t.setCpf(rs.getString("cpf"));
            t.setCategoriaHabilitacao(rs.getString("categoria_habilitacao"));
            t.setNumeroRegistroCir(rs.getString("numero_registro_cir"));
            t.setDataVencimentoCir(rs.getDate("data_vencimento_cir"));
            t.setStatus(rs.getString("status"));
            lista.add(t);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

public boolean excluir(int id) {
    String sql = "DELETE FROM tripulantes WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, id);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}

```

15.29 ViagemDAO.java

salvar(...): Grava uma viagem convertendo LocalDateTime para Timestamp.

embarcacaoEmViagem(...) e comandanteEmViagem(...): Consultam se há impedimentos por viagens ativas.

finalizarViagem(...), cancelarViagem(...) e excluirViagem(...): Atualizam status e registram horários de chegada.

```
package dao;
import model.Viagem;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class ViagemDAO {
    public boolean salvar(Viagem v) {
        String sql = "INSERT INTO viagens (id_embarcacao, id_comandante, rota_destino,
data_hora_partida, quantidade_passageiros, status) " +
            "VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, v.getIdEmbarcacao());
            stmt.setInt(2, v.getIdComandante());
            stmt.setString(3, v.getRotaDestino());
            stmt.setTimestamp(4, Timestamp.valueOf(v.getDataHoraPartida()));
            stmt.setInt(5, v.getQuantidadePassageiros());
            stmt.setString(6, v.getStatus());

            return stmt.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public boolean embarcacaoEmViagem(int idEmbarcacao) {
        String sql = "SELECT COUNT(*) FROM viagens WHERE id_embarcacao = ? AND status =
'EM_ANDAMENTO'";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, idEmbarcacao);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return false;
    }

    public boolean comandanteEmViagem(int idComandante) {
        String sql = "SELECT COUNT(*) FROM viagens WHERE id_comandante = ? AND status =
'EM_ANDAMENTO'";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, idComandante);
            ResultSet rs = stmt.executeQuery();
            if (rs.next()) {
                return rs.getInt(1) > 0;
            }
        }
    }
}
```



```

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

public List<Viagem> listarTodas() {
    List<Viagem> lista = new ArrayList<>();
    String sql = "SELECT v.*, e.nome AS nome_embarcacao, t.nome AS nome_comandante " +
        "FROM viagens v " +
        "JOIN embarcacoes e ON v.id_embarcacao = e.id " +
        "JOIN tripulantes t ON v.id_comandante = t.id " +
        "ORDER BY v.data_hora_partida DESC";

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            Timestamp partida = rs.getTimestamp("data_hora_partida");
            Timestamp chegada = rs.getTimestamp("data_hora_chegada");

            lista.add(new Viagem(
                rs.getInt("id"),
                rs.getInt("id_embarcacao"),
                rs.getString("nome_embarcacao"),
                rs.getInt("id_comandante"),
                rs.getString("nome_comandante"),
                rs.getString("rota_destino"),
                partida != null ? partida.toLocalDateTime() : null,
                chegada != null ? chegada.toLocalDateTime() : null,
                rs.getInt("quantidade_passageiros"),
                rs.getString("status")
            ));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

public boolean finalizarViagem(int idViagem) {
    String sql = "UPDATE viagens SET status = 'CONCLUIDA', data_hora_chegada = ? WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setTimestamp(1, Timestamp.valueOf(LocalDateTime.now()));
        stmt.setInt(2, idViagem);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public boolean cancelarViagem(int idViagem) {
    String sql = "UPDATE viagens SET status = 'CANCELADA' WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, idViagem);
    }
}

```

```

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

// MÉTODO NOVO PARA EXCLUIR O REGISTRO DO BANCO
public boolean excluirViagem(int idViagem) {
    String sql = "DELETE FROM viagens WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, idViagem);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}

```

15.30 AbastecimentoDAO.java

Armazena os dados do abastecimento de combustível, lista os registros utilizando o DTO AbastecimentoDTO e calcula despesas acumuladas por período.

```

package dao;
import model.Abastecimento;
import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class AbastecimentoDAO {

    public boolean salvar(Abastecimento abastecimento) {
        String sql = "INSERT INTO abastecimentos (id_embarcacao, data_abastecimento, quantidade_litros, valor_total, fornecedor_posto) VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setInt(1, abastecimento.getIdEmbarcacao());
            stmt.setTimestamp(2, Timestamp.valueOf(abastecimento.getData().atStartOfDay()));
            stmt.setDouble(3, abastecimento.getLitros());
            stmt.setDouble(4, abastecimento.getValorTotal());
            stmt.setString(5, abastecimento.getFornecedor());
            return stmt.executeUpdate() > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }

    public List<AbastecimentoDTO> listarTodos() {
        List<AbastecimentoDTO> lista = new ArrayList<>();
        String sql = "SELECT a.id, e.nome AS embarcacao, a.data_abastecimento, a.quantidade_litros, a.valor_total, a.fornecedor_posto " +
            "FROM abastecimentos a " +
            "JOIN embarcacoes e ON a.id_embarcacao = e.id " +
            "ORDER BY a.data_abastecimento DESC";
        try (Connection conn = ConexaoDAO.obterConexao());
    }
}

```

```

        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery() {
        while (rs.next()) {
            lista.add(new AbastecimentoDTO(
                rs.getInt("id"),
                rs.getString("embarcacao"),
                rs.getTimestamp("data_abastecimento").toLocalDateTime().toLocalDate(),
                rs.getDouble("quantidade_litros"),
                rs.getDouble("valor_total"),
                rs.getString("fornecedor_posto")
            ));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

public double getTotalAbastecimentoPorEmbarcacao(int embarcacaoId, LocalDate inicio,
LocalDate fim) {
    String sql = "SELECT SUM(valor_total) FROM abastecimentos WHERE id_embarcacao = ? AND
data_abastecimento BETWEEN ? AND ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, embarcacaoId);
        stmt.setTimestamp(2, Timestamp.valueOf(inicio.atStartOfDay()));
        stmt.setTimestamp(3, Timestamp.valueOf(fim.atTime(23, 59, 59)));
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            return rs.getDouble(1);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0.0;
}

public boolean excluir(int idAbastecimento) {
    String sql = "DELETE FROM abastecimento WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, idAbastecimento);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

}

public static class AbastecimentoDTO {
    private final int id;
    private final String nomeEmbarcacao;
    private final LocalDate data;
    private final double litros;
    private final double valorTotal;
    private final String fornecedor;

    public AbastecimentoDTO(int id, String nomeEmbarcacao, LocalDate data, double litros, double
valorTotal, String fornecedor) {
        this.id = id;
        this.nomeEmbarcacao = nomeEmbarcacao;

```

```

        this.data = data;
        this.litros = litros;
        this.valorTotal = valorTotal;
        this.fornecedor = fornecedor;
    }

    public int getId() { return id; }
    public String getNomeEmbarcacao() { return nomeEmbarcacao; }
    public LocalDate getData() { return data; }
    public double getLitros() { return litros; }
    public double getValorTotal() { return valorTotal; }
    public String getFornecedor() { return fornecedor; }
}
}

```

15.31 RotaDAO.java

Executam consultas simples de leitura para popular seleções de rotas cadastradas.

```

package dao;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class RotaDAO {

    public List<String> listarTodas() {
        List<String> lista = new ArrayList<>();
        String sql = "SELECT nome FROM rotas ORDER BY nome ASC";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                lista.add(rs.getString("nome"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return lista;
    }
}

```

15.32 FornecedorDAO.java

Executam consultas simples de leitura para popular seleções de fornecedores

```

package dao;
import model.Fornecedor;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class FornecedorDAO {

    public List<Fornecedor> listarTodos() {
        List<Fornecedor> lista = new ArrayList<>();
        String sql = "SELECT * FROM fornecedores ORDER BY nome";
    }
}

```

```

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            lista.add(new Fornecedor(rs.getInt("id"), rs.getString("nome")));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}
}

```

15.33 ConsultaHorariosDAO.java

Executa a consulta SQL completa trazendo detalhes combinados de embarcação, comandante, rotas e horários para alimentar a classe ConsultaHorarioDTO.

```

package dao;
import dto.ConsultaHorarioDTO;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class ConsultaHorariosDAO {

    public List<ConsultaHorarioDTO> listarHorariosSaida() {
        List<ConsultaHorarioDTO> lista = new ArrayList<>();
        String sql = "SELECT v.id, e.nome AS embarcacao, t.nome AS comandante, v.rota_destino, " +
            "      v.data_hora_partida, v.data_hora_chegada, v.quantidade_passageiros, v.status " +
            "FROM viagens v " +
            "JOIN embarcacoes e ON v.id_embarcacao = e.id " +
            "JOIN tripulantes t ON v.id_comandante = t.id " +
            "ORDER BY v.data_hora_partida DESC";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                Timestamp tsChegada = rs.getTimestamp("data_hora_chegada");
                LocalDateTime chegada = tsChegada != null ? tsChegada.toLocalDateTime() : null;

                ConsultaHorarioDTO dto = new ConsultaHorarioDTO(
                    rs.getInt("id"),
                    rs.getString("embarcacao"),
                    rs.getString("comandante"),
                    rs.getString("rota_destino"),
                    rs.getTimestamp("data_hora_partida").toLocalDateTime(),
                    chegada,
                    rs.getInt("quantidade_passageiros"),
                    rs.getString("status")
                );
                lista.add(dto);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

        return lista;
    }

    public List<String> listarTodasEmbarcacoes() {
        List<String> lista = new ArrayList<>();
        String sql = "SELECT nome FROM embarcacoes ORDER BY nome";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                lista.add(rs.getString("nome"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return lista;
    }

    public List<String> listarTodosComandantes() {
        List<String> lista = new ArrayList<>();
        String sql = "SELECT nome FROM tripulantes ORDER BY nome";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                lista.add(rs.getString("nome"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return lista;
    }

    public List<String> listarTodosDestinos() {
        List<String> lista = new ArrayList<>();
        String sql = "SELECT DISTINCT rota_destino FROM viagens WHERE rota_destino IS NOT NULL AND rota_destino != '' ORDER BY rota_destino";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                lista.add(rs.getString("rota_destino"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return lista;
    }
}

```

15.34 RelatorioDAO.java

obterResumoCustosPorEmbarcacao(): Agrega no banco os custos totais provenientes de manutenções e abastecimentos via subconsultas SQL LEFT JOIN.

```

package dao;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

```

```

public class RelatorioDAO {

    public List<Object[]> obterResumoCustosPorEmbarcacao() {
        List<Object[]> lista = new ArrayList<>();
        String sql = "SELECT e.id, e.nome AS embarcacao, " +
            "COALESCE(m.total_manutencao, 0) AS total_manutencao, " +
            "COALESCE(a.total_abastecimento, 0) AS total_abastecimento, " +
            "(COALESCE(m.total_manutencao, 0) + COALESCE(a.total_abastecimento, 0)) AS
custo_total " +
            "FROM embarcacoes e " +
            "LEFT JOIN (SELECT id_embarcacao, SUM(custo_total) AS total_manutencao FROM
manutencoes WHERE status != 'CANCELADA' GROUP BY id_embarcacao) m ON e.id =
m.id_embarcacao " +
            "LEFT JOIN (SELECT id_embarcacao, SUM(valor_total) AS total_abastecimento FROM
abastecimentos GROUP BY id_embarcacao) a ON e.id = a.id_embarcacao " +
            "ORDER BY custo_total DESC";

        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                lista.add(new Object[]{
                    rs.getInt("id"),
                    rs.getString("embarcacao"),
                    rs.getDouble("total_manutencao"),
                    rs.getDouble("total_abastecimento"),
                    rs.getDouble("custo_total")
                });
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return lista;
    }
}

```

15.35 TecnicoDAO.java

concluirOS(...): Executa uma transação de banco de dados com suporte a Rollback (conn.setAutoCommit(false)), atualizando simultaneamente o custo/status da manutenção e o horímetro/status da embarcação.

listarAlertasHorimetro(): Retorna embarcações com horímetro próximo (diferença <= 50 horas) da manutenção preventiva agendada.

```

package dao;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class TecnicoDAO {

    public List<Object[]> listarOSAbertas() {
        List<Object[]> lista = new ArrayList<>();
        String sql = "SELECT m.id, e.nome AS embarcacao, m.tipo_manutencao, m.descricao_servico, "
+
            "m.data_agendamento, m.custo_total, m.status, m.id_embarcacao " +
            "FROM manutencoes m " +
            "JOIN embarcacoes e ON m.id_embarcacao = e.id " +
            "WHERE m.status IN ('AGENDADA', 'EM_ANDAMENTO') " +

```

```

        "ORDER BY m.data_agendamento ASC";

try (Connection conn = ConexaoDAO.obterConexao();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        lista.add(new Object[]{
            rs.getInt("id"),
            rs.getString("embarcacao"),
            rs.getString("tipo_manutencao"),
            rs.getString("descricao_servico"),
            rs.getDate("data_agendamento"),
            rs.getDouble("custo_total"),
            rs.getString("status"),
            rs.getInt("id_embarcacao")
        });
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return lista;
}

public boolean salvarNovaOS(int idEmbarcacao, String tipo, String descricao, Integer
horimetroAgendado, Date dataAgendamento) {
    String sql = "INSERT INTO manutencoes (id_embarcacao, tipo_manutencao, descricao_servico,
horimetro_agendado, data_agendamento, status) " +
        "VALUES (?, ?, ?, ?, ?, 'AGENDADA)";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, idEmbarcacao);
        stmt.setString(2, tipo);
        stmt.setString(3, descricao);
        if (horimetroAgendado != null && horimetroAgendado > 0) {
            stmt.setInt(4, horimetroAgendado);
        } else {
            stmt.setNull(4, Types.INTEGER);
        }
        stmt.setDate(5, dataAgendamento);

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public boolean concluirOS(int idManutencao, int idEmbarcacao, int novoHorimetro, double
custoFinal) {
    String sqlOS = "UPDATE manutencoes SET status = 'CONCLUIDA', custo_total = ?,
data_execucao = CURRENT_DATE WHERE id = ?";
    String sqlEmb = "UPDATE embarcacoes SET horimetro_horas = ?, status = 'ATIVA' WHERE id =
?";

    Connection conn = null;
    try {
        conn = ConexaoDAO.obterConexao();
        conn.setAutoCommit(false);
    }
}

```



```

try (PreparedStatement stmtOS = conn.prepareStatement(sqlOS);
    PreparedStatement stmtEmb = conn.prepareStatement(sqlEmb)) {

    stmtOS.setDouble(1, custoFinal);
    stmtOS.setInt(2, idManutencao);
    stmtOS.executeUpdate();

    stmtEmb.setInt(1, novoHorimetro);
    stmtEmb.setInt(2, idEmbarcacao);
    stmtEmb.executeUpdate();

    conn.commit();
    return true;
} catch (SQLException e) {
    conn.rollback();
    e.printStackTrace();
    return false;
}
} catch (SQLException e) {
    e.printStackTrace();
    return false;
}
}

public List<Object[]> listarAlertasHorimetro() {
    List<Object[]> lista = new ArrayList<>();
    String sql = "SELECT e.id, e.nome, e.horimetro_horas, m.horimetro_agendado,
m.descricao_servico " +
        "FROM embarcacoes e " +
        "JOIN manutencoes m ON e.id = m.id_embarcacao " +
        "WHERE m.status = 'AGENDADA' AND m.horimetro_agendado IS NOT NULL " +
        "AND e.horimetro_horas >= (m.horimetro_agendado - 50)";

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            lista.add(new Object[]{
                rs.getInt("id"),
                rs.getString("nome"),
                rs.getInt("horimetro_horas"),
                rs.getInt("horimetro_agendado"),
                rs.getString("descricao_servico")
            });
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

public List<Object[]> listarHistoricoCompleto() {
    List<Object[]> lista = new ArrayList<>();
    String sql = "SELECT m.id, e.nome, m.tipo_manutencao, m.descricao_servico,
m.data_execucao, m.custo_total " +
        "FROM manutencoes m " +
        "JOIN embarcacoes e ON m.id_embarcacao = e.id " +
        "WHERE m.status = 'CONCLUIDA' " +

```

```

        "ORDER BY m.data_execucao DESC";

try (Connection conn = ConexaoDAO.obterConexao();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        lista.add(new Object[]{
            rs.getInt("id"),
            rs.getString("nome"),
            rs.getString("tipo_manutencao"),
            rs.getString("descricao_servico"),
            rs.getDate("data_execucao"),
            rs.getDouble("custo_total")
        });
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return lista;
}

public List<Object[]> obterEmbarcacoesSimplificado() {
    List<Object[]> lista = new ArrayList<>();
    String sql = "SELECT id, nome FROM embarcacoes ORDER BY nome ASC";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            lista.add(new Object[]{rs.getInt("id"), rs.getString("nome")});
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}
}

```

15.36 IncidenteDAO.java

Gerencia os registros de problemas ou acidentes, permitindo vincular opcionalmente um ID de viagem (Types.INTEGER) e listar por status.

```

package dao;
import model.Incidente;
import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class IncidenteDAO {

    public boolean salvar(Incidente incidente) {
        String sql = "INSERT INTO incidentes (id_embarcacao, id_viagem, data_incidente, descricao,
        gravidade, status) VALUES (?, ?, ?, ?, ?, ?)";
        try (Connection conn = ConexaoDAO.obterConexao();
            PreparedStatement stmt = conn.prepareStatement(sql)) {

            stmt.setInt(1, incidente.getIdEmbarcacao());

```

```

        if (incidente.getIdViagem() != null && incidente.getIdViagem() > 0) {
            stmt.setInt(2, incidente.getIdViagem());
        } else {
            stmt.setNull(2, Types.INTEGER);
        }

        stmt.setTimestamp(3, Timestamp.valueOf(incidente.getDataIncidente()));
        stmt.setString(4, incidente.getDescricao());
        stmt.setString(5, incidente.getGravidade());
        stmt.setString(6, incidente.getStatus() != null ? incidente.getStatus() : "PENDENTE");

        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public List<Object[]> listarIncidentesPendentes() {
    List<Object[]> lista = new ArrayList<>();
    String sql = "SELECT i.id, e.nome AS embarcacao, i.data_incidente, i.descricao, i.gravidade,
i.status, i.id_embarcacao " +
        "FROM incidentes i " +
        "JOIN embarcacoes e ON i.id_embarcacao = e.id " +
        "WHERE i.status != 'RESOLVIDO' " +
        "ORDER BY i.data_incidente DESC";

    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            lista.add(new Object[]{
                rs.getInt("id"),
                rs.getString("embarcacao"),
                rs.getTimestamp("data_incidente"),
                rs.getString("descricao"),
                rs.getString("gravidade"),
                rs.getString("status"),
                rs.getInt("id_embarcacao")
            });
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return lista;
}

/**
 * Retorna todos os incidentes formatados em array de objetos pronto para preencher tabelas.
 */
public List<Object[]> listarTodosParaTabela() {
    List<Object[]> lista = new ArrayList<>();
    String sql = "SELECT i.id, e.nome AS embarcacao, i.data_incidente, i.descricao, i.gravidade,
i.status " +
        "FROM incidentes i " +
        "LEFT JOIN embarcacoes e ON i.id_embarcacao = e.id " +
        "ORDER BY i.id DESC";

```

```

try (Connection conn = ConexaoDAO.obterConexao();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        Timestamp ts = rs.getTimestamp("data_incidente");
        LocalDateTime dataHora = (ts != null) ? ts.toLocalDateTime() : null;

        lista.add(new Object[]{
            rs.getInt("id"),
            rs.getString("embarcacao") != null ? rs.getString("embarcacao") : "N/I",
            dataHora,
            rs.getString("descricao"),
            rs.getString("gravidade"),
            rs.getString("status") != null ? rs.getString("status") : "PENDENTE"
        });
    }
} catch (SQLException e) {
    e.printStackTrace();
}
return lista;
}

public boolean atualizarStatus(int idIncidente, String novoStatus) {
    String sql = "UPDATE incidentes SET status = ? WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, novoStatus);
        stmt.setInt(2, idIncidente);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public boolean excluir(int idIncidente) {
    String sql = "DELETE FROM incidentes WHERE id = ?";
    try (Connection conn = ConexaoDAO.obterConexao();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setInt(1, idIncidente);
        return stmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}
}

```

Camada Model (Entidades)

15.37 Usuário.java

Representa o usuário do sistema contendo ID, nome, e-mail, senha, perfil de acesso (ADMINISTRADOR, OPERADOR, TECNICO) e status.

package model;

```

/**
 * Classe de Entidade representando o Usuário do sistema.
 */
public class Usuario {
    private int id;
    private String nome;
    private String email;
    private String senha;
    private String perfil; // ADMINISTRADOR, OPERADOR, TECNICO
    private String status; // ATIVO, INATIVO

    public Usuario() {}

    public Usuario(int id, String nome, String email, String senha, String perfil, String status) {
        this.id = id;
        this.nome = nome;
        this.email = email;
        this.senha = senha;
        this.perfil = perfil;
        this.status = status;
    }
    // Getters e Setters (Encapsulamento)
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }

    public String getPerfil() { return perfil; }
    public void setPerfil(String perfil) { this.perfil = perfil; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

```

15.38 Embarcacao.java

Armazena as propriedades técnicas das embarcações, como nome, modelo, capacidade de passageiros/carga, horímetro atual e status.

```

package model;
public class Embarcacao {

    private int id;
    private String nome;
    private String modelo;
    private int capacidadePassageiros;
    private double capacidadeCargaTon;
    private int anoFabricacao;
    private int horimetroHoras;
    private String status; // ATIVA, EM_MANUTENCAO, INATIVA

    public Embarcacao() {}

    public Embarcacao(int id, String nome, String modelo, int capacidadePassageiros,

```

```

        double capacidadeCargaTon, int anoFabricacao, int horimetroHoras, String status) {
    this.id = id;
    this.nome = nome;
    this.modelo = modelo;
    this.capacidadePassageiros = capacidadePassageiros;
    this.capacidadeCargaTon = capacidadeCargaTon;
    this.anoFabricacao = anoFabricacao;
    this.horimetroHoras = horimetroHoras;
    this.status = status;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getNome() { return nome; }
public void setNome(String nome) { this.nome = nome; }

public String getModelo() { return modelo; }
public void setModelo(String modelo) { this.modelo = modelo; }

public int getCapacidadePassageiros() { return capacidadePassageiros; }
public void setCapacidadePassageiros(int capacidadePassageiros) { this.capacidadePassageiros =
capacidadePassageiros; }

public double getCapacidadeCargaTon() { return capacidadeCargaTon; }
public void setCapacidadeCargaTon(double capacidadeCargaTon) { this.capacidadeCargaTon =
capacidadeCargaTon; }

public int getAnoFabricacao() { return anoFabricacao; }
public void setAnoFabricacao(int anoFabricacao) { this.anoFabricacao = anoFabricacao; }

public int getHorimetroHoras() { return horimetroHoras; }
public void setHorimetroHoras(int horimetroHoras) { this.horimetroHoras = horimetroHoras; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

@Override
public String toString() {
    return nome;
}
}

```

15.39 Manutencao.java

Entidade de Ordem de Serviço com tipo, datas de agendamento e execução, valor final e controle de status.

```

package model;
import java.sql.Date;

public class Manutencao {

    private int id;
    private int idEmbarcacao;
    private String nomeEmbarcacao; // Exibição na tabela
    private String tipoManutencao; // PREVENTIVA, CORRETIVA
    private String descricaoServico;
    private Integer horimetroAgendado;
    private Date dataAgendamento;
    private Date dataExecucao;

```

```

private double custoTotal;
private String status; // AGENDADA, EM_ANDAMENTO, CONCLUIDA, CANCELADA

public Manutencao() {}

public Manutencao(int id, int idEmbarcacao, String nomeEmbarcacao, String tipoManutencao,
    String descricaoServico, Integer horimetroAgendado, Date dataAgendamento,
    Date dataExecucao, double custoTotal, String status) {
    this.id = id;
    this.idEmbarcacao = idEmbarcacao;
    this.nomeEmbarcacao = nomeEmbarcacao;
    this.tipoManutencao = tipoManutencao;
    this.descricaoServico = descricaoServico;
    this.horimetroAgendado = horimetroAgendado;
    this.dataAgendamento = dataAgendamento;
    this.dataExecucao = dataExecucao;
    this.custoTotal = custoTotal;
    this.status = status;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }

public int getIdEmbarcacao() { return idEmbarcacao; }
public void setIdEmbarcacao(int idEmbarcacao) { this.idEmbarcacao = idEmbarcacao; }

public String getNomeEmbarcacao() { return nomeEmbarcacao; }
public void setNomeEmbarcacao(String nomeEmbarcacao) { this.nomeEmbarcacao =
nomeEmbarcacao; }

public String getTipoManutencao() { return tipoManutencao; }
public void setTipoManutencao(String tipoManutencao) { this.tipoManutencao = tipoManutencao; }

public String getDescricaoServico() { return descricaoServico; }
public void setDescricaoServico(String descricaoServico) { this.descricaoServico =
descricaoServico; }

public Integer getHorimetroAgendado() { return horimetroAgendado; }
public void setHorimetroAgendado(Integer horimetroAgendado) { this.horimetroAgendado =
horimetroAgendado; }

public Date getDataAgendamento() { return dataAgendamento; }
public void setDataAgendamento(Date dataAgendamento) { this.dataAgendamento =
dataAgendamento; }

public Date getDataExecucao() { return dataExecucao; }
public void setDataExecucao(Date dataExecucao) { this.dataExecucao = dataExecucao; }

public double getCustoTotal() { return custoTotal; }
public void setCustoTotal(double custoTotal) { this.custoTotal = custoTotal; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }
}

```

15.40 Tripulante.java

Mapeia as informações cadastrais e de habilitação (código CIR, categoria e data de vencimento) do tripulante.

```

package model;
import java.sql.Date;

public class Tripulante {

    private int id;
    private String nome;
    private String cpf;
    private String categoriaHabilitacao;
    private String numeroRegistroCir;
    private Date dataVencimentoCir;
    private String status;

    public Tripulante() {}

    public Tripulante(int id, String nome, String cpf, String categoriaHabilitacao, String
numeroRegistroCir, Date dataVencimentoCir, String status) {
        this.id = id;
        this.nome = nome;
        this.cpf = cpf;
        this.categoriaHabilitacao = categoriaHabilitacao;
        this.numeroRegistroCir = numeroRegistroCir;
        this.dataVencimentoCir = dataVencimentoCir;
        this.status = status;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getCpf() { return cpf; }
    public void setCpf(String cpf) { this.cpf = cpf; }

    public String getCategoriaHabilitacao() { return categoriaHabilitacao; }
    public void setCategoriaHabilitacao(String categoriaHabilitacao) { this.categoriaHabilitacao =
categoriaHabilitacao; }

    public String getNumeroRegistroCir() { return numeroRegistroCir; }
    public void setNumeroRegistroCir(String numeroRegistroCir) { this.numeroRegistroCir =
numeroRegistroCir; }

    public Date getDataVencimentoCir() { return dataVencimentoCir; }
    public void setDataVencimentoCir(Date dataVencimentoCir) { this.dataVencimentoCir =
dataVencimentoCir; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    @Override
    public String toString() {
        return nome;
    }
}

```

15.41 Viagem.java

Guarda o vínculo entre embarcação e comandante, rota de destino, passageiros e datas de partida/chegada.

```
package model;
import java.time.LocalDateTime;

public class Viagem {
    private int id;
    private int idEmbarcacao;
    private String nomeEmbarcacao;
    private int idComandante;
    private String nomeComandante;
    private String rotaDestino;
    private LocalDateTime dataHoraPartida;
    private LocalDateTime dataHoraChegada;
    private int quantidadePassageiros;
    private String status;

    public Viagem() {}
    public Viagem(int idEmbarcacao, int idComandante, String rotaDestino,
        LocalDateTime dataHoraPartida, int quantidadePassageiros) {
        this.idEmbarcacao = idEmbarcacao;
        this.idComandante = idComandante;
        this.rotaDestino = rotaDestino;
        this.dataHoraPartida = dataHoraPartida;
        this.quantidadePassageiros = quantidadePassageiros;
        this.status = "EM_ANDAMENTO";
    }

    public Viagem(int id, int idEmbarcacao, String nomeEmbarcacao, int idComandante,
        String nomeComandante, String rotaDestino, LocalDateTime dataHoraPartida,
        LocalDateTime dataHoraChegada, int quantidadePassageiros, String status) {
        this.id = id;
        this.idEmbarcacao = idEmbarcacao;
        this.nomeEmbarcacao = nomeEmbarcacao;
        this.idComandante = idComandante;
        this.nomeComandante = nomeComandante;
        this.rotaDestino = rotaDestino;
        this.dataHoraPartida = dataHoraPartida;
        this.dataHoraChegada = dataHoraChegada;
        this.quantidadePassageiros = quantidadePassageiros;
        this.status = status;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public int getIdEmbarcacao() { return idEmbarcacao; }
    public String getNomeEmbarcacao() { return nomeEmbarcacao; }
    public int getIdComandante() { return idComandante; }
    public String getNomeComandante() { return nomeComandante; }
    public String getRotaDestino() { return rotaDestino; }
    public LocalDateTime getDataHoraPartida() { return dataHoraPartida; }
    public LocalDateTime getDataHoraChegada() { return dataHoraChegada; }
    public int getQuantidadePassageiros() { return quantidadePassageiros; }
    public String getStatus() { return status; }
}
```

15.42 Abastecimento.java

Registra a quantidade de litros, valor cobrado, fornecedor e data do reabastecimento.

```

package model;
import java.time.LocalDate;

public class Abastecimento {
    private int id;
    private int embarcacaoid;
    private LocalDate data;
    private double litros;
    private double valorTotal;
    private String fornecedor;

    public Abastecimento() {}

    public Abastecimento(int embarcacaoid, LocalDate data, double litros, double valorTotal, String
fornecedor) {
        this.embarcacaoid = embarcacaoid;
        this.data = data;
        this.litros = litros;
        this.valorTotal = valorTotal;
        this.fornecedor = fornecedor;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getEmbarcacaoid() { return embarcacaoid; }
    public void setEmbarcacaoid(int embarcacaoid) { this.embarcacaoid = embarcacaoid; }

    public LocalDate getData() { return data; }
    public void setData(LocalDate data) { this.data = data; }

    public double getLitros() { return litros; }
    public void setLitros(double litros) { this.litros = litros; }

    public double getValorTotal() { return valorTotal; }
    public void setValorTotal(double valorTotal) { this.valorTotal = valorTotal; }

    public String getFornecedor() { return fornecedor; }
    public void setFornecedor(String fornecedor) { this.fornecedor = fornecedor; }
}

```

15.43 Incidente.java

Representa ocorrências registradas no diário de bordo com nível de gravidade e vínculo com embarcação/viagem.

```

package model;
import java.time.LocalDateTime;

public class Incidente {
    private int id;
    private int idEmbarcacao;
    private Integer idViagem;
    private LocalDateTime dataIncidente;
    private String descricao;
    private String gravidade;
    private String status;

    public Incidente() {}

```

```

    public Incidente(int idEmbarcacao, Integer idViagem, LocalDateTime dataIncidente, String
descricao, String gravidade) {
        this.idEmbarcacao = idEmbarcacao;
        this.idViagem = idViagem;
        this.dataIncidente = dataIncidente;
        this.descricao = descricao;
        this.gravidade = gravidade;
        this.status = "PENDENTE";
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public int getIdEmbarcacao() { return idEmbarcacao; }
    public void setIdEmbarcacao(int idEmbarcacao) { this.idEmbarcacao = idEmbarcacao; }

    public Integer getIdViagem() { return idViagem; }
    public void setIdViagem(Integer idViagem) { this.idViagem = idViagem; }

    public LocalDateTime getDataIncidente() { return dataIncidente; }
    public void setDataIncidente(LocalDateTime dataIncidente) { this.dataIncidente = dataIncidente; }

    public String getDescricao() { return descricao; }
    public void setDescricao(String descricao) { this.descricao = descricao; }

    public String getGravidade() { return gravidade; }
    public void setGravidade(String gravidade) { this.gravidade = gravidade; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

```

Camada DTO(Data Transfer Objects)

Transporta dados otimizados entre as camadas sem expor todas as entidades.

15.44 ConsultaHorarioDTO.java

```

package dto;
import java.time.LocalDateTime;

public class ConsultaHorarioDTO {
    private int idViagem;
    private String embarcacao;
    private String comandante;
    private String rotaDestino;
    private LocalDateTime dataHoraPartida;
    private LocalDateTime dataHoraChegada;
    private int quantidadePassageiros;
    private String status;

    public ConsultaHorarioDTO(int idViagem, String embarcacao, String comandante, String
rotaDestino,
        LocalDateTime dataHoraPartida, LocalDateTime dataHoraChegada,
        int quantidadePassageiros, String status) {
        this.idViagem = idViagem;
        this.embarcacao = embarcacao;
    }
}

```

```

        this.comandante = comandante;
        this.rotaDestino = rotaDestino;
        this.dataHoraPartida = dataHoraPartida;
        this.dataHoraChegada = dataHoraChegada;
        this.quantidadePassageiros = quantidadePassageiros;
        this.status = status;
    }

    public int getIdViagem() { return idViagem; }
    public String getEmbarcacao() { return embarcacao; }
    public String getComandante() { return comandante; }
    public String getRotaDestino() { return rotaDestino; }
    public LocalDateTime getDataHoraPartida() { return dataHoraPartida; }
    public LocalDateTime getDataHoraChegada() { return dataHoraChegada; }
    public int getQuantidadePassageiros() { return quantidadePassageiros; }
    public String getStatus() { return status; }
}

```

15.45 CustoConsolidadoDTO.java

```

package dto;

public class CustoConsolidadoDTO {
    private String nomeEmbarcacao;
    private double totalManutencao;
    private double totalAbastecimento;
    private double totalGeral;

    public CustoConsolidadoDTO(String nomeEmbarcacao, double totalManutencao, double
totalAbastecimento) {
        this.nomeEmbarcacao = nomeEmbarcacao;
        this.totalManutencao = totalManutencao;
        this.totalAbastecimento = totalAbastecimento;
        this.totalGeral = totalManutencao + totalAbastecimento;
    }

    public String getNomeEmbarcacao() { return nomeEmbarcacao; }
    public double getTotalManutencao() { return totalManutencao; }
    public double getTotalAbastecimento() { return totalAbastecimento; }
    public double getTotalGeral() { return totalGeral; }
}

```

Service

Módulos responsáveis por rotinas transversais como envio de notificações via e-mail (EmailService), validação avançada de documentos e exportação de PDF (PDFService).

15.46 EmailService.java

```

package service;
import javax.mail.*;
import javax.mail.internet.*;
import java.io.File;
import java.util.Properties;

public class EmailService {

```

```

// Preencha com seu e-mail real e a Senha de App de 16 dígitos gerada no Google
private static final String REMETENTE = "seu.email.real@gmail.com";
private static final String SENHA_APP = "xxxx xxxx xxxx xxxx";

public static boolean enviarRelatorioPorEmail(String destinatario, String caminhoAnexo) {
    Properties props = new Properties();
    props.put("mail.smtp.host", "smtp.gmail.com");
    props.put("mail.smtp.port", "587");
    props.put("mail.smtp.auth", "true");
    props.put("mail.smtp.starttls.enable", "true");
    props.put("mail.smtp.ssl.protocols", "TLSv1.2");
    props.put("mail.smtp.ssl.trust", "smtp.gmail.com");

    Session session = Session.getInstance(props, new Authenticator() {
        @Override
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication(REMETENTE, SENHA_APP);
        }
    });

    try {
        Message message = new MimeMessage(session);
        message.setFrom(new InternetAddress(REMETENTE, "Gestão Náutica"));
        message.setRecipients(Message.RecipientType.TO, InternetAddress.parse(destinatario));
        message.setSubject("Relatório de Custos Operacionais - Gestão Náutica");

        BodyPart messageBodyPart = new MimeBodyPart();
        messageBodyPart.setText("Segue em anexo o relatório atualizado de custos da frota.");

        Multipart multipart = new MimeMultipart();
        multipart.addBodyPart(messageBodyPart);

        // Anexo em PDF
        if (caminhoAnexo != null && !caminhoAnexo.isEmpty()) {
            File arquivo = new File(caminhoAnexo);
            if (arquivo.exists()) {
                MimeBodyPart attachmentPart = new MimeBodyPart();
                attachmentPart.attachFile(arquivo);
                multipart.addBodyPart(attachmentPart);
            }
        }

        message.setContent(multipart);
        Transport.send(message);
        return true;
    } catch (Exception e) {
        System.err.println("Erro ao enviar e-mail SMTP: " + e.getMessage());
        e.printStackTrace();
        return false;
    }
}
}

```

15.47 GeradorPDFService.java

```

package service;
import com.lowagie.text.*;
import com.lowagie.text.pdf.*;

```

```

import java.awt.Color;
import java.io.File;
import java.io.FileOutputStream;
import java.text.NumberFormat;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class GeradorPDFService {

    public static String gerarRelatorioCustos(List<Object[]> dadosCustos) throws Exception {
        String userHome = System.getProperty("user.home");
        String caminhoDownloads = userHome + File.separator + "Downloads" + File.separator +
"Relatorio_Custos_Embarcacoes.pdf";

        Document document = new Document(PageSize.A4, 36, 36, 36, 36);
        PdfWriter.getInstance(document, new FileOutputStream(caminhoDownloads));

        document.open();

        // Cabeçalho do Documento
        Font fontTitulo = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 18, Color.BLACK);
        Font fontSubtitulo = FontFactory.getFont(FontFactory.HELVETICA, 10, Color.GRAY);
        Font fontHeaderTabela = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 10,
Color.WHITE);
        Font fontDados = FontFactory.getFont(FontFactory.HELVETICA, 10, Color.BLACK);

        Paragraph pTitulo = new Paragraph("GESTÃO NÁUTICA - RELATÓRIO DE CUSTOS",
fontTitulo);
        pTitulo.setAlignment(Element.ALIGN_CENTER);
        document.add(pTitulo);

        String dataAtual = new SimpleDateFormat("dd/MM/yyyy HH:mm").format(new Date());
        Paragraph pSub = new Paragraph("Gerado em: " + dataAtual + " | Módulo Administrador",
fontSubtitulo);
        pSub.setAlignment(Element.ALIGN_CENTER);
        pSub.setSpacingAfter(20);
        document.add(pSub);

        // Tabela de Custos
        PdfPTable table = new PdfPTable(4);
        table.setWidthPercentage(100);
        table.setWidths(new float[]{3f, 2f, 2f, 2f});

        String[] headers = {"Embarcação", "Manutenção (R$)", "Combustível (R$)", "Custo Total (R$)"};
        for (String header : headers) {
            PdfPCell cell = new PdfPCell(new Phrase(header, fontHeaderTabela));
            cell.setBackgroundColor(new Color(41, 128, 185));
            cell.setHorizontalAlignment(Element.ALIGN_CENTER);
            cell.setPadding(6);
            table.addCell(cell);
        }

        NumberFormat currencyFormat =
NumberFormat.getCurrencyInstance(Locale.forLanguageTag("pt-BR"));
        double geralTotal = 0;

        for (Object[] row : dadosCustos) {

```

```
double maint = (double) row[2];
double fuel = (double) row[3];
double total = (double) row[4];

geralTotal += total;

table.addCell(new Phrase((String) row[1], fontDados));
table.addCell(new Phrase(currencyFormat.format(maint), fontDados));
table.addCell(new Phrase(currencyFormat.format(fuel), fontDados));
table.addCell(new Phrase(currencyFormat.format(total), fontDados));
}

document.add(table);

// Resumo Geral
Paragraph pResumo = new Paragraph("\nTOTAL GERAL OPERACIONAL: " +
currencyFormat.format(geralTotal), FontFactory.getFont(FontFactory.HELVETICA_BOLD, 12,
Color.DARK_GRAY));
pResumo.setAlignment(Element.ALIGN_RIGHT);
document.add(pResumo);

document.close();
return caminhoDownloads;
}
}
```

APÊNDICE A – REGISTRO DE OBSERVAÇÕES DE ATIVIDADE PRÁTICA

DADOS DOS ESTUDANTES	
Estudante 1: Letícia Alves Salvador Turma: 2025/1	
Estudante 2: Magayver Kaon do Nascimento Silva Turma: 2025/1	
Estudante 3: Vitória Monteiro de Carvalho Turma: 2025/1	
Estudante 4: Yosef Klinsman Moreira Cruz Turma: 2025/1	
Unidade Escolar/Município: CETAM Galiléia, Manaus Curso: Técnico de Nível Médio em Desenvolvimento de Sistemas	
Componente Curricular: Prática Profissional Supervisionada(PPS)	
Instrutor Responsável: Priscila Leylianne da Silva Gonçalves	

LOCAL DE REALIZAÇÃO DA ATIVIDADE PRÁTICA	
Local:	
_____ Setor envolvido (visitado/observado):	

[illegible]

Data: ____/____/____. 1. _____ 2. _____ 3. _____ Assinatura dos Estudantes.	Data: ____/____/____. _____ Assinatura do Instrutor.
--	--